

```

indicator("Daily bars", behind_chart = false, overlay = true)

// Use gaps to return data only when the 1D timeframe completes, and to return `na` otherwise.
[o, h, l, c] = request.security(syminfo.tickerid, "D", [open, high, low, close], gaps = barmerge.gaps_on)

const color UP_COLOR = color.silver
const color DN_COLOR = color.blue
color wickColor = c >= o ? UP_COLOR : DN_COLOR
color bodyColor = c >= o ? color.new(UP_COLOR, 70) : color.new(DN_COLOR, 70)
// Plot candles on intraday timeframes,
// and when non `na` values are returned by `request.security()` because a HTF bar has completed.
plotcandle(timeframe.isintraday ? o : na, h, l, c, color = bodyColor, wickcolor = wickColor)

```



Figure 102: image

Note that:

- We set the `behind_chart` parameter of the indicator declaration to `false`. This causes our script's candles to appear on top of the chart's candles. Selecting "Visual Order/Bring to Front" from the script's "More" menu achieves the same result.
- The script displays candles only when two conditions are met:
 - The chart is using an intraday timeframe (see the check on `timeframe.isintraday` in the `plotcandle()` call). We do this because it's not useful to show a daily value on timeframes higher or equal to 1D.
 - The `request.security()` function returns non `na` values (see `gaps = barmerge.gaps_on` in the function call).
- We use a tuple (`[open, high, low, close]`) with `request.security()` to fetch four values in one call.
- We create a lighter transparency for the body of our candles in the `bodyColor` variable initialization, so they don't obstruct the chart's candles.

Plotting bars with `plotbar()`

The signature of `plotbar()` is:

```
plotbar(open, high, low, close, title, color, editable, show_last, display, force_overlay) → void
```

Note that `plotbar()` has no parameter for `bordercolor` or `wickcolor`, as there are no borders or wicks on conventional bars.

This plots conventional bars using the same coloring logic as in the second example of the previous section:

```

//@version=6
indicator("Dual-color bars")

```

```
paletteColor = close >= open ? color.lime : color.red
plotbar(open, high, low, close, color = paletteColor)
```



Figure 103: image

[Previous

Bar coloring](#bar-coloring)[Next

Bar states](#bar-states) User Manual/Concepts/Bar states

Bar states

Introduction

A set of built-in variables in the `barstate` namespace allow your script to detect different properties of the bar on which the script is currently executing.

These states can be used to restrict the execution or the logic of your code to specific bars.

Some built-ins return information on the trading session the current bar belongs to. They are explained in the Session states section.

Bar state built-in variables

Note that while indicators and libraries run on all price or volume updates in real time, strategies not using `calc_on_every_tick` will not; they will only execute when the realtime bar closes. This will affect the detection of bar states in that type of script. On open markets, for example, this code will not display a background until the realtime closes because that is when the strategy runs:

```
//@version=6
strategy("S")
bgcolor(barstate.islast ? color.silver : na)
```

`barstate.isfirst`

`barstate.isfirst` is only `true` on the dataset's first bar, i.e., when `bar_index` is zero.

It can be useful to initialize variables on the first bar only, e.g.:

```
// Declare array and set its values on the first bar only.
FILL_COLOR = color.green
var fillColors = array.new_color(0)
if barstate.isfirst
    // Initialize the array elements with progressively lighter shades of the fill color.
    array.push(fillColors, color.new(FILL_COLOR, 70))
    array.push(fillColors, color.new(FILL_COLOR, 75))
```

```

array.push(fillColors, color.new(FILL_COLOR, 80))
array.push(fillColors, color.new(FILL_COLOR, 85))
array.push(fillColors, color.new(FILL_COLOR, 90))

```

barstate.islast

`barstate.islast` is **true** if the current bar is the last one on the chart, whether that bar is a realtime bar or not.

It can be used to restrict the execution of code to the chart's last bar, which is often useful when drawing lines, labels or tables. Here, we use it to determine when to update a label which we want to appear only on the last bar. We create the label only once and then update its properties using `label.set_*()` functions because it is more efficient:

```

//@version=6
indicator("", "", true)
// Create label on the first bar only.
var label hiLabel = label.new(na, na, "")
// Update the label's position and text on the last bar,
// including on all realtime bar updates.
if barstate.islast
    label.set_xy(hiLabel, bar_index, high)
    label.set_text(hiLabel, str.tostring(high, format.mintick))

```

barstate.ishistory

`barstate.ishistory` is **true** on all historical bars. It can never be **true** on a bar when `barstate.isrealtime` is also **true**, and it does not become **true** on a realtime bar's closing update, when `barstate.isconfirmed` becomes **true**. On closed markets, it can be **true** on the same bar where `barstate.islast` is also **true**.

barstate.isrealtime

`barstate.isrealtime` is **true** if the current data update is a real-time bar update, **false** otherwise (thus it is historical). Note that `barstate.islast` is also **true** on all realtime bars.

barstate.isnew

`barstate.isnew` is **true** on all historical bars and on the realtime bar's first (opening) update.

All historical bars are considered *new* bars because the Pine Script™ runtime executes your script on each bar sequentially, from the chart's first bar in time, to the last. Each historical bar is thus *discovered* by your script as it executes, bar to bar.

`barstate.isnew` can be useful to reset varip variables when a new realtime bar comes in. The following code will reset `updateNo` to 1 on all historical bars and at the beginning of each realtime bar. It calculates the number of realtime updates during each realtime bar:

```

//@version=6
indicator("")
updateNo() =>
    varip int updateNo = na
    if barstate.isnew
        updateNo := 1
    else
        updateNo += 1
plot(updateNo())

```

barstate.isconfirmed

`barstate.isconfirmed` is **true** on all historical bars and on the last (closing) update of a realtime bar.

It can be useful to avoid repainting by requiring the realtime bar to be closed before a condition can become **true**. We use it here to hold plotting of our RSI until the realtime bar closes and becomes an elapsed realtime bar. It will plot on historical bars because `barstate.isconfirmed` is always **true** on them:

```

//@version=6
indicator("")
myRSI = ta.rsi(close, 20)

```

```
plot(barstate.isconfirmed ? myRSI : na)
```

`barstate.isconfirmed` will not work when used in a `request.security()` call.

`barstate.islastconfirmedhistory`

`barstate.islastconfirmedhistory` is `true` if the script is executing on the dataset's last bar when the market is closed, or on the bar immediately preceding the realtime bar if the market is open.

It can be used to detect the first realtime bar with `barstate.islastconfirmedhistory[1]`, or to postpone server-intensive calculations until the last historical bar, which would otherwise be undetectable on open markets.

Example

Here is an example of a script using `barstate.*` variables:

```
//@version=6
indicator("Bar States", overlay = true, max_labels_count = 500)

stateText() =>
    string txt = ""
    txt += barstate.isfirst      ? "isfirst\n"      : ""
    txt += barstate.islast      ? "islast\n"        : ""
    txt += barstate.ishistory    ? "ishistory\n"     : ""
    txt += barstate.isrealtime   ? "isrealtime\n"    : ""
    txt += barstate.isnew        ? "isnew\n"         : ""
    txt += barstate.isconfirmed  ? "isconfirmed\n"   : ""
    txt += barstate.islastconfirmedhistory ? "islastconfirmedhistory\n" : ""

labelColor = switch
    barstate.isfirst              => color.fuchsia
    barstate.islastconfirmedhistory => color.gray
    barstate.ishistory            => color.silver
    barstate.isconfirmed          => color.orange
    barstate.isnew                => color.red
    => color.yellow

label.new(bar_index, na, stateText(), yloc = yloc.abovebar, color = labelColor)
```

Note that:

- Each state's name will appear in the label's text when it is `true`.
- There are five possible colors for the label's background:
 - fuchsia on the first bar
 - silver on historical bars
 - gray on the last confirmed historical bar
 - orange when a realtime bar is confirmed (when it closes and becomes an elapsed realtime bar)
 - red on the realtime bar's first execution
 - yellow for other executions of the realtime bar

We begin by adding the indicator to the chart of an open market, but before any realtime update is received. Note how the last confirmed history bar is identified in `#1`, and how the last bar is identified as the last one, but is still considered a historical bar because no realtime updates have been received.

Let's look at what happens when realtime updates start coming in:

Note that:

- The realtime bar is red because it is its first execution, because `barstate.isnew` is `true` and `barstate.ishistory` is no longer `true`, so our switch structure determining our color uses the `barstate.isnew => color.red` branch. This will usually not last long because on the next update `barstate.isnew` will no longer be `true` so the label's color will turn yellow.
- The label of elapsed realtime bars is orange because those bars were not historical bars when they closed. Accordingly, the `barstate.ishistory => color.silver` branch in the switch structure was not executed, but the next one, `barstate.isconfirmed => color.orange` was.

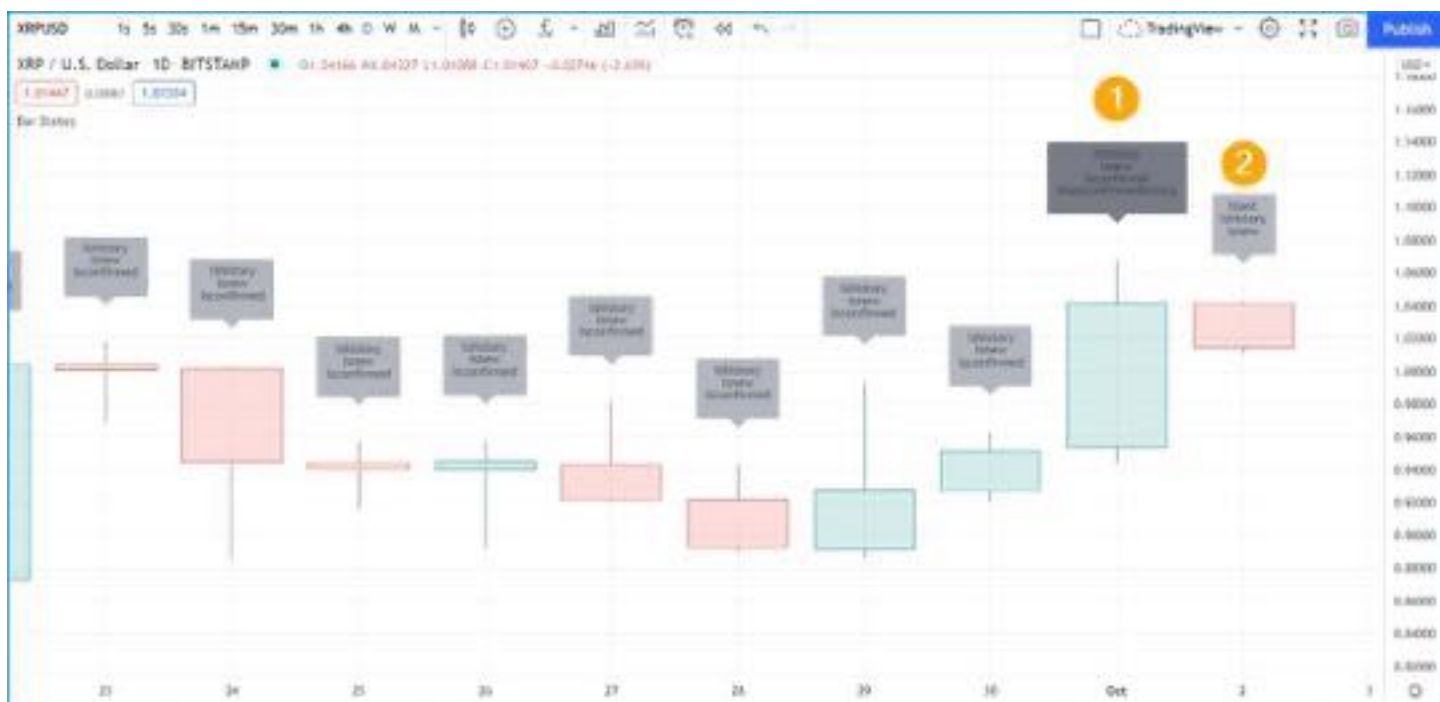


Figure 104: image

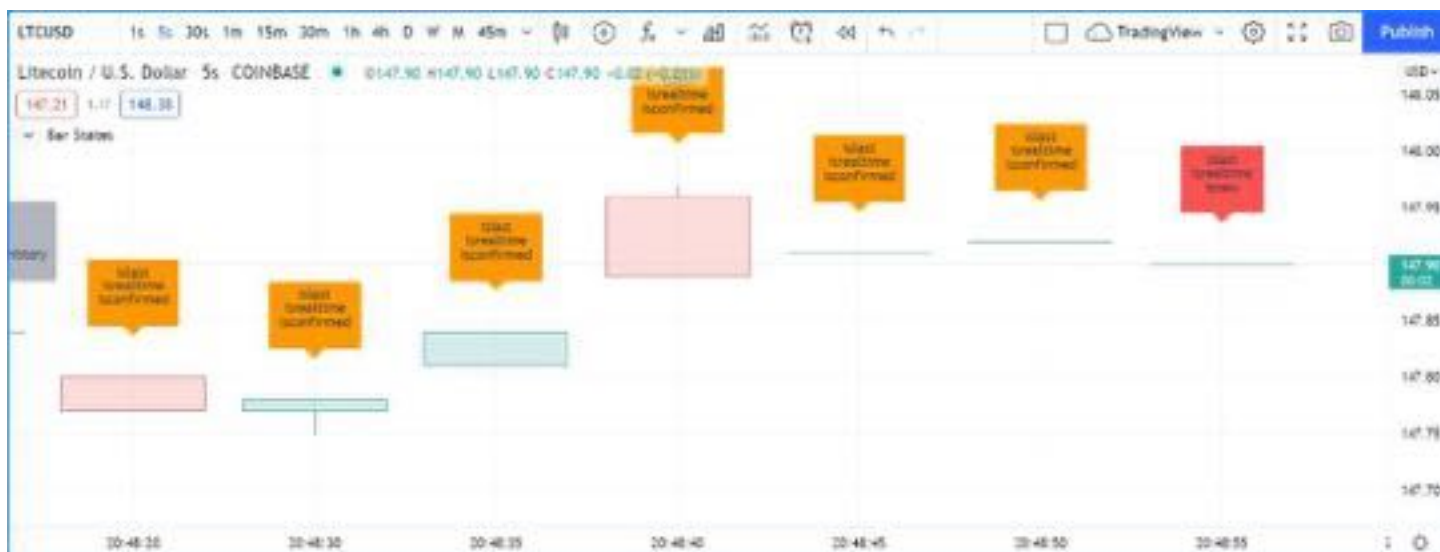


Figure 105: image

This last example shows how the realtime bar's label will turn yellow after the first execution on the bar. This is the way the label will usually appear on realtime bars:

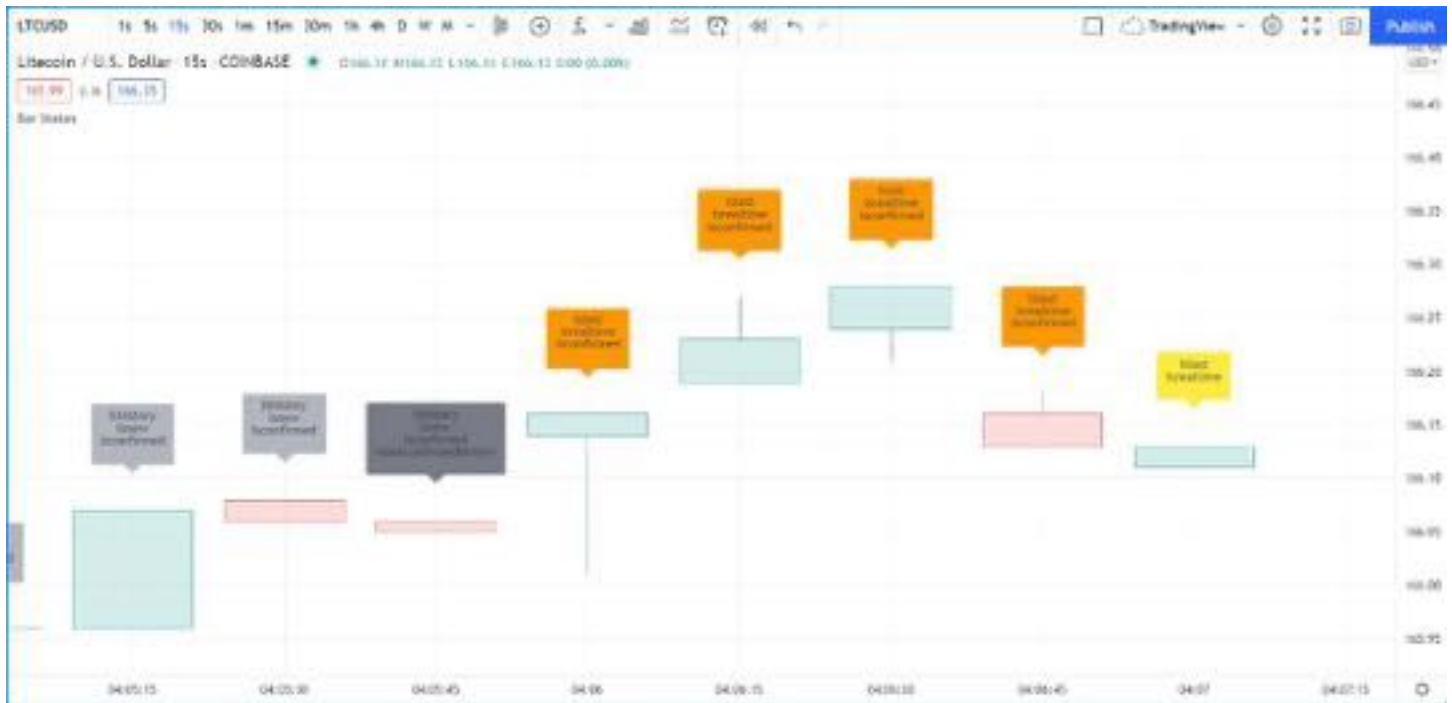


Figure 106: image

[Previous

Bar plotting](#bar-plotting)[Next

Chart information](#chart-information) User Manual/Concepts/Chart information

Chart information

Introduction

The way scripts can obtain information about the chart and symbol they are currently running on is through a subset of Pine Script™'s built-in variables. The ones we cover here allow scripts to access information relating to:

- The chart's prices and volume
- The chart's symbol
- The chart's timeframe
- The session (or time period) the symbol trades on

Prices and volume

The built-in variables for OHLCV values are:

- open: the bar's opening price.
- high: the bar's highest price, or the highest price reached during the realtime bar's elapsed time.
- low: the bar's lowest price, or the lowest price reached during the realtime bar's elapsed time.
- close: the bar's closing price, or the **current price** in the realtime bar.
- volume: the volume traded during the bar, or the volume traded during the realtime bar's elapsed time. The unit of volume information varies with the instrument. It is in shares for stocks, in lots for forex, in contracts for futures, in the base currency for crypto, etc.

Other values are available through:

- hl2: the average of the bar's high and low values.
- hlc3: the average of the bar's high, low and close values.

- ohlc4: the average of the bar's open, high, low and close values.

On historical bars, the values of the above variables do not vary during the bar because only OHLCV information is available on them. When running on historical bars, scripts execute on the bar's close, when all the bar's information is known and cannot change during the script's execution on the bar.

Realtime bars are another story altogether. When indicators (or strategies using `calc_on_every_tick = true`) run in realtime, the values of the above variables (except open) will vary between successive iterations of the script on the realtime bar, because they represent their **current** value at one point in time during the progress of the realtime bar. This may lead to one form of repainting. See the page on Pine Script™'s execution model for more details.

The `[]` history-referencing operator can be used to refer to past values of the built-in variables, e.g., `close[1]` refers to the value of close on the previous bar, relative to the particular bar the script is executing on.

Symbol information

Built-in variables in the `syminfo` namespace provide scripts with information on the symbol of the chart the script is running on. This information changes every time a script user changes the chart's symbol. The script then re-executes on all the chart's bars using the new values of the built-in variables:

- `syminfo.basecurrency`: the base currency, e.g., "BTC" in "BTCUSD", or "EUR" in "EURUSD".
- `syminfo.currency`: the quote currency, e.g., "USD" in "BTCUSD", or "CAD" in "USDCAD".
- `syminfo.description`: The long description of the symbol.
- `syminfo.main_tickerid`: The symbol's *main* ticker identifier. It behaves almost identically to `syminfo.tickerid`, referencing the symbol's exchange prefix, name, and additional ticker data. However, this variable *always* represents the *current* chart's ticker ID, even within requested contexts.
- `syminfo.mincontract`: The symbol's smallest tradable amount, which is set by its exchange. For example, the minimum for NASDAQ asset "AAPL" is 1 token, while the minimum for BITSTAMP cryptocurrency "ETHUSD" is 0.0001 tokens.
- `syminfo.mintick`: The symbol's tick value, or the minimum increment price can move in. Not to be confused with *pips* or *points*. On "ES1!" ("S&P 500 E-Mini") the tick size is 0.25 because that is the minimal increment the price moves in.
- `syminfo.pointvalue`: The point value is the multiple of the underlying asset determining a contract's value. On "ES1!" ("S&P 500 E-Mini") the point value is 50, so a contract is worth 50 times the price of the instrument.
- `syminfo.prefix`: The prefix is the exchange or broker's identifier: "NASDAQ" or "BATS" for "AAPL", "CME_MINI_DL" for "ES1!".
- `syminfo.root`: It is the ticker's prefix for structured tickers like those of futures. It is "ES" for "ES1!", "ZW" for "ZW1!".
- `syminfo.session`: It reflects the session setting on the chart for that symbol. If the "Chart settings/Symbol/Session" field is set to "Extended", it will only return "extended" if the symbol and the user's feed allow for extended sessions. It is rarely displayed and used mostly as an argument to the `session` parameter in `ticker.new()`.
- `syminfo.ticker`: It is the symbol's name, without the exchange part (`syminfo.prefix`): "BTCUSD", "AAPL", "ES1!", "USDCAD".
- `syminfo.tickerid`: The symbol's ticker identifier, consisting of its exchange prefix and symbol name, e.g., "NASDAQ:MSFT". It can also include ticker information beyond the "prefix:ticker" form, such as extended hours, dividend adjustments, currency conversion, etc. To retrieve the standard "prefix:ticker" form only, pass the variable to `ticker.standard()`. When used in a `request.*()` call's `expression` argument, this variable references the *requested* context's ticker ID. Otherwise, it references the current chart's ticker ID.
- `syminfo.timezone`: The timezone the symbol is traded in. The string is an IANA time zone database name (e.g., "America/New_York").
- `syminfo.type`: The type of market the symbol belongs to. The values are "stock", "futures", "index", "forex", "crypto", "fund", "dr", "cfd", "bond", "warrant", "structured" and "right".

This script displays these built-in variables and their values for the current symbol in a table on the chart:

```
//@version=6
indicator("`syminfo.*` built-ins demo", overlay = true)

//@variable The `syminfo.*` built-ins, displayed in the left column of the table.
string txtLeft =
    "syminfo.basecurrency: " + "\n" +
    "syminfo.currency: " + "\n" +
    "syminfo.description: " + "\n" +
    "syminfo.main_tickerid: " + "\n" +
```



Figure 107: image

```

"syminfo.mincontract: " + "\n" +
"syminfo.mintick: " + "\n" +
"syminfo.pointvalue: " + "\n" +
"syminfo.prefix: " + "\n" +
"syminfo.root: " + "\n" +
"syminfo.session: " + "\n" +
"syminfo.ticker: " + "\n" +
"syminfo.tickerid: " + "\n" +
"syminfo.timezone: " + "\n" +
"syminfo.type: "

//@variable The values of the `syminfo.*` built-ins, displayed in the right column of the table.
string txtRight =
    syminfo.basecurrency + "\n" +
    syminfo.currency + "\n" +
    syminfo.description + "\n" +
    syminfo.main_tickerid + "\n" +
    str.toString(syminfo.mincontract) + "\n" +
    str.toString(syminfo.mintick) + "\n" +
    str.toString(syminfo.pointvalue) + "\n" +
    syminfo.prefix + "\n" +
    syminfo.root + "\n" +
    syminfo.session + "\n" +
    syminfo.ticker + "\n" +
    syminfo.tickerid + "\n" +
    syminfo.timezone + "\n" +
    syminfo.type

if barstate.islast
    var table t = table.new(position.middle_right, 2, 1)
    table.cell(t, 0, 0, txtLeft, bgcolor = color.yellow, text_halign = text.align_right)
    table.cell(t, 1, 0, txtRight, bgcolor = color.yellow, text_halign = text.align_left)

```

Chart timeframe

A script can obtain information on the type of timeframe used on the chart using these built-ins, which all return a “simple bool” result:

- timeframe.isseconds
- timeframe.isminutes

- `timeframe.isintraday`
- `timeframe.isdaily`
- `timeframe.isweekly`
- `timeframe.ismonthly`
- `timeframe.isdwm`

Additional built-ins return more specific timeframe information:

- `timeframe.multiplier` returns a “simple int” containing the multiplier of the timeframe unit. A chart timeframe of one hour will return 60 because intraday timeframes are expressed in minutes. A 30sec timeframe will return 30 (seconds), a daily chart will return 1 (day), a quarterly chart will return 3 (months), and a yearly chart will return 12 (months). The value of this variable cannot be used as an argument to `timeframe` parameters in built-in functions, as they expect a string in timeframe specifications format.
- `timeframe.period` holds a “string” representing the script’s timeframe. It follows Pine’s timeframe string specifications, where the string consists of a quantity (multiplier) and unit, e.g., “1D”, “2W”, “3M”. When used in a `request.*()` call’s `expression` argument, this variable references the *requested* context’s timeframe. Otherwise, it references the script’s main timeframe.
- `timeframe.main_period` holds a “string” representing the *main* timeframe, which is either the `timeframe` argument specified in the `indicator()` declaration, or the current chart’s timeframe. It behaves almost identically to `timeframe.period`. However, this variable *always* represents the script’s *main* timeframe, even within requested contexts.

See the page on Timeframes for more information.

Session information

Session information is available in different forms:

- The `syminfo.session` built-in variable returns a value that is either `session.regular` or `session.extended`. It reflects the session setting on the chart for that symbol. If the “Chart settings/Symbol/Session” field is set to “Extended”, it will only return “extended” if the symbol and the user’s feed allow for extended sessions. It is used when a session type is expected, for example as the argument for the `session` parameter in `ticker.new()`.
- Session state built-ins provide information on the trading session a bar belongs to.

[Previous

Bar states](#bar-states)[Next

Colors](#colors) User Manual/Concepts/Colors

Colors

Introduction

Script visuals can play a critical role in the usability of the indicators we write in Pine Script™. Well-designed plots and drawings make indicators easier to use and understand. Good visual designs establish a visual hierarchy that allows the more important information to stand out, and the less important one to not get in the way.

Using colors in Pine can be as simple as you want, or as involved as your concept requires. The 4,294,967,296 possible assemblies of color and transparency available in Pine Script™ can be applied to:

- Any element you can plot or draw in an indicator’s visual space, be it lines, fills, text or candles.
- The background of a script’s visual space, whether the script is running in its own pane, or in overlay mode on the chart.
- The color of bars or the body of candles appearing on a chart.

A script can only color the elements it places in its own visual space. The only exception to this rule is that a pane indicator can color chart bars or candles.

Pine Script™ has built-in colors such as `color.green`, as well as functions like `color.rgb()` which allow you to dynamically generate any color in the RGBA color space.

Transparency

Each color in Pine Script™ is defined by four values:

- Its red, green and blue components (0-255), following the RGB color model.
- Its transparency (0-100), often referred to as the Alpha channel outside Pine, as defined in the RGBA color model. Even though transparency is expressed in the 0-100 range, its value can be a “float” when used in functions, which gives you access to the 256 underlying values of the alpha channel.

The transparency of a color defines how opaque it is: zero is fully opaque, 100 makes the color — whichever it is — invisible. Modulating transparency can be crucial in more involved color visuals or when using backgrounds, to control which colors dominate the others, and how they mix together when superimposed.

Z-index

When you place elements in a script’s visual space, they have relative depth on the *z* axis; some will appear on top of others. The *z-index* is a value that represents the position of elements on the *z* axis. Elements with the highest z-index appear on top.

Elements drawn in Pine Script™ are divided in groups. Each group has its own position in the *z* space, and **within the same group**, elements created last in the script’s logic will appear on top of other elements from the same group. An element of one group cannot be placed outside the region of the *z* space attributed to its group, so a plot can never appear on top of a table, for example, because tables have the highest z-index.

This list contains the groups of visual elements, ordered by increasing z-index, so background colors are always at the bottom of *z* space, and tables will always appear on top of all other elements:

- Background colors
- Fills
- Plots
- Hlines
- LineFills
- Lines
- Boxes
- Labels
- Tables

Note that by using `explicit_plot_zorder = true` in `indicator()` or `strategy()`, you can control the relative z-index of `plot*`(), `hline()` and `fill()` visuals using their sequential order in the script.

Constant colors

There are 17 built-in colors in Pine Script™. This table lists their names, hexadecimal equivalent, and RGB values as arguments to `color.rgb()`:

Name	Hex	RGB values
<code>color.aqua</code>	<code>#00BCD4</code>	<code>color.rgb(0, 188, 212)</code>
<code>color.black</code>	<code>#363A45</code>	<code>color.rgb(54, 58, 69)</code>
<code>color.blue</code>	<code>#2196F3</code>	<code>color.rgb(33, 150, 243)</code>
<code>color.fuchsia</code>	<code>#E040FB</code>	<code>color.rgb(224, 64, 251)</code>
<code>color.gray</code>	<code>#787B86</code>	<code>color.rgb(120, 123, 134)</code>
<code>color.green</code>	<code>#4CAF50</code>	<code>color.rgb(76, 155, 80)</code>
<code>color.lime</code>	<code>#00E676</code>	<code>color.rgb(0, 230, 118)</code>
<code>color.maroon</code>	<code>#880E4F</code>	<code>color.rgb(136, 14, 79)</code>
<code>color.navy</code>	<code>#311B92</code>	<code>color.rgb(49, 27, 146)</code>
<code>color.olive</code>	<code>#808000</code>	<code>color.rgb(128, 128, 0)</code>
<code>color.orange</code>	<code>#FF9800</code>	<code>color.rgb(255, 152, 0)</code>
<code>color.purple</code>	<code>#9C27B0</code>	<code>color.rgb(156, 39, 176)</code>
<code>color.red</code>	<code>#F23645</code>	<code>color.rgb(242, 54, 69)</code>
<code>color.silver</code>	<code>#B2B5BE</code>	<code>color.rgb(178, 181, 190)</code>
<code>color.teal</code>	<code>#089981</code>	<code>color.rgb(8, 153, 129)</code>
<code>color.white</code>	<code>FFFFFF</code>	<code>color.rgb(255, 255, 255)</code>
<code>color.yellow</code>	<code>#FDD835</code>	<code>color.rgb(253, 216, 53)</code>

The following script shows three different ways to express `color.olive` with 40% transparency. All three methods are functionally equivalent:

```
//@version=6
indicator("Constant colors demo", overlay = true)

// Create a plot using the hex color code equivalent for `color.olive` with `99` as the alpha value (60% opacity)
plot(ta.sma(close, 10), "10-bar SMA", #80800099, 3)
// Create a plot using `color.new()` to modify `color.olive` with 40% transparency.
plot(ta.sma(close, 30), "30-bar SMA", color.new(color.olive, 40), 3)
// Create a plot using `color.rgb()` with the `r`, `g`, and `b` components of `color.olive` and 40% transparency
plot(ta.sma(close, 50), "50-bar SMA", color.rgb(128, 128, 0, 40), 3)
```

Note that:

- An alpha value of 99 in a hexadecimal color code is equivalent to 60% opacity, meaning the resulting color is 40% transparent.
- Transparency does *not* affect plot outputs in the status line, price scale, or Data Window. All these locations show the color with 0% transparency.



Figure 108: image

The colors in the previous script do not vary as the script executes bar to bar. Sometimes, however, colors need to be created as the script executes on each bar because they depend on conditions that are unknown at compile time, or when the script begins execution on bar zero. For those cases, programmers have two options:

1. Use conditional statements to select colors from a few pre-determined base colors.
2. Build new colors dynamically, by calculating them as the script executes bar to bar, to implement color gradients, for example.

Conditional coloring

Let's say you want to color a moving average in different colors, depending on some conditions you define. To do so, you can use a conditional statement that will select a different color for each of your states. Let's start by coloring a moving average in a bull color when it's rising, and in a bear color when it's not:



Figure 109: image

```
//@version=6
indicator("Conditional colors", "", true)
int lengthInput = input.int(20, "Length", minval = 2)
color maBullColorInput = input.color(color.green, "Bull")
color maBearColorInput = input.color(color.maroon, "Bear")
float ma = ta.sma(close, lengthInput)
// Define our states.
bool maRising = ta.rising(ma, 1)
```

```
// Build our color.
color c_ma = maRising ? maBullColorInput : maBearColorInput
plot(ma, "MA", c_ma, 2)
```

Note that:

- We provide users of our script a selection of colors for our bull/bear colors.
- We define an `maRising` boolean variable which will hold `true` when the moving average is higher on the current bar than it was on the last.
- We define a `c_ma` color variable that is assigned one of our two colors, depending on the value of the `maRising` boolean. We use the `? :` ternary operator to write our conditional statement.

You can also use conditional colors to avoid plotting under certain conditions. Here, we plot high and low pivots using a line, but we do not want to plot anything when a new pivot comes in, to avoid the joints that would otherwise appear in pivot transitions. To do so, we test for pivot changes and use `na` as the color value when a change is detected, so that no line is plotted on that bar:



Figure 110: image

```
//@version=6
indicator("Conditional colors", "", true)
int legsInput = input.int(5, "Pivot Legs", minval = 1)
color pHiColorInput = input.color(color.olive, "High pivots")
color pLoColorInput = input.color(color.orange, "Low pivots")
// Initialize the pivot level variables.
var float pHi = na
var float pLo = na
// When a new pivot is detected, save its value.
pHi := nz(ta.pivohigh(legsInput, legsInput), pHi)
pLo := nz(ta.pivotlow(legsInput, legsInput), pLo)
// When a new pivot is detected, do not plot a color.
plot(pHi, "High", ta.change(pHi) != 0 ? na : pHiColorInput, 2, plot.style_line)
plot(pLo, "Low", ta.change(pLo) != 0 ? na : pLoColorInput, 2, plot.style_line)
```

To understand how this code works, one must first know that `ta.pivohigh()` and `ta.pivotlow()`, used as they are here without an argument to the `source` parameter, will return a value when they find a high/low pivot, otherwise they return `na`.

When we test the value returned by the pivot function for `na` using the `nz()` function, we allow the value returned to be assigned to the `pHi` or `pLo` variables only when it is not `na`, otherwise the previous value of the variable is simply reassigned to it, which has no impact on its value. Keep in mind that previous values of `pHi` and `pLo` are preserved bar to bar because we use the `var` keyword when initializing them, which causes the initialization to only occur on the first bar.

All that's left to do next is, when we plot our lines, to insert a ternary conditional statement that will yield `na` for the color when the pivot value changes, or the color selected in the script's inputs when the pivot level does not change.

Calculated colors

Using functions like `color.new()`, `color.rgb()` and `color.from_gradient()`, one can build colors on the fly, as the script executes bar to bar.

`color.new()` is most useful when you need to generate different transparency levels from a base color.

`color.rgb()` is useful when you need to build colors dynamically from red, green, blue, or transparency components. While `color.rgb()` creates a color, its sister functions `color.r()`, `color.g()`, `color.b()` and `color.t()` can be used to extract the red, green, blue or transparency values from a color, which can in turn be used to generate a variant.

`color.from_gradient()` is useful to create linear gradients between two base colors. It determines which intermediary color to use by evaluating a source value against minimum and maximum values.

`color.new()`

Let's put `color.new(color, transp)` to use to create different transparencies for volume columns using one of two bull/bear base colors:



Figure 111: image

```
//@version=6
indicator("Volume")
// We name our color constants to make them more readable.
var color GOLD_COLOR    = #CCCC00ff
var color VIOLET_COLOR  = #AA00FFff
color bullColorInput = input.color(GOLD_COLOR,    "Bull")
color bearColorInput = input.color(VIOLET_COLOR, "Bear")
int levelsInput = input.int(10, "Gradient levels", minval = 1)
// We initialize only once on bar zero with `var`, otherwise the count would reset to zero on each bar.
var float riseFallCnt = 0
// Count the rises/falls, clamping the range to: 1 to `i_levels`.
riseFallCnt := math.max(1, math.min(levelsInput, riseFallCnt + math.sign(volume - nz(volume[1]))))
// Rescale the count on a scale of 80, reverse it and cap transparency to <80 so that colors remains visible.
float transparency = 80 - math.abs(80 * riseFallCnt / levelsInput)
// Build the correct transparency of either the bull or bear color.
color volumeColor = color.new(close > open ? bullColorInput : bearColorInput, transparency)
plot(volume, "Volume", volumeColor, 1, plot.style_columns)
```

Note that:

- In the next to last line of our script, we dynamically calculate the column color by varying both the base color used, depending on whether the bar is up or down, **and** the transparency level, which is calculated from the cumulative rises or falls of volume.

- We offer the script user control over not only the base bull/bear colors used, but also on the number of brightness levels we use. We use this value to determine the maximum number of rises or falls we will track. Giving users the possibility to manage this value allows them to adapt the indicator's visuals to the timeframe or market they use.
- We take care to control the maximum level of transparency we use so that it never goes higher than 80. This ensures our colors always retain some visibility.
- We also set the minimum value for the number of levels to 1 in the inputs. When the user selects 1, the volume columns will be either in bull or bear color of maximum brightness — or transparency zero.

color.rgb()

In our next example we use `color.rgb(red, green, blue, transp)` to build colors from RGBA values. We use the result in a holiday season gift for our friends, so they can bring their TradingView charts to parties:



Figure 112: image

```
//@version=6
indicator("Holiday candles", "", true)
float r = math.random(0, 255)
float g = math.random(0, 255)
float b = math.random(0, 255)
float t = math.random(0, 100)
color holidayColor = color.rgb(r, g, b, t)
plotcandle(open, high, low, close, color = holidayColor, wickcolor = holidayColor, bordercolor = holidayColor)
```

Note that:

- We generate values in the zero to 255 range for the red, green and blue channels, and in the zero to 100 range for transparency. Also note that because `math.random()` returns float values, the float 0.0-100.0 range provides access to the full 0-255 transparency values of the underlying alpha channel.
- We use the `math.random(min, max, seed)` function to generate pseudo-random values. We do not use an argument for the third parameter of the function: `seed`. Using it is handy when you want to ensure the repeatability of the function's results. Called with the same seed, it will produce the same sequence of values.

color.from_gradient()

Our last examples of color calculations will use `color.from_gradient(value, bottom_value, top_value, bottom_color, top_color)`. Let's first use it in its simplest form, to color a CCI signal in a version of the indicator that otherwise looks like the built-in:

```
//@version=6
indicator(title="CCI line gradient", precision=2, timeframe="")
var color GOLD_COLOR = #CCCC00
var color VIOLET_COLOR = #AA00FF
var color BEIGE_COLOR = #9C6E1B
float srcInput = input.source(close, title="Source")
int lenInput = input.int(20, "Length", minval = 5)
color bullColorInput = input.color(GOLD_COLOR, "Bull")
```

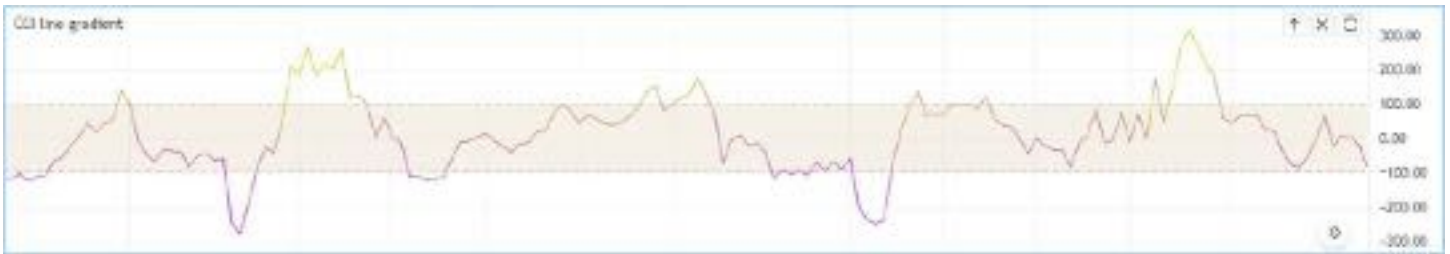



Figure 113: image

```
color bearColorInput = input.color(BEIGE_COLOR, "Bear")
float signal = ta.cci(srcInput, lenInput)
color signalColor = color.from_gradient(signal, -200, 200, bearColorInput, bullColorInput)
plot(signal, "CCI", signalColor)
bandTopPlotID = hline(100, "Upper Band", color.silver, hline.style_dashed)
bandBotPlotID = hline(-100, "Lower Band", color.silver, hline.style_dashed)
fill(bandTopPlotID, bandBotPlotID, color.new(BEIGE_COLOR, 90), "Background")
```

Note that:

- To calculate the gradient, `color.from_gradient()` requires minimum and maximum values against which the argument used for the `value` parameter will be compared. The fact that we want a gradient for an unbounded signal like CCI (i.e., without fixed boundaries such as RSI, which always oscillates between 0-100), does not entail we cannot use `color.from_gradient()`. Here, we solve our conundrum by providing values of -200 and 200 as arguments. They do not represent the real minimum and maximum values for CCI, but they are at levels from which we do not mind the colors no longer changing, as whenever the series is outside the `bottom_value` and `top_value` limits, the colors used for `bottom_color` and `top_color` will apply.
- The color progression calculated by `color.from_gradient()` is linear. If the value of the series is halfway between the `bottom_value` and `top_value` arguments, the generated color's RGBA components will also be halfway between those of `bottom_color` and `top_color`.
- Many common indicator calculations are available in Pine Script™ as built-in functions. Here we use `ta.cci()` instead of calculating it the long way.

The argument used for `value` in `color.from_gradient()` does not necessarily have to be the value of the line we are calculating. Anything we want can be used, as long as arguments for `bottom_value` and `top_value` can be supplied. Here, we enhance our CCI indicator by coloring the band using the number of bars since the signal has been above/below the centerline:

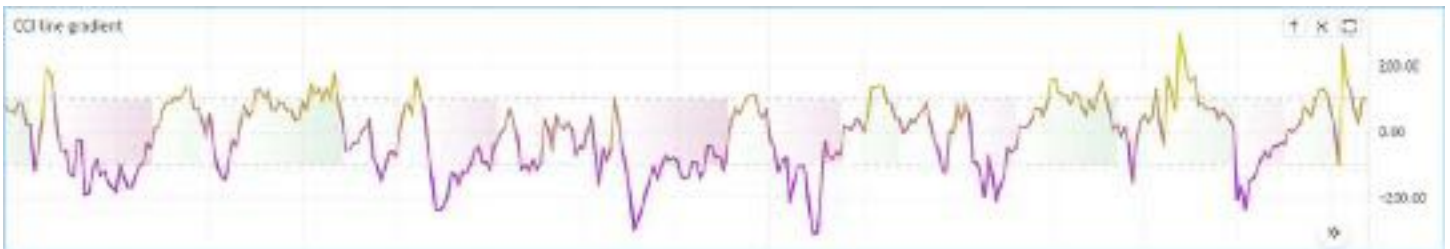


Figure 114: image

```
//@version=6
indicator(title="CCI line gradient", precision=2, timeframe="")
var color GOLD_COLOR = #CCCC00
var color VIOLET_COLOR = #AA00FF
var color GREEN_BG_COLOR = color.new(color.green, 70)
var color RED_BG_COLOR = color.new(color.maroon, 70)
float srcInput = input.source(close, "Source")
int lenInput = input.int(20, "Length", minval = 5)
int stepsInput = input.int(50, "Gradient levels", minval = 1)
color bullColorInput = input.color(GOLD_COLOR, "Line: Bull", inline = "11")
color bearColorInput = input.color(VIOLET_COLOR, "Bear", inline = "11")
color bullBgColorInput = input.color(GREEN_BG_COLOR, "Background: Bull", inline = "12")
```

```

color bearBgColorInput = input.color(RED_BG_COLOR, "Bear", inline = "12")

// Plot colored signal line.
float signal = ta.cci(srcInput, lenInput)
color signalColor = color.from_gradient(signal, -200, 200, color.new(bearColorInput, 0), color.new(bullColorIn
plot(signal, "CCI", signalColor, 2)

// Detect crosses of the centerline.
bool signalX = ta.cross(signal, 0)
// Count no of bars since cross. Capping it to the no of steps from inputs.
int gradientStep = math.min(stepsInput, nz(ta.barssince(signalX)))
// Choose bull/bear end color for the gradient.
color endColor = signal > 0 ? bullBgColorInput : bearBgColorInput
// Get color from gradient going from no color to `c_endColor`
color bandColor = color.from_gradient(gradientStep, 0, stepsInput, na, endColor)
bandTopPlotID = hline(100, "Upper Band", color.silver, hline.style_dashed)
bandBotPlotID = hline(-100, "Lower Band", color.silver, hline.style_dashed)
fill(bandTopPlotID, bandBotPlotID, bandColor, title = "Band")

```

Note that:

- The signal plot uses the same base colors and gradient as in our previous example. We have however increased the width of the line from the default 1 to 2. It is the most important component of our visuals; increasing its width is a way to give it more prominence, and ensure users are not distracted by the band, which has become busier than it was in its original, flat beige color.
- The fill must remain unobtrusive for two reasons. First, it is of secondary importance to the visuals, as it provides complementary information, i.e., the duration for which the signal has been in bull/bear territory. Second, since fills have a greater z-index than plots, the fill will cover the signal plot. For these reasons, we make the fill's base colors fairly transparent, at 70, so they do not mask the plots. The gradient used for the band starts with no color at all (see the na used as the argument to `bottom_color` in the `color.from_gradient()` call), and goes to the base bull/bear colors from the inputs, which the conditional, `c_endColor` color variable contains.
- We provide users with distinct bull/bear color selections for the line and the band.
- When we calculate the `gradientStep` variable, we use `nz()` on `ta.barssince()` because in early bars of the dataset, when the condition tested has not occurred yet, `ta.barssince()` will return na. Because we use `nz()`, the value returned is replaced with zero in those cases.

Mixing transparencies

In this example we take our CCI indicator in another direction. We will build dynamically adjusting extremes zone buffers using a Donchian Channel (historical highs/low) calculated from the CCI. We build the top/bottom bands by making them 1/4 the height of the DC. We will use a dynamically adjusting lookback to calculate the DC. To modulate the lookback, we will calculate a simple measure of volatility by keeping a ratio of a short-period ATR to a long one. When that ratio is higher than 50 of its last 100 values, we consider the volatility high. When the volatility is high/low, we decrease/increase the lookback.

Our aim is to provide users of our indicator with:

- The CCI line colored using a bull/bear gradient, as we illustrated in our most recent examples.
- The top and bottom bands of the Donchian Channel, filled in such a way that their color darkens as a historical high/low becomes older and older.
- A way to appreciate the state of our volatility measure, which we will do by painting the background with one color whose intensity increases when volatility increases.

This is what our indicator looks like using the light theme:



Figure 115: image

And with the dark theme:

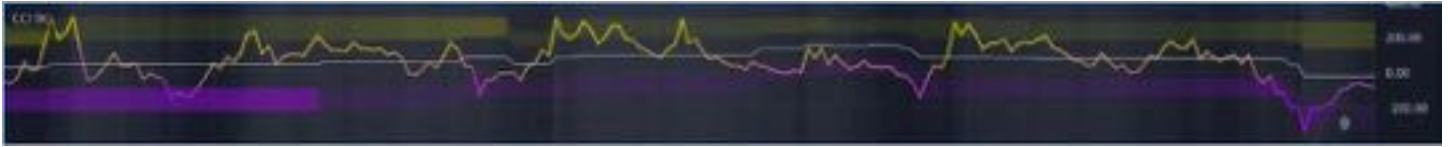


Figure 116: image

```
//@version=6
indicator("CCI DC", precision = 6)
color GOLD_COLOR = #CCCC00ff
color VIOLET_COLOR = #AA00FFff
int lengthInput = input.int(20, "Length", minval = 5)
color bullColorInput = input.color(GOLD_COLOR, "Bull")
color bearColorInput = input.color(VIOLET_COLOR, "Bear")

// ----- Function clamps `val` between `min` and `max`.
clamp(val, min, max) =>
    math.max(min, math.min(max, val))

// ----- Volatility expressed as 0-100 value.
float v = ta.ATR(lengthInput / 5) / ta.ATR(lengthInput * 5)
float vPct = ta.percentrank(v, lengthInput * 5)

// ----- Calculate dynamic lookback for DC. It increases/decreases on low/high volatility.
bool highVolatility = vPct > 50
var int lookBackMin = lengthInput * 2
var int lookBackMax = lengthInput * 10
var float lookBack = math.avg(lookBackMin, lookBackMax)
lookBack += highVolatility ? -2 : 2
lookBack := clamp(lookBack, lookBackMin, lookBackMax)

// ----- Dynamic lookback length Donchian channel of signal.
float signal = ta.cci(close, lengthInput)
// `lookBack` is a float; need to cast it to int to be used as a length.
float hiTop = ta.highest(signal, int(lookBack))
float loBot = ta.lowest(signal, int(lookBack))
// Get margin of 25% of the DC height to build high and low bands.
float margin = (hiTop - loBot) / 4
float hiBot = hiTop - margin
float loTop = loBot + margin
// Center of DC.
float center = math.avg(hiTop, loBot)

// ----- Create colors.
color signalColor = color.from_gradient(signal, -200, 200, bearColorInput, bullColorInput)
// Bands: Calculate transparencies so the longer since the hi/lo has changed,
// the darker the color becomes. Cap highest transparency to 90.
float hiTransp = clamp(100 - (100 * math.max(1, nz(ta.barssince(ta.change(hiTop) != 0) + 1)) / 255), 60, 90)
float loTransp = clamp(100 - (100 * math.max(1, nz(ta.barssince(ta.change(loBot) != 0) + 1)) / 255), 60, 90)
color hiColor = color.new(bullColorInput, hiTransp)
color loColor = color.new(bearColorInput, loTransp)
// Background: Rescale the 0-100 range of `vPct` to 0-25 to create 75-100 transparencies.
color bgColor = color.new(color.gray, 100 - (vPct / 4))

// ----- Plots
// Invisible lines for band fills.
hiTopPlotID = plot(hiTop, color = na)
hiBotPlotID = plot(hiBot, color = na)
```

```

loTopPlotID = plot(loTop, color = na)
loBotPlotID = plot(loBot, color = na)
// Plot signal and centerline.
p_signal = plot(signal, "CCI", signalColor, 2)
plot(center, "Centerline", color.silver, 1)

// Fill the bands.
fill(hiTopPlotID, hiBotPlotID, hiColor)
fill(loTopPlotID, loBotPlotID, loColor)

// ----- Background.
bgcolor(bgColor)

```

Note that:

- We clamp the transparency of the background to a 100-75 range so that it doesn't overwhelm. We also use a neutral color that will not distract too much. The darker the background is, the higher our measure of volatility.
- We also clamp the transparency values for the band fills between 60 and 90. We use 90 so that when a new high/low is found and the gradient resets, the starting transparency makes the color somewhat visible. We do not use a transparency lower than 60 because we don't want those bands to hide the signal line.
- We use the very handy `ta.percentrank()` function to generate a 0-100 value from our ATR ratio measuring volatility. It is useful to convert values whose scale is unknown into known values that can be used to produce transparencies.
- Because we must clamp values three times in our script, we wrote an `f_clamp()` function, instead of explicitly coding the logic three times.

Tips

Maintaining automatic color selectors

Under certain conditions, PineScript can automatically display all of the colors used in script's plots in the "Settings/Styles" menu. These plots are graphics created by all `plot*()` functions, `barcolor()`, and `bgcolor()`. The user can change the colors using a color picker. This feature allows colors in scripts to be customized without any extra code.

For example, this simple script has a `plot()` that is colored either teal or red, depending on the relationship between the bar's close and open. The script does not specify that these colors should be editable, nor does it create any color-related inputs. Nevertheless, PineScript automatically displays the colors in the "Settings/Styles" menu and allows the user to change them, along with the style of the plot:

```

//@version=6
indicator("Color picker showcase")
plotColor = close > open ? color.teal : color.red
plot(close, color = plotColor)

```

The colors in the above script can be automatically displayed in this way because they are *not dynamically calculated* and are known as soon as the script has finished compiling. All colors of the "const" type, and all colors of type "input" that are *not modified* via the `color.new()` or `color.rgb()` functions can be automatically displayed like this.

However, if *even a single calculated color* is of type "simple color" or "series color", or if an "input color" is passed to `color.new()` or `color.rgb()`, *all* colors are calculated in the script's runtime, and no color pickers are available in the "Style" section.

In practice, the creation of "simple" or "series" colors is also most often due to using `color.new()` and `color.rgb()` functions. The qualifier of the color that these functions return is the strongest qualifier of the values passed to them. If each call to these functions passes only "const" values, the resulting colors are also "const", and the script *does* display them in the "Styles" menu.

For example, let's try to make the plots in the script above semi-transparent by adding a transparency of 50 to its colors via `color.new()`. The easiest way to do this is to wrap the `plotColor` variable with `color.new()`, like in the example below:

```

//@version=6
indicator("Color picker showcase")
plotColor = color.new(close > open ? color.teal : color.red, 50)
plot(close, color = plotColor)

```

Color picker showcase



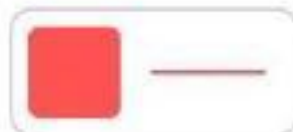
Style **Visibility**

☒ Plot

Color 0



Color 1



OUTPUTS

Precision

Default



☒ Labels on price scale

☒ Values in status line

Defaults



Cancel

Ok

Figure 117: image

Unfortunately, with these changes the “Style” tab does not display a color picker any longer. This is because we use the “series bool” condition `close > open` to decide the color, and then pass the result of this expression to a single `color.new()` call. The qualified type of the calculated color that it returns is “series color”.

To avoid this, we can ensure that every calculated color created by `color.new()` is a “const color”. Below, we wrap teal and red separately with `color.new()` – creating two constant calculated colors in the process – and then decide which one to assign to `plotColor` based on the condition. And while the `plotColor` variable is a “series color”, each `color.new()` call returns a constant color, so the script displays a color picker in the “Style” tab:

```
//@version=6
indicator("Color picker showcase")
plotColor = close > open ? color.new(color.teal, 50) : color.new(color.red, 50)
plot(close, color = plotColor)
```

To calculate the colors at runtime, create custom color inputs for all of the colors that are to be editable. This approach requires more effort, but allows significantly more control over what the user can affect. Learn more about creating color inputs on the [Inputs](#) page.

Designing usable colors schemes

If you write scripts intended for other traders, try to avoid colors that will not work well in some environments, whether it be for plots, labels, tables or fills. At a minimum, test your visuals to ensure they perform satisfactorily with both the light and dark TradingView themes; they are the most commonly used. Colors such as black and white, for example, should be avoided.

Build the appropriate inputs to provide script users the flexibility to adapt your script’s visuals to their particular environments.

Take care to build a visual hierarchy of the colors you use that matches the relative importance of your script’s visual components. Good designers understand how to achieve the optimal balance of color and weight so the eye is naturally drawn to the most important elements of the design. When you make everything stand out, nothing does. Make room for some elements to stand out by toning down the visuals surrounding it.

Providing a selection of color presets in your inputs — rather than a single color that can be changed — can help color-challenged users. Our [Technical Ratings](#) demonstrates one way of achieving this.

Plot crisp lines

It is best to use zero transparency to plot the important lines in your visuals, to keep them crisp. This way, they will show through fills more precisely. Keep in mind that fills have a higher z-index than plots, so they are placed on top of them. A slight increase of a line’s width can also go a long way in making it stand out.

If you want a special plot to stand out, you can also give it more importance by using multiple plots for the same line. These are examples where we modulate the successive width and transparency of plots to achieve this:

```
//@version=6
indicator("")
plot(high, "", color.new(color.orange, 80), 8)
plot(high, "", color.new(color.orange, 60), 4)
plot(high, "", color.new(color.orange, 00), 1)

plot(hl2, "", color.new(color.orange, 60), 4)
plot(hl2, "", color.new(color.orange, 00), 1)

plot(low, "", color.new(color.orange, 0), 1)
```

Customize gradients

When building gradients, adapt them to the visuals they apply to. If you are using a gradient to color candles, for example, it is usually best to limit the number of steps in the gradient to ten or less, as it is more difficult for the eye to perceive intensity variations of discrete objects. As we did in our examples, cap minimum and maximum transparency levels so your visual elements remain visible and do not overwhelm when it’s not necessary.

[\[Previous](#)

Color picker showcase



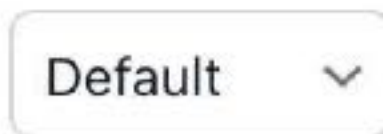
Style **Visibility**

☒ Plot



OUTPUTS

Precision



☒ Labels on price scale

☒ Values in status line

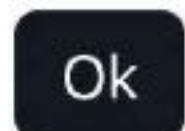
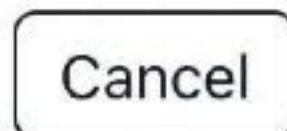


Figure 118: image

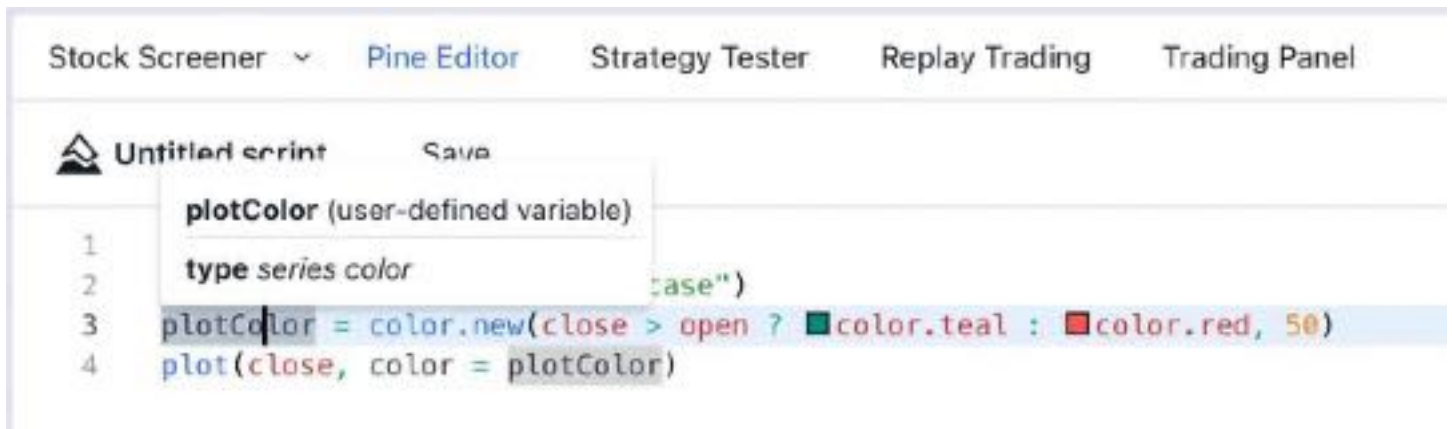


Figure 119: image

Chart information](#chart-information)[Next

Fills](#fills) User Manual/Concepts/Fills

Fills

Introduction

Some of Pine Script's visual outputs, including plots, hlines, lines, boxes, and polylines, allow one to fill the chart space they occupy with colors. Three different mechanisms facilitate filling the space between such outputs:

- The `fill()` function fills the space between two plots from `plot()` calls or two horizontal lines (`hlines`) from `hline()` calls with a specified color.
- Objects of the `linefill` type fill the space between line instances created with `line.new()`.
- Other drawing types, namely boxes and polylines, have built-in properties that allow the drawings to fill the visual spaces they occupy.

`plot()` and `hline()` fills

The `fill()` function fills the space between two plots or horizontal lines. It has the following two signatures:

`fill(plot1, plot2, color, title, editable, show_last, fillgaps) → void`
`fill(hline1, hline2, color, title, editable, show_last, fillgaps) → void`

The `plot1`, `plot2`, `hline1`, and `hline2` parameters accept plot or `hline` IDs returned by `plot()` and `hline()` function calls. The `fill()` function is the only built-in that can use these IDs.

This simple example demonstrates how the `fill()` function works with plot and `hline` IDs. It calls `plot()` and `hline()` three times to display arbitrary values on the chart. Each of these calls returns an ID, which the script assigns to variables for use in the `fill()` function. The values of `p1`, `p2`, and `p3` are “plot” IDs, whereas `h1`, `h2`, and `h3` reference “`hline`” IDs:

```

//@version=6
indicator("Example 1")

// Assign "plot" IDs to the `p1`, `p2`, and `p3` variables.
p1 = plot(math.sin(high), "Sine of `high`")
p2 = plot(math.cos(low), "Cosine of `low`")
p3 = plot(math.sin(close), "Sine of `close`")
// Fill the space between `p1` and `p2` with 90% transparent red.
fill(p1, p3, color.new(color.red, 90), "`p1`-`p3` fill")
// Fill the space between `p2` and `p3` with 90% transparent blue.
fill(p2, p3, color.new(color.blue, 90), "`p2`-`p3` fill")

// Assign "hline" IDs to the `h1`, `h2`, and `h3` variables.
h1 = hline(0, "First level")
h2 = hline(1.0, "Second level")

```

Color picker showcase



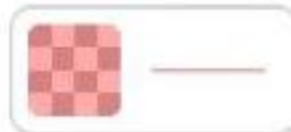
Style Visibility

☒ Plot

Color 0



Color 1



OUTPUTS

Precision

Default

☒ Labels on price scale

☒ Values in status line

Defaults

Cancel

Ok

Figure 120: image

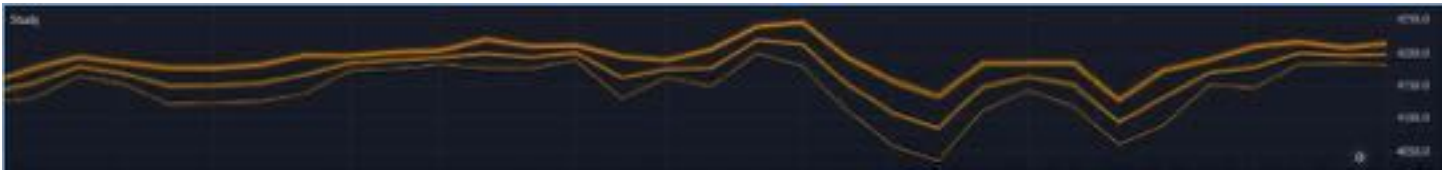


Figure 121: image



Figure 122: image

```

h3 = hline(0.5, "Third level")
h4 = hline(1.5, "Fourth level")
// Fill the space between `h1` and `h2` with 90% transparent yellow.
fill(h1, h2, color.new(color.yellow, 90), "`h1`-`h2` fill")
// Fill the space between `h3` and `h4` with 90% transparent lime.
fill(h3, h4, color.new(color.lime, 90), "`h3`-`h4` fill")

```

It's important to note that the `fill()` function requires *either* two “plot” IDs or two “hline” IDs. One *cannot* mix and match these types in the function call. Consequently, programmers will sometimes need to use `plot()` where they otherwise might have used `hline()` if they want to fill the space between a consistent level and a fluctuating series.

For example, this script calculates an **oscillator** based on the percentage distance between the chart's close price and a 10-bar SMA, then plots it on the chart pane. In this case, we wanted to fill the area between the **oscillator** and zero. Although we can display the zero level with `hline()` since its value does not change, we cannot pass a “plot” and “hline” ID to the `fill()` function. Therefore, we must use a `plot()` call for the level to allow the script to fill the space:



Figure 123: image

```

//@version=6
indicator("Example 2")

//@variable The 10-bar moving average of `close` prices.
float ma = ta.sma(close, 10)
//@variable The distance from the `ma` to the `close` price, as a percentage of the `ma`.
float oscillator = 100 * (ma - close) / ma

//@variable The ID of the `oscillator` plot for use in the `fill()` function.
oscPlotID = plot(oscillator, "Oscillator")
//@variable The ID of the zero level plot for use in the `fill()` function.
// Requires a "plot" ID since the `fill()` function can't use "plot" and "hline" IDs at the same time
zeroPlotID = plot(0, "Zero level", color.silver, 1, plot.style_circles)

// Filll the space between the `oscPlotID` and `zeroPlotID` with 90% transparent blue.
fill(oscPlotID, zeroPlotID, color.new(color.blue, 90), "Oscillator fill")

```

The `color` parameter of the `fill()` function accepts a “series color” argument, meaning the fill's color can change across chart bars. For example, this code fills the space between two moving average plots with 90% transparent green or red colors based on whether `ma1` is above `ma2`:

```

//@version=6

```



Figure 124: image

```
indicator("Example 3", overlay = true)
```

```
//@variable The 5-bar moving average of `close` prices.
```

```
float ma1 = ta.sma(close, 5)
```

```
//@variable The 20-bar moving average of `close` prices.
```

```
float ma2 = ta.sma(close, 20)
```

```
//@variable The 90% transparent color of the space between MA plots. Green if `ma1 > ma2`, red otherwise.
color fillColor = ma1 > ma2 ? color.new(color.green, 90) : color.new(color.red, 90)
```

```
//@variable The ID of the `ma1` plot for use in the `fill()` function.
```

```
ma1PlotID = plot(ma1, "5-bar SMA")
```

```
//@variable The ID of the `ma2` plot for use in the `fill()` function.
```

```
ma2PlotID = plot(ma2, "20-bar SMA")
```

```
// Fill the space between the `ma1PlotID` and `ma2PlotID` using the `fillColor`.
```

```
fill(ma1PlotID, ma2PlotID, fillColor, "SMA plot fill")
```

Line fills

While the `fill()` function allows a script to fill the space between two plots or hlines, it does not work with line objects. When a script needs to fill the space between lines, it requires a linefill object created by the `linefill.new()` function. The function has the following signature:

```
linefill.new(line1, line2, color) → series linefill
```

The `line1` and `line2` parameters accept line IDs. These IDs determine the chart region that the linefill object will fill with its specified `color`. A script can update the `color` property of a linefill ID returned by this function by calling `linefill.set_color()` with the ID as its `id` argument.

The behavior of linefills depends on the lines they reference. Scripts cannot move linefills directly, as the lines that a linefill uses determine the space it will fill. To retrieve the IDs of the lines referenced by a linefill object, use the `linefill.get_line1()` and `linefill.get_line2()` functions.

Any pair of line instances can only have *one* linefill between them. Successive calls to `linefill.new()` using the same `line1` and `line2` arguments will create a new linefill ID that *replaces* the previous one associated with them.

The example below demonstrates a simple use case for linefills. The script calculates a `pivotHigh` and `pivotLow` series

using the built-in `ta.pivohigh()` and `ta.pivotlow()` functions with constant `leftbars` and `rightbars` arguments. On the last confirmed historical bar, the script draws two extended lines. The first line connects the two most recent non-na `pivotHigh` values, and the second connects the most recent non-na `pivotLow` values.

To emphasize the “channel” formed by these lines, the script fills the space between them using `linefill.new()`:



Figure 125: image

```
//@version=6
indicator("Linefill demo", "Channel", true)

//@variable The number bars to the left of a detected pivot.
int LEFT_BARS = 15
//@variable The number bars to the right for pivot confirmation.
int RIGHT_BARS = 5

//@variable The price of the pivot high point.
float pivotHigh = ta.pivohigh(LEFT_BARS, RIGHT_BARS)
//@variable The price of the pivot low point.
float pivotLow = ta.pivotlow(LEFT_BARS, RIGHT_BARS)

// Initialize the chart points the lines will use.
var firstHighPoint = chart.point.new(na, na, na)
var secondHighPoint = chart.point.new(na, na, na)
var firstLowPoint = chart.point.new(na, na, na)
var secondLowPoint = chart.point.new(na, na, na)

// Update the `firstHighPoint` and `secondHighPoint` when `pivotHigh` is not `na`.
if not na(pivotHigh)
    firstHighPoint := secondHighPoint
    secondHighPoint := chart.point.from_index(bar_index - RIGHT_BARS, pivotHigh)
// Update the `firstLowPoint` and `secondLowPoint` when `pivotlow` is not `na`.
if not na(pivotLow)
    firstLowPoint := secondLowPoint
    secondLowPoint := chart.point.from_index(bar_index - RIGHT_BARS, pivotLow)
```

```

if barstate.islastconfirmedhistory
    //@variable An extended line that passes through the `firstHighPoint` and `secondHighPoint`.
    line pivotHighLine = line.new(firstHighPoint, secondHighPoint, extend = extend.right)
    //@variable An extended line that passes through the `firstLowPoint` and `secondLowPoint`.
    line pivotLowLine = line.new(firstLowPoint, secondLowPoint, extend = extend.right)
    //@variable The color of the space between the lines.
    color fillColor = switch
        secondHighPoint.price > firstHighPoint.price and secondLowPoint.price > firstLowPoint.price => color.l
        secondHighPoint.price < firstHighPoint.price and secondLowPoint.price < firstLowPoint.price => color.r
    => color.s
    //@variable A linefill that colors the space between the `pivotHighLine` and `pivotLowLine`.
    linefill channelFill = linefill.new(pivotHighLine, pivotLowLine, color.new(fillColor, 90))

```

Box and polyline fills

The box and polyline types allow scripts to draw geometric shapes and other formations on the chart. Scripts create boxes and polylines with the `box.new()` and `polyline.new()` functions, which include parameters that allow the drawings to fill their visual spaces.

To fill the space inside the borders of a box with a specified color, include a `bgcolor` argument in the `box.new()` function. To fill a polyline's visual space, pass a `fill_color` argument to the `polyline.new()` function.

For example, this script draws an octagon with a polyline and an inscribed rectangle with a box on the last confirmed historical bar. It determines the size of the drawings using the value from the `radius` variable, which corresponds to approximately one-fourth of the number of bars visible on the chart. We included `fill_color = color.new(color.blue, 60)` in the `polyline.new()` call to fill the octagon with a translucent blue color, and we used `bgcolor = color.purple` in the `box.new()` call to fill the inscribed rectangle with opaque purple:

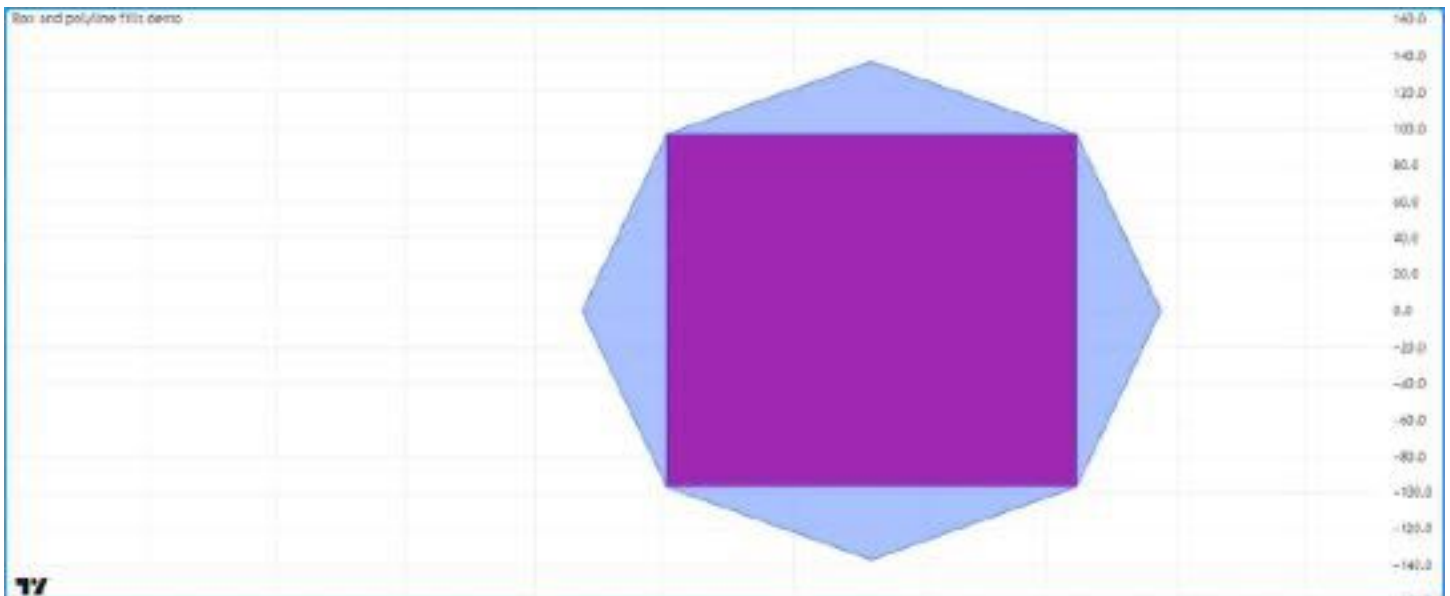


Figure 126: image

```

//@version=6
indicator("Box and polyline fills demo")

//@variable The number of visible chart bars, excluding the leftmost and rightmost bars.
var int barCount = 0
if time > chart.left_visible_bar_time and time < chart.right_visible_bar_time
    barCount += 1

//@variable The approximate radius used to calculate the octagon and rectangle coordinates.
int radius = math.ceil(barCount / 4)

if barstate.islastconfirmedhistory

```

```

//@variable An array of chart points. The polyline uses all points in this array, but the box only needs t
array<chart.point> points = array.new<chart.point>()
//@variable The counterclockwise angle of each point, in radians. Updates on each loop iteration.
float angle = 0.0
//@variable The radians to add to the `angle` on each loop iteration.
float increment = 0.25 * math.pi
// Loop 8 times to calculate octagonal points.
for i = 0 to 7
    //@variable The point's x-coordinate (bar offset).
    int x = int(math.round(math.cos(angle) * radius))
    //@variable The point's y-coordinate.
    float y = math.round(math.sin(angle) * radius)
    // Push a new chart point into the `points` array and increase the `angle`.
    points.push(chart.point.from_index(bar_index - radius + x, y))
    angle += increment
// Create a closed polyline to draw the octagon and fill it with translucent blue.
polyline.new(points, closed = true, fill_color = color.new(color.blue, 60))
// Create a box for the rectangle using index 3 and 7 for the top-left and bottom-right corners,
// and fill it with opaque purple.
box.new(points.get(3), points.get(7), bgcolor = color.purple)

```

See this manual's Lines and boxes page to learn more about working with these types.

[\[Previous](#)

[Colors\]\(#colors\)\[Next](#)

[Inputs\]\(#inputs\) User Manual/Concepts/Inputs](#)

Inputs

Introduction

Inputs receive values that users can change from a script's "Settings/Inputs" tab. By utilizing inputs, programmers can write scripts that users can more easily adapt to their preferences.

The following script plots a 20-period simple moving average (SMA) using `ta.sma(close, 20)`. While it is straightforward to write, the code is not very *flexible* because the function call uses specific **source** and **length** arguments that users cannot change without modifying the code:

```

//@version=6
indicator("MA", "", true)
plot(ta.sma(close, 20))

```

If we write our script this way instead, it becomes much more flexible, as users can select the **source** and the **length** values they want to use from the "Settings/Inputs" tab without changing the source code:

```

//@version=6
indicator("MA", "", true)
sourceInput = input(close, "Source")
lengthInput = input(20, "Length")
plot(ta.sma(sourceInput, lengthInput))

```

Inputs are only accessible while a script runs on a chart. Users can access script inputs from the "Settings" dialog box. To open this dialog, users can:

- Double-click on the name of an on-chart indicator
- Right-click on the script's name and choose the "Settings" item from the dropdown menu
- Choose the "Settings" item from the "More" menu icon (three dots) that appears when hovering over the indicator's name on the chart
- Double-click on the indicator's name from the Data Window (fourth icon down to the right of the chart)

The "Settings" dialog always contains the "Style" and "Visibility" tabs, which allow users to specify their preferences about the script's visuals and the chart timeframes that can display its outputs.

When a script contains calls to `input.*()` functions, an “Inputs” tab also appears in the “Settings” dialog box.

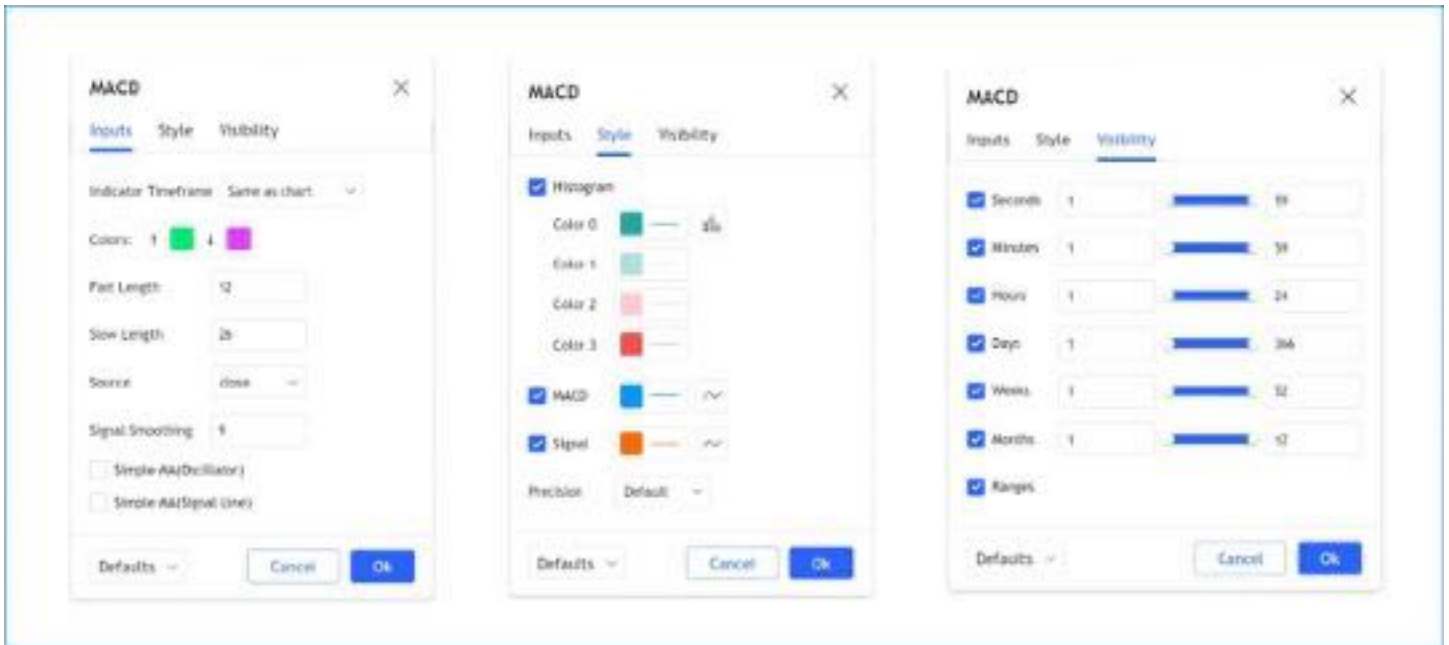


Figure 127: image

Scripts process inputs when users add them to the chart or change the values in the script’s “Settings/Inputs” tab. Any changes to a script’s inputs prompt it to re-execute across all available data using the new specified values.

Input functions

Pine Script™ features the following input functions:

- `input()`
- `input.int()`
- `input.float()`
- `input.bool()`
- `input.color()`
- `input.string()`
- `input.text_area()`
- `input.timeframe()`
- `input.symbol()`
- `input.price()`
- `input.source()`
- `input.session()`
- `input.time()`
- `input.enum()`

Scripts create input *widgets* in the “Inputs” tab that accept different types of inputs based on their `input.*()` function calls. By default, each input appears on a new line of the “Inputs” tab in the order of the `input.*()` calls. Programmers can also organize inputs in different ways by using the `input.*()` functions’ `group` and `inline` parameters. See this section below for more information.

Our Style guide recommends placing `input.*()` calls at the beginning of the script.

Input functions typically contain several parameters that allow programmers to define their default values, value limits, their organization in the “Inputs” tab, and other properties.

Since an `input.*()` call is simply another function call in Pine Script™, programmers can combine them with arithmetic, comparison, logical, and ternary operators to assign expressions to variables. This simple script compares the result from a call to `input.string()` to the “On” string and assigns the result to the `plotDisplayInput` variable. This variable is of the “input bool” type because the `==` operator returns a “bool” value:

```
//@version=6
```



```
indicator("Input in an expression", "", true)
bool plotDisplayInput = input.string("On", "Plot Display", options = ["On", "Off"]) == "On"
plot(plotDisplayInput ? close : na)
```

All values returned by `input.*()` functions except “source” ones are “input” qualified values. See our User Manual’s section on type qualifiers for more information.

Input function parameters

The parameters common to all input functions are: `defval`, `title`, `tooltip`, `inline`, `group`, and `display`. Some input functions also include other parameters: `options`, `minval`, `maxval`, `step` and `confirm`.

All these parameters expect “const” arguments. The only exceptions are for the `defval` and `options` parameters of the source and enum inputs, as `input.source()` uses “series float” value, and `input.enum()` uses *members* of an enum type.

Since `input.*()` parameters accept “const” arguments in most cases and the “input” and other qualifiers are stronger than “const”, it follows that one cannot use the result from one `input.*()` call as an argument in another `input.*()` call.

Let’s go over each parameter:

- `defval` is the first parameter of all input functions. It is the default value that appears in the input widget. It requires an argument of the type of input value that the function applies to.
- `title` requires a “const string” argument. It is the field’s label.
- `tooltip` requires a “const string” argument. When the parameter is used, a question mark icon will appear to the right of the field. When users hover over it, the tooltip’s text will appear. Note that if multiple input fields are grouped on one line using `inline`, the tooltip will always appear to the right of the rightmost field, and display the text of the last `tooltip` argument used in the line. Newlines (`\n`) are supported in the argument string.
- `inline` requires a “const string” argument. Using the same argument for the parameter in multiple `input.*()` calls will group their input widgets on the same line. There is a limit to the width the “Inputs” tab will expand, so a limited quantity of input fields can be fitted on one line. Using one `input.*()` call with a unique argument for `inline` has the effect of bringing the input field left, immediately after the label, foregoing the default left-alignment of all input fields used when no `inline` argument is used.
- `group` requires a “const string” argument. Use it to group any number of inputs in an organized section. The string used as the `group` argument becomes the section’s heading. All `input.*()` calls to be grouped together must use the same string for their `group` argument.
- `options` requires a comma-separated list of elements enclosed in square brackets (e.g., `["ON", "OFF"]`, `[1, 2, 3]`, `[myEnum.On, myEnum.Off]`). The input uses the specified elements as menu selections in its resulting dropdown widget. Users can only select one menu item at a time. When supplying an `options` list, the `defval` value must be one of the list’s elements. Inputs that allow `minval`, `maxval`, or `step` parameters cannot use those parameters and the `options` parameter simultaneously.
- `minval` requires a “const int/float” argument, depending on the type of the `defval` value. It is the minimum valid value for the input field.
- `maxval` requires a “const int/float” argument, depending on the type of the `defval` value. It is the maximum valid value for the input field.
- `step` is the increment by which the field’s value will move when the widget’s up/down arrows are used.
- `confirm` requires a “const bool” (`true` or `false`) argument. This parameter affects the behavior of the script when it is added to a chart. `input.*()` calls using `confirm = true` will cause the “Settings/Inputs” tab to popup when the script is added to the chart. `confirm` is useful to ensure that users configure a particular field.

The `minval`, `maxval` and `step` parameters are only present in the signature of the `input.int()` and `input.float()` functions.

Input types

The next sections explain what each input function does. As we proceed, we will explore the different ways you can use input functions and organize their display.

Generic input

`input()` is a simple, generic function that supports the fundamental Pine Script™ types: “int”, “float”, “bool”, “color” and “string”. It also supports “source” inputs, which are price-related values such as `close`, `hl2`, `hlc3`, and `hlcc4`, or which can be used to receive the output value of another script.

Its signature is:

```
input(defval, title, tooltip, inline, group) → input int/float/bool/color/string | series float
```

The function automatically detects the type of input by analyzing the type of the `defval` argument used in the function call. This script shows all the supported types and the qualified type returned by the function when used with `defval` arguments of different types:

```
//@version=6
indicator("`input()`", "", true)
a = input(1, "input int")
b = input(1.0, "input float")
c = input(true, "input bool")
d = input(color.orange, "input color")
e = input("1", "input string")
f = input(close, "series float")
plot(na)
```

Integer input

Two signatures exist for the `input.int()` function; one when `options` is not used, the other when it is:

`input.int(defval, title, minval, maxval, step, tooltip, inline, group, confirm) → input int`
`input.int(defval, title, minval, maxval, step, tooltip, inline, group, confirm, options) → input int`

This call uses the `options` parameter to propose a pre-defined list of lengths for the MA:

```
//@version=6
indicator("MA", "", true)
maLengthInput = input.int(10, options = [3, 5, 7, 10, 14, 20, 50, 100, 200])
ma = ta.sma(close, maLengthInput)
plot(ma)
```

This one uses the `minval` parameter to limit the length:

```
//@version=6
indicator("MA", "", true)
maLengthInput = input.int(10, minval = 2)
ma = ta.sma(close, maLengthInput)
plot(ma)
```

The version with the `options` list uses a dropdown menu for its widget. When the `options` parameter is not used, a simple input widget is used to enter the value:

Float input

Two signatures exist for the `input.float()` function; one when `options` is not used, the other when it is:

`input.float(defval, title, minval, maxval, step, tooltip, inline, group, confirm) → input float`
`input.float(defval, title, minval, maxval, step, tooltip, inline, group, confirm, options) → input float`

Here, we use a “float” input for the factor used to multiple the standard deviation, to calculate Bollinger Bands:

```
//@version=6
indicator("MA", "", true)
maLengthInput = input.int(10, minval = 1)
bbFactorInput = input.float(1.5, minval = 0, step = 0.5)
ma = ta.sma(close, maLengthInput)
bbWidth = ta.stdev(ma, maLengthInput) * bbFactorInput
bbHi = ma + bbWidth
bbLo = ma - bbWidth
plot(ma)
plot(bbHi, "BB Hi", color.gray)
plot(bbLo, "BB Lo", color.gray)
```

The input widgets for floats are similar to the ones used for integer inputs:

Boolean input

Let’s continue to develop our script further, this time by adding a boolean input to allow users to toggle the display of the BBs:

input()

×

Inputs

Style

Visibility

Input int

1

Input float

1

☒ Input bool

Input color

Input string

1

Series float

close

▼

Defaults ▼

Cancel

Ok

Figure 128: image

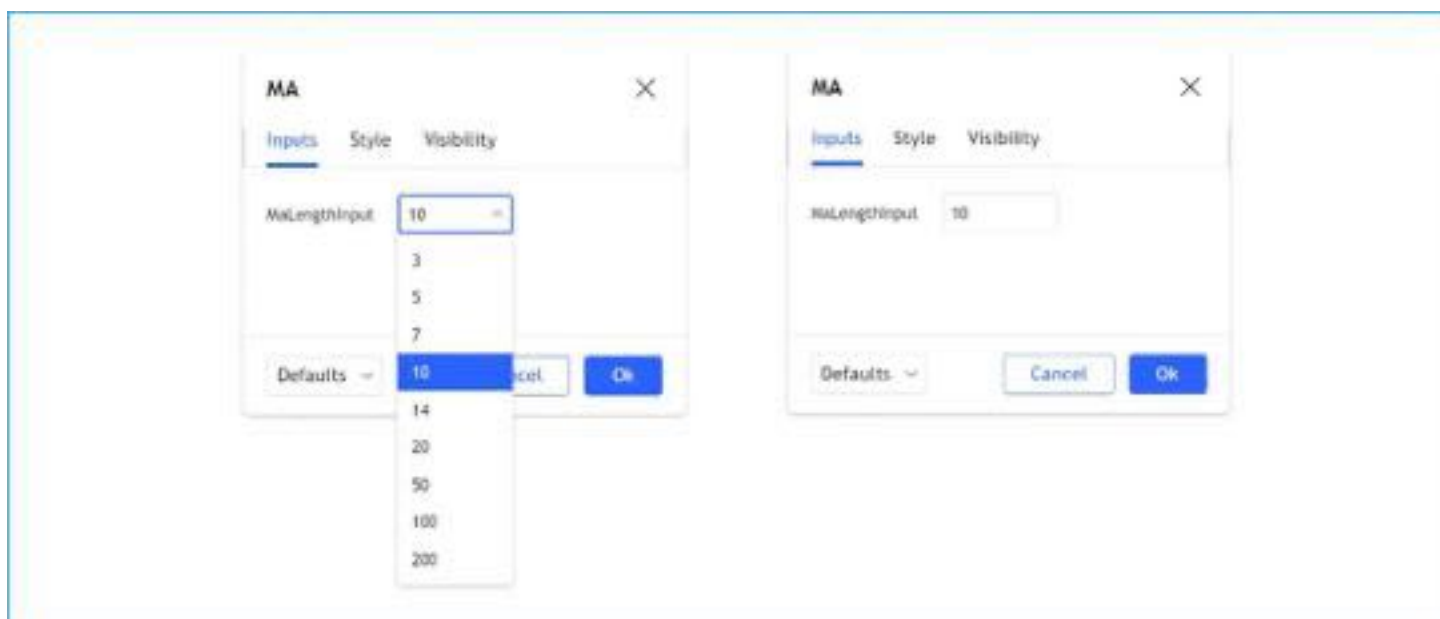


Figure 129: image

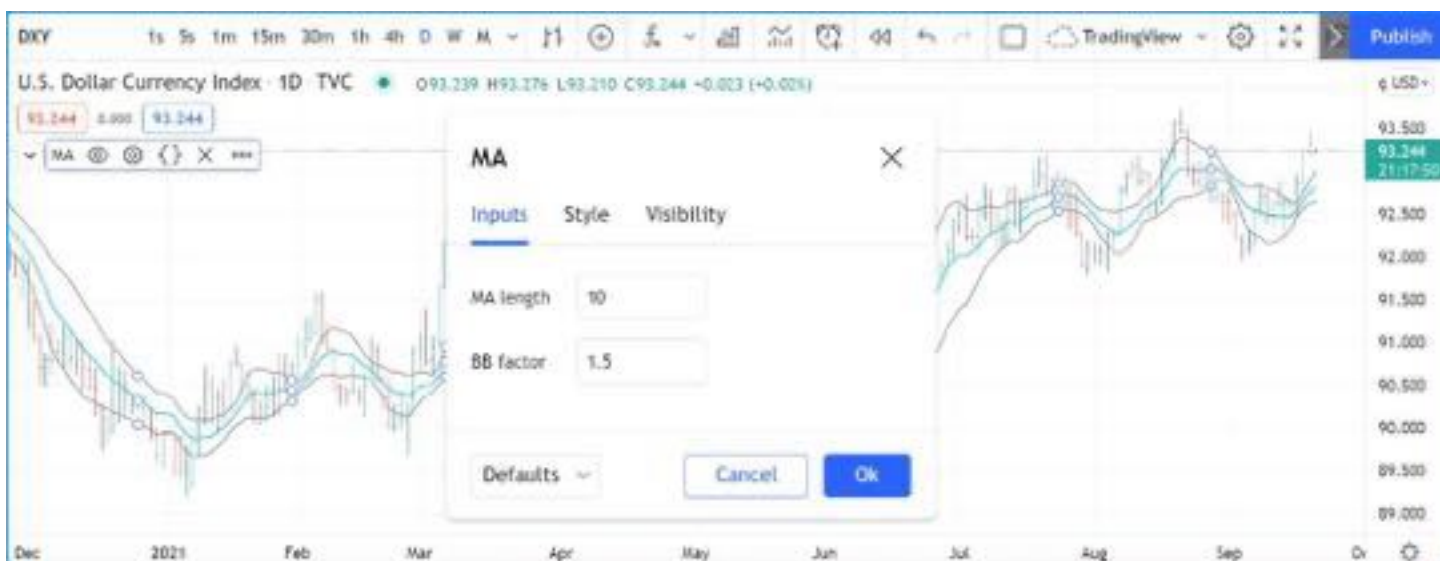


Figure 130: image

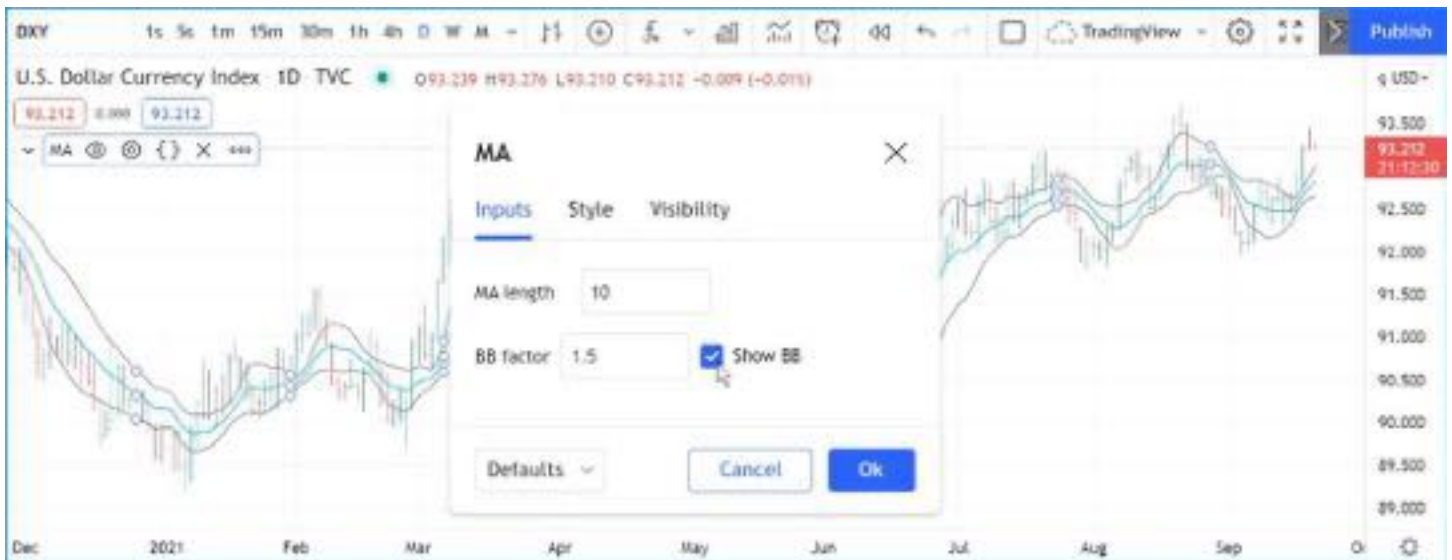


Figure 131: image

```
//@version=6
indicator("MA", "", true)
maLengthInput = input.int(10, "MA length", minval = 1)
bbFactorInput = input.float(1.5, "BB factor", inline = "01", minval = 0, step = 0.5)
showBBInput = input.bool(true, "Show BB", inline = "01")
ma = ta.sma(close, maLengthInput)
bbWidth = ta.stdev(ma, maLengthInput) * bbFactorInput
bbHi = ma + bbWidth
bbLo = ma - bbWidth
plot(ma, "MA", color.aqua)
plot(showBBInput ? bbHi : na, "BB Hi", color.gray)
plot(showBBInput ? bbLo : na, "BB Lo", color.gray)
```

Note that:

- We have added an input using `input.bool()` to set the value of `showBBInput`.
- We use the `inline` parameter in that input and in the one for `bbFactorInput` to bring them on the same line. We use "01" for its argument in both cases. That is how the Pine Script™ compiler recognizes that they belong on the same line. The particular string used as an argument is unimportant and does not appear anywhere in the “Inputs” tab; it is only used to identify which inputs go on the same line.
- We have vertically aligned the `title` arguments of our `input.*()` calls to make them easier to read.
- We use the `showBBInput` variable in our two `plot()` calls to plot conditionally. When the user unchecks the checkbox of the `showBBInput` input, the variable’s value becomes `false`. When that happens, our `plot()` calls plot the `na` value, which displays nothing. We use `true` as the default value of the input, so the BBs plot by default.
- Because we use the `inline` parameter for the `bbFactorInput` variable, its input field in the “Inputs” tab does not align vertically with that of `maLengthInput`, which doesn’t use `inline`.

Color input

As explained in this section of the Colors page, selecting the colors of a script’s outputs via the “Settings/Style” tab is not always possible. In the case where one cannot choose colors from the “Style” tab, programmers can create color inputs with the `input.color()` function to allow color customization from the “Settings/Inputs” tab.

Suppose we wanted to plot our BBs with a lighter transparency when the high and low values are higher/lower than the BBs. We can use a code like this to create the colors:

```
bbHiColor = color.new(color.gray, high > bbHi ? 60 : 0)
bbLoColor = color.new(color.gray, low < bbLo ? 60 : 0)
```

When using dynamic (“series”) color components like the `transp` arguments in the above code, the color widgets in the “Settings/Style” tab will no longer appear. Let’s create our own input for color selection, which will appear in the “Settings/Inputs” tab:

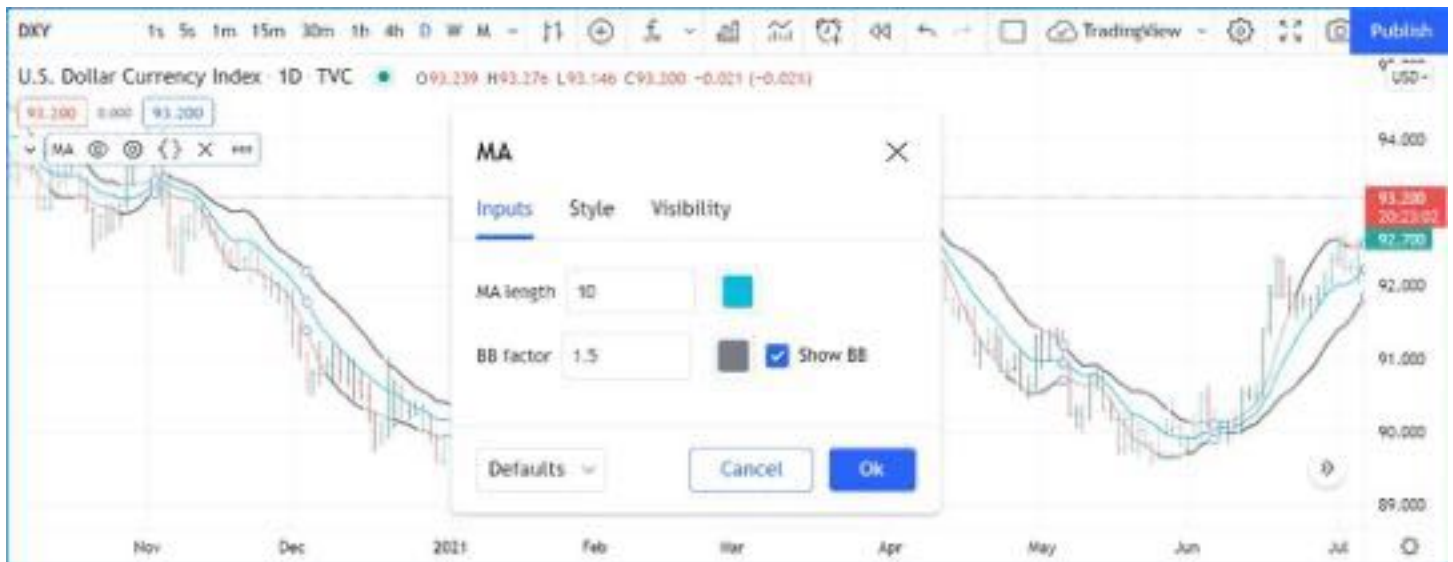


Figure 132: image

```
//@version=6
indicator("MA", "", true)
maLengthInput = input.int(10, "MA length", inline = "01", minval = 1)
maColorInput = input.color(color.aqua, "", inline = "01")
bbFactorInput = input.float(1.5, "BB factor", inline = "02", minval = 0, step = 0.5)
bbColorInput = input.color(color.gray, "", inline = "02")
showBBInput = input.bool(true, "Show BB", inline = "02")
ma = ta.sma(close, maLengthInput)
bbWidth = ta.stdev(ma, maLengthInput) * bbFactorInput
bbHi = ma + bbWidth
bbLo = ma - bbWidth
bbHiColor = color.new(bbColorInput, high > bbHi ? 60 : 0)
bbLoColor = color.new(bbColorInput, low < bbLo ? 60 : 0)
plot(ma, "MA", maColorInput)
plot(showBBInput ? bbHi : na, "BB Hi", bbHiColor, 2)
plot(showBBInput ? bbLo : na, "BB Lo", bbLoColor, 2)
```

Note that:

- We have added two calls to `input.color()` to gather the values of the `maColorInput` and `bbColorInput` variables. We use `maColorInput` directly in the `plot(ma, "MA", maColorInput)` call, and we use `bbColorInput` to build the `bbHiColor` and `bbLoColor` variables, which modulate the transparency using the position of price relative to the BBs. We use a conditional value for the `transp` value we call `color.new()` with, to generate different transparencies of the same base color.
- We do not use a `title` argument for our new color inputs because they are on the same line as other inputs allowing users to understand to which plots they apply.
- We have reorganized our `inline` arguments so they reflect the fact we have inputs grouped on two distinct lines.

Timeframe input

The `input.timeframe()` function creates a dropdown input containing *timeframe choices*. It returns a “string” value representing the selected timeframe in our specification format, which scripts can use in `request.*()` calls to retrieve data from user-selected timeframes.

The following script uses `request.security()` on each bar to fetch the value of a `ta.sma()` call from a user-specified higher timeframe, then plots the result on the chart:

```
//@version=6
indicator("Timeframe input demo", "MA", true)

//@variable The timeframe of the requested data.
```

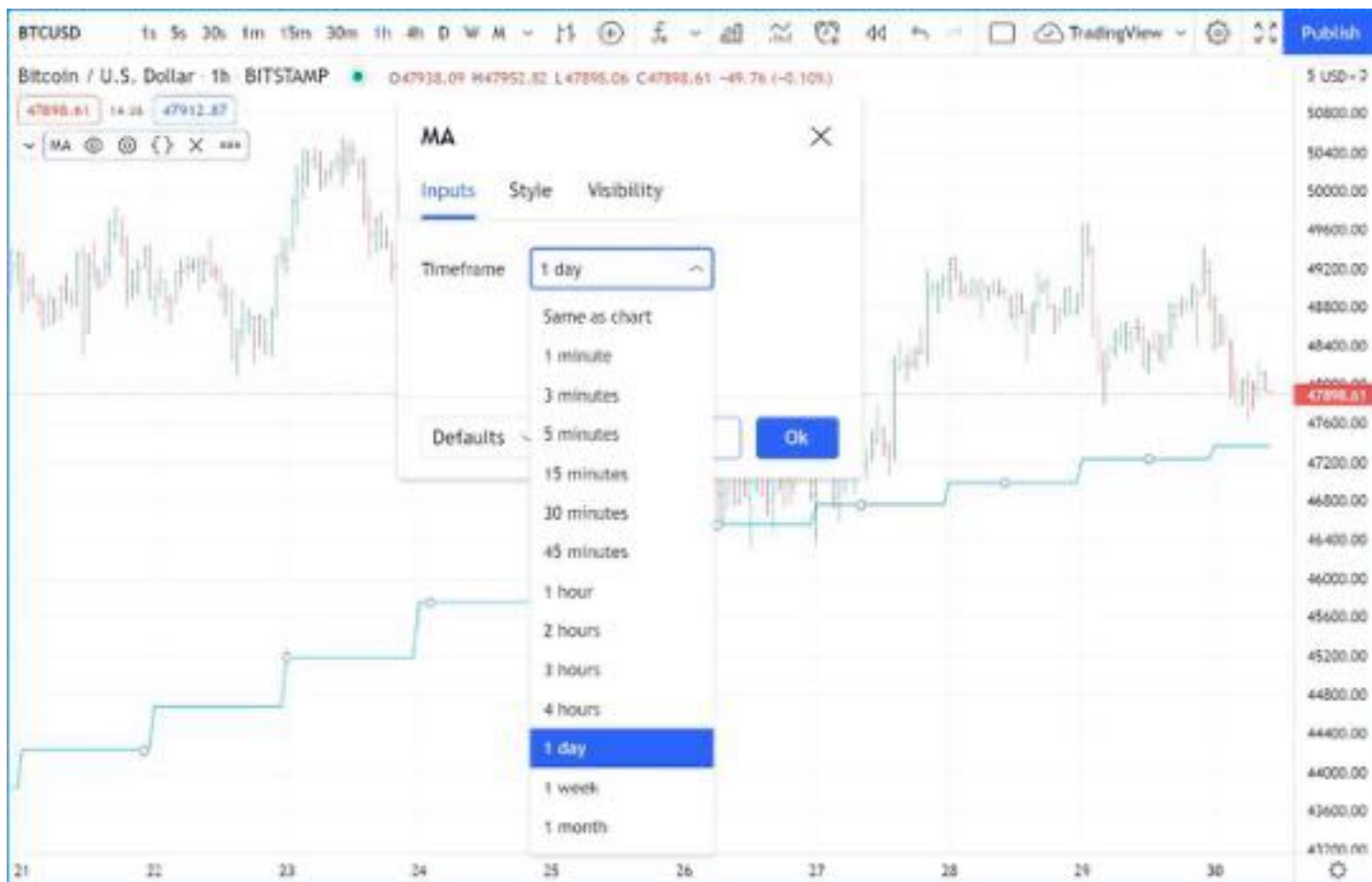


Figure 133: image


```

string tfInput = input.timeframe("1D", "Timeframe")

// Get the typical number of seconds in the chart's timeframe and the `tfInput` timeframe.
int chartSeconds = timeframe.in_seconds()
int tfSeconds    = timeframe.in_seconds(tfInput)
// Raise an error if the `tfInput` is a lower timeframe.
if tfSeconds < chartSeconds
    runtime.error("The 'Timeframe' input must represent a timeframe higher than or equal to the chart's.")

//@variable The offset of the requested expression. 1 when `tfInput` is a higher timeframe, 0 otherwise.
int offset = chartSeconds == tfSeconds ? 0 : 1
//@variable The 20-bar SMA of `close` prices for the current symbol from the `tfInput` timeframe.
float maHTF = request.security(syminfo.tickerid, tfInput, ta.sma(close, 20)[offset], lookahead = barmerge.look

// Plot the `maHTF` value.
plot(maHTF, "MA", color.aqua)

```

Note that:

- By default, the `input.timeframe()` call's dropdown contains options for the chart's timeframe and all timeframes listed in the chart's "Time interval" menu. To restrict the available options to specific preset timeframes, pass a tuple of timeframe strings to the function's `options` parameter.
- This script raises a runtime error if the estimated number of seconds in the `tfInput` timeframe is *less* than the number of seconds in the main timeframe, preventing it from requesting lower-timeframe data. See this section of the Other timeframes and data page to learn more.
- The `request.security()` call uses `barmerge.lookahead_on` as its `lookahead` argument, and it offsets the `expression` argument by one bar when the `tfInput` represents a *higher timeframe* to avoid repainting.

Symbol input

The `input.symbol()` function creates an input widget that mirrors the chart's "Symbol Search" widget. It returns a "string" *ticker identifier* representing the chosen symbol and exchange, which scripts can use in `request.*()` calls to retrieve data from other contexts.

The script below uses `request.security()` to retrieve the value of a `ta.rsi()` call evaluated on a user-specified symbol's prices. It plots the requested result on the chart in a separate pane:

```

//@version=6
indicator("Symbol input demo", "RSI")

//@variable The ticker ID of the requested data. By default, it is an empty "string", which specifies the main
string symbolInput = input.symbol("", "Symbol")

//@variable The 14-bar RSI of `close` prices for the `symbolInput` symbol on the script's main timeframe.
float symbolRSI = request.security(symbolInput, timeframe.period, ta.rsi(close, 14))

// Plot the `symbolRSI` value.
plot(symbolRSI, "RSI", color.aqua)

```

Note that:

- The `defval` argument in the `input.symbol()` call is an empty "string". When the `request.security()` call in this example uses this default value as the `symbol` argument, it calculates the RSI using the *chart symbol's* data. If the user wants to revert to the chart's symbol after choosing another symbol, they can select "Reset settings" from the "Defaults" dropdown at the bottom of the "Settings" menu.

Session input

Session inputs are useful to gather start-stop values for periods of time. The `input.session()` built-in function creates an input widget allowing users to specify the beginning and end time of a session. Selections can be made using a dropdown menu, or by entering time values in "hh:mm" format.

The value returned by `input.session()` is a valid string in session format. See the manual's page on sessions for more information.

Session information can also contain information on the days where the session is valid. We use an `input.string()` function call here to input that day information:

```
//@version=6
indicator("Session input", "", true)
string sessionInput = input.session("0600-1700", "Session")
string daysInput = input.string("1234567", tooltip = "1 = Sunday, 7 = Saturday")
sessionString = sessionInput + ":" + daysInput
inSession = not na(time(timeframe.period, sessionString))
bgcolor(inSession ? color.silver : na)
```

Note that:

- This script proposes a default session of “0600-1700”.
- The `input.string()` call uses a tooltip to provide users with help on the format to use to enter day information.
- A complete session string is built by concatenating the two strings the script receives as inputs.
- We explicitly declare the type of our two inputs with the `string` keyword to make it clear those variables will contain a string.
- We detect if the chart bar is in the user-defined session by calling `time()` with the session string. If the current bar’s time value (the time at the bar’s open) is not in the session, `time()` returns `na`, so `inSession` will be `true` whenever `time()` returns a value that is not `na`.

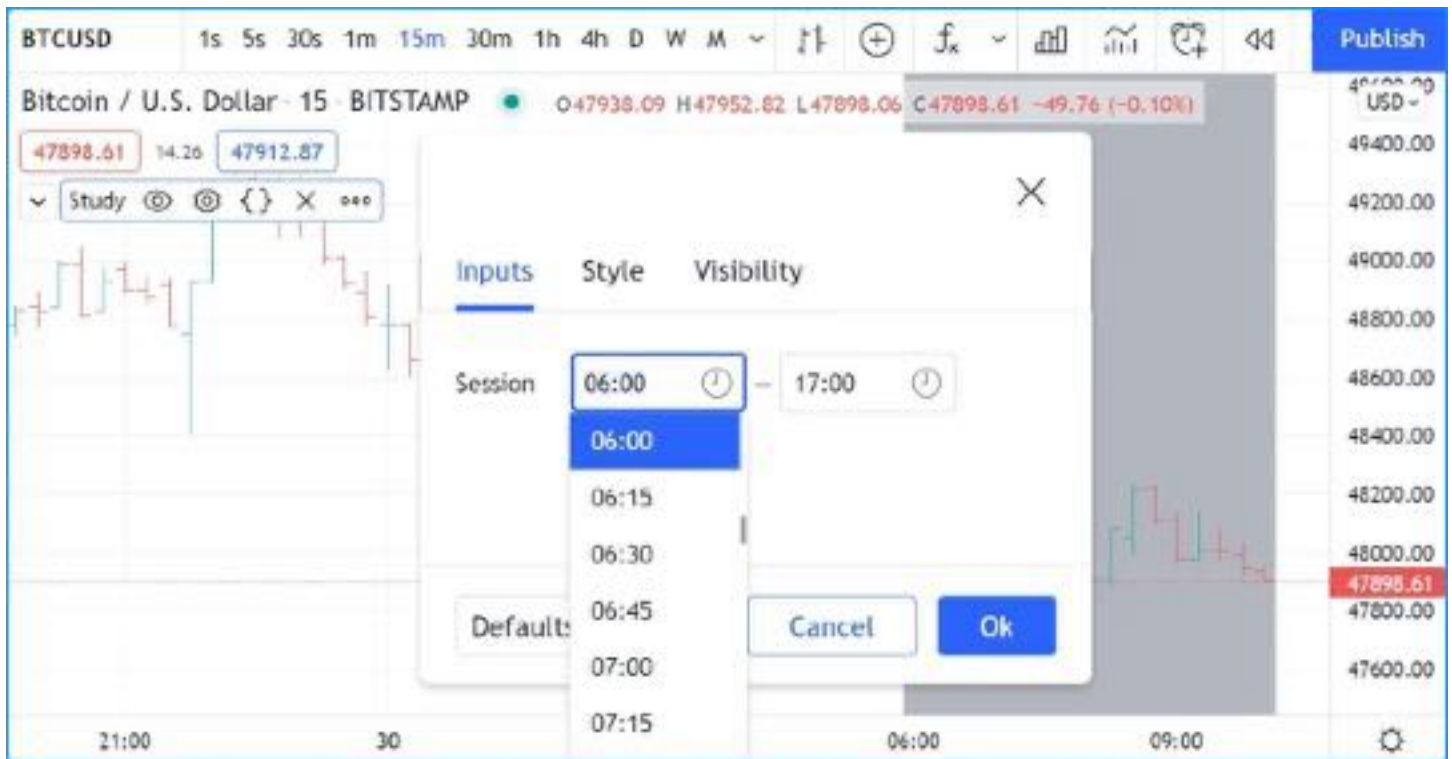


Figure 134: image

Source input

Source inputs are useful to provide a selection of two types of sources:

- Price values, namely: open, high, low, close, `hl2`, `hlc3`, and `ohlc4`.
- The values plotted by other scripts on the chart. This can be useful to “link” two or more scripts together by sending the output of one as an input to another script.

This script simply plots the user’s selection of source. We propose the high as the default value:

```
//@version=6
indicator("Source input", "", true)
srcInput = input.source(high, "Source")
plot(srcInput, "Src", color.new(color.purple, 70), 6)
```

This shows a chart where, in addition to our script, we have loaded an “Arnaud Legoux Moving Average” indicator. See [here](#) how we use our script’s source input widget to select the output of the ALMA script as an input into our script. Because our script plots that source in a light-purple thick line, you see the plots from the two scripts overlap because they plot the same value:

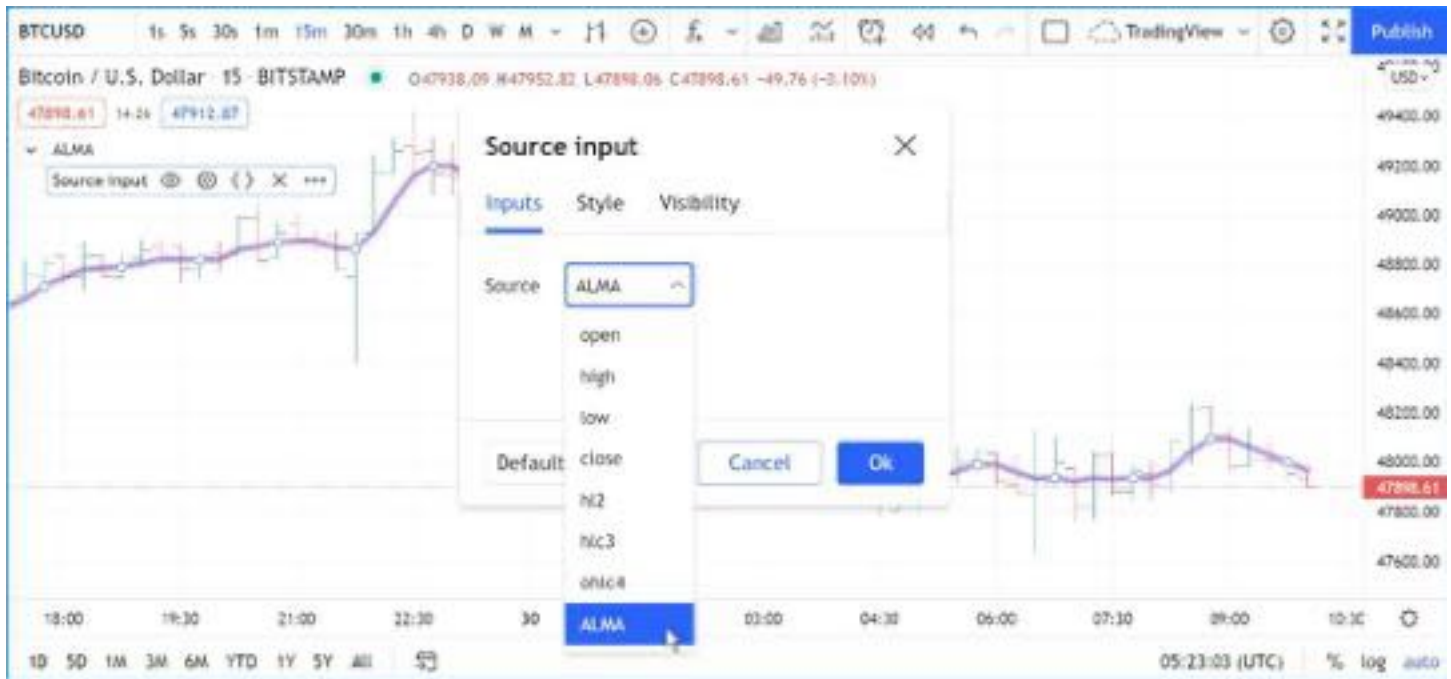


Figure 135: image

Time input

Time inputs use the `input.time()` function. The function returns a Unix time in milliseconds (see the [Time](#) page for more information). This type of data also contains date information, so the `input.time()` function returns a time **and** a date. That is the reason why its widget allows for the selection of both.

Here, we test the bar’s time against an input value, and we plot an arrow when it is greater:

```
//@version=6
indicator("Time input", "T", true)
timeAndDateInput = input.time(timestamp("1 Aug 2021 00:00 +0300"), "Date and time")
barIsLater = time > timeAndDateInput
plotchar(barIsLater, "barIsLater", " ", location.top, size = size.tiny)
```

Note that the `defval` value we use is a call to the `timestamp()` function.

Enum input

The `input.enum()` function creates a dropdown input that displays *field titles* corresponding to distinct *members* (possible values) of an enum type. The function returns one of the unique, named values from a declared enum, which scripts can use in calculations and logic requiring more strict control over allowed values and operations. Supply a list of enum members to the `options` parameter to specify the members users can select from the dropdown. If one does not specify an enum field’s title, its title is the “string” representation of its *name*.

This example declares a `SignalType` enum with four fields representing named signal display modes: `long`, `short`, `both`, and `none`. The script uses a member of this enum type as the `defval` argument in the `input.enum()` call to generate a dropdown in the “Inputs” tab, allowing users to select one of the enum’s titles to control which signals it displays on the chart:

```
//@version=6
indicator("Enum input demo", overlay = true)

//@enum          An enumeration of named values representing signal display modes.
//@field long     Named value to specify that only long signals are allowed.
```



Figure 136: image

```
//@field short    Named value to specify that only short signals are allowed.
//@field both     Named value to specify that either signal type is allowed.
//@field none     Named value to specify that no signals are allowed.
enum SignalType
    long  = "Only long signals"
    short = "Only short signals"
    both  = "Long and short signals"
    none

//@variable An enumerator (member) of the `SignalType` enum. Controls the script's signals.
SignalType sigInput = input.enum(SignalType.long, "Signal type")

// Calculate moving averages.
float ma1 = ta.sma(ohlc4, 10)
float ma2 = ta.sma(ohlc4, 200)
// Calculate cross signals.
bool longCross  = ta.crossover(close, math.max(ma1, ma2))
bool shortCross = ta.crossunder(close, math.min(ma1, ma2))
// Calculate long and short signals based on the selected `sigInput` value.
bool longSignal = (sigInput == SignalType.long or sigInput == SignalType.both) and longCross
bool shortSignal = (sigInput == SignalType.short or sigInput == SignalType.both) and shortCross

// Plot shapes for the `longSignal` and `shortSignal`.
plotshape(longSignal, "Long signal", shape.triangleup, location.belowbar, color.teal, size = size.normal)
plotshape(shortSignal, "Short signal", shape.triangledown, location.abovebar, color.maroon, size = size.normal)
// Plot the moving averages.
plot(ma1, "Fast MA")
plot(ma2, "Slow MA")
```

Note that:

- The `sigInput` value is the `SignalType` member whose field contains the selected title.
- Since we did not specify a title for the `none` field of the enum, its title is the “string” representation of its name (“none”), as we see in the above image of the enum input’s dropdown.

By default, an enum input displays the titles of all an enum’s members within its dropdown. If we supply an `options` argument to the `input.enum()` call, it will only allow users to select the members included in that list, e.g.:

```
SignalType sigInput = input.enum(SignalType.long, "Signal type", options = [SignalType.long, SignalType.short])
```

The above `options` argument specifies that users can only view and select the titles of the `long` and `short` fields from the `SignalType` enum. No other options are allowed:



Figure 137: image

Other features affecting Inputs

Some parameters of the `indicator()` and `strategy()` functions populate a script's "Settings/Inputs" tab with additional inputs. These parameters are `timeframe`, `timeframe_gaps`, and `calc_bars_count`. For example:

```
//@version=6
indicator("MA", "", true, timeframe = "D", timeframe_gaps = false)
plot(ta.vwma(close, 10))
```

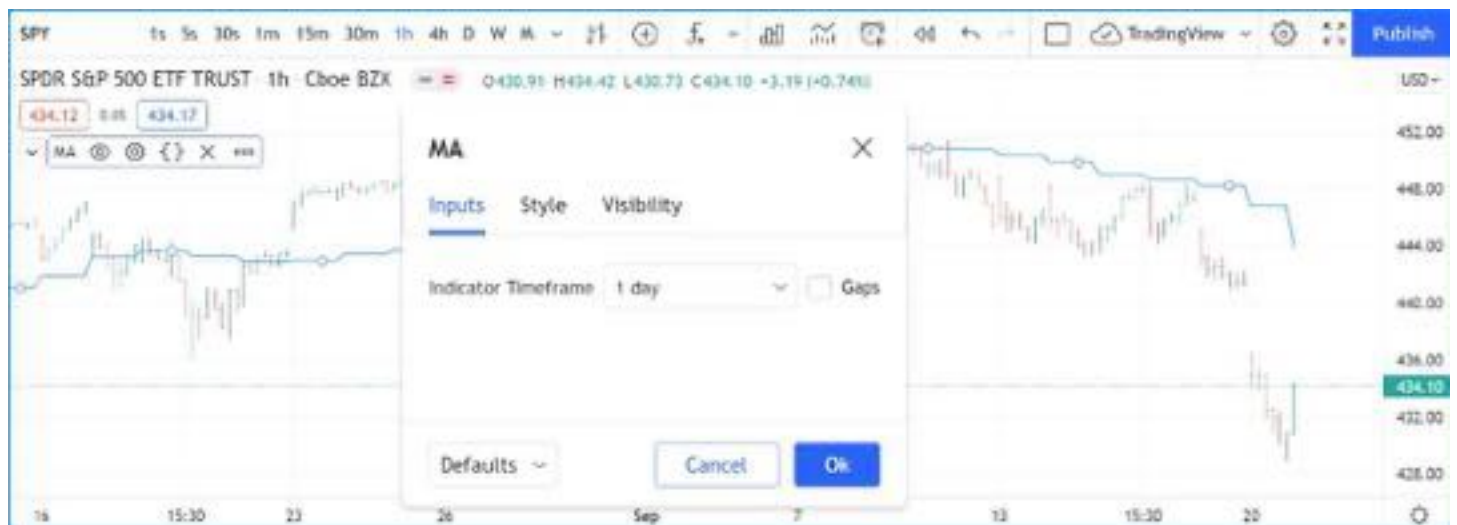


Figure 138: image

Tips

The design of your script's inputs has an important impact on the usability of your scripts. Well-designed inputs are more intuitively usable and make for a better user experience:

- Choose clear and concise labels (your input's `title` argument).
- Choose your default values carefully.

- Provide **minval** and **maxval** values that will prevent your code from producing unexpected results, e.g., limit the minimal value of lengths to 1 or 2, depending on the type of MA you are using.
- Provide a **step** value that is congruent with the value you are capturing. Steps of 5 can be more useful on a 0-200 range, for example, or steps of 0.05 on a 0.0-1.0 scale.
- Group related inputs on the same line using **inline**; bull and bear colors for example, or the width and color of a line.
- When you have many inputs, group them into meaningful sections using **group**. Place the most important sections at the top.
- Do the same for individual inputs **within** sections.

It can be advantageous to vertically align different arguments of multiple **input.*()** calls in your code. When you need to make global changes, this will allow you to use the Editor's multi-cursor feature to operate on all the lines at once.

Because It is sometimes necessary to use Unicode spaces to In order to achieve optimal alignment in inputs. This is an example:

```
//@version=6
indicator("Aligned inputs", "", true)

var GRP1 = "Not aligned"
ma1SourceInput = input(close, "MA source", inline = "11", group = GRP1)
ma1LengthInput = input(close, "Length", inline = "11", group = GRP1)
long1SourceInput = input(close, "Signal source", inline = "12", group = GRP1)
long1LengthInput = input(close, "Length", inline = "12", group = GRP1)

var GRP2 = "Aligned"
// The three spaces after "MA source" are Unicode EN spaces (U+2002).
ma2SourceInput = input(close, "MA source  ", inline = "21", group = GRP2)
ma2LengthInput = input(close, "Length", inline = "21", group = GRP2)
long2SourceInput = input(close, "Signal source", inline = "22", group = GRP2)
long2LengthInput = input(close, "Length", inline = "22", group = GRP2)

plot(ta.vwma(close, 10))
```

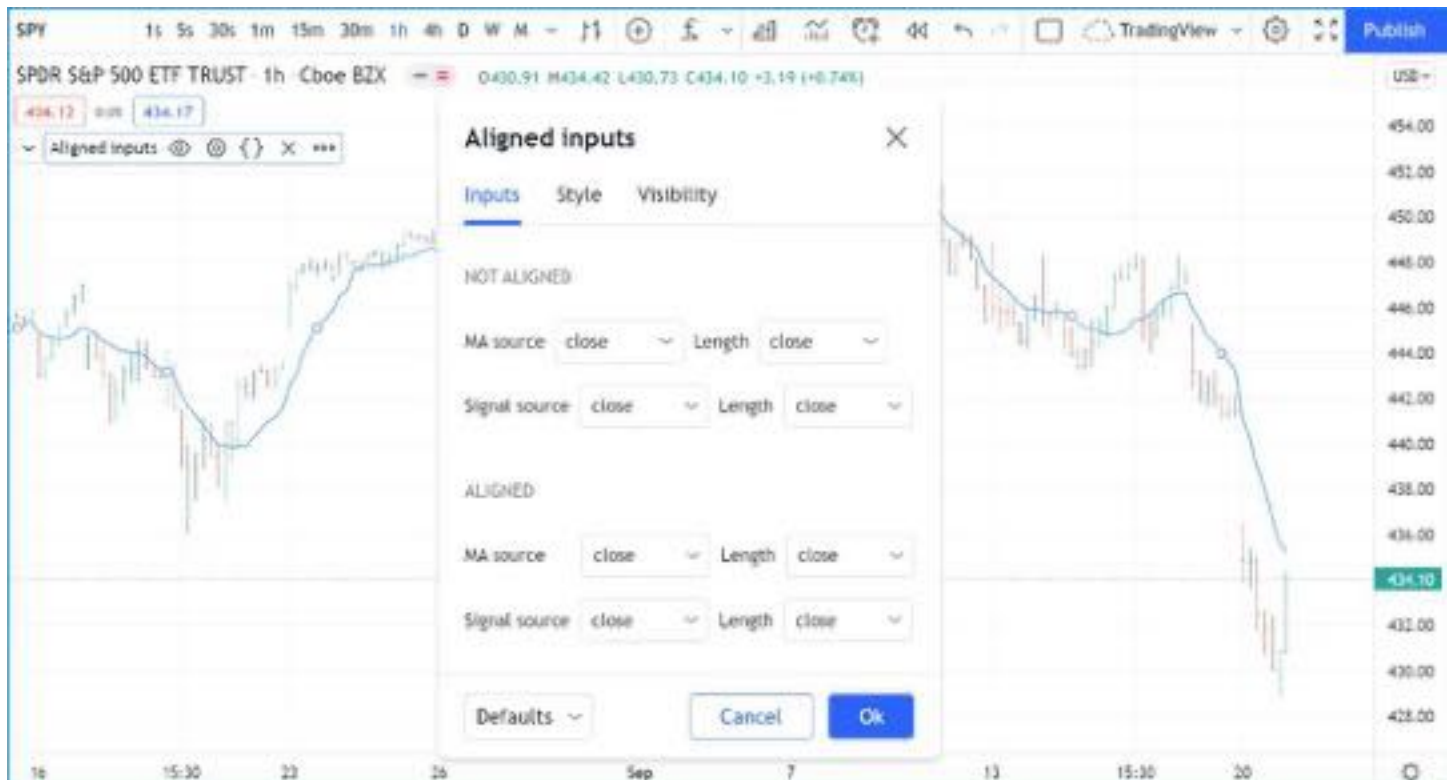


Figure 139: image

Note that:

- We use the **group** parameter to distinguish between the two sections of inputs. We use a constant to hold the name of the groups. This way, if we decide to change the name of the group, we only need to change it in one place.
- The first sections inputs widgets do not align vertically. We are using **inline**, which places the input widgets immediately to the right of the label. Because the labels for the **ma1SourceInput** and **long1SourceInput** inputs are of different lengths the labels are in different *y* positions.
- To make up for the misalignment, we pad the **title** argument in the **ma2SourceInput** line with three Unicode EN spaces (U+2002). Unicode spaces are necessary because ordinary spaces would be stripped from the label. You can achieve precise alignment by combining different quantities and types of Unicode spaces. See here for a list of Unicode spaces of different widths.

[Previous

Fills](#fills)[Next

Levels](#levels) User Manual/Concepts/Levels

Levels

hline() levels

Levels are lines plotted using the **hline()** function. It is designed to plot **horizontal** levels using a **single color**, i.e., it does not change on different bars. See the Levels section of the page on **plot()** for alternative ways to plot levels when **hline()** won't do what you need.

The function has the following signature:

```
hline(price, title, color, linestyle, linewidth, editable) → hline
```

hline() has a few constraints when compared to **plot()**:

- Since the function's objective is to plot horizontal lines, its **price** parameter requires an "input int/float" argument, which means that "series float" values such as **close** or dynamically-calculated values cannot be used.
- Its **color** parameter requires an "input int" argument, which precludes the use of dynamic colors, i.e., colors calculated on each bar — or "series color" values.
- Three different line styles are supported through the **linestyle** parameter: **hline.style_solid**, **hline.style_dotted** and **hline.style_dashed**.

Let's see **hline()** in action in the "True Strength Index" indicator:

```
//@version=6
indicator("TSI")
myTSI = 100 * ta.tsi(close, 25, 13)

hline( 50, "+50", color.lime)
hline( 25, "+25", color.green)
hline( 0, "Zero", color.gray, linestyle = hline.style_dotted)
hline(-25, "-25", color.maroon)
hline(-50, "-50", color.red)

plot(myTSI)
```



Figure 140: image

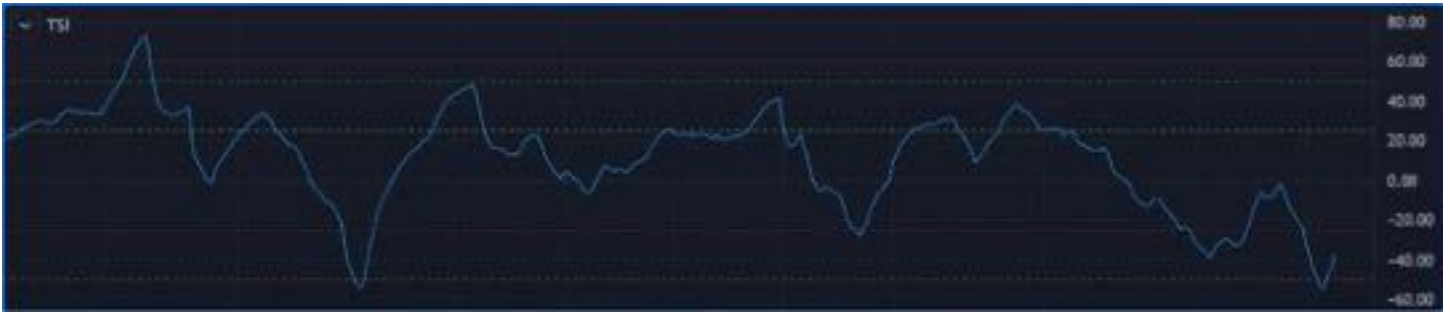


Figure 141: image

Note that:

- We display 5 levels, each of a different color.
- We use a different line style for the zero centerline.
- We choose colors that will work well on both light and dark themes.
- The usual range for the indicator's values is +100 to -100. Since the `ta.tsi()` built-in returns values in the +1 to -1 range, we make the adjustment in our code.

Fills between levels

The space between two levels plotted with `hline()` can be colored using `fill()`. Keep in mind that **both** plots must have been plotted with `hline()`.

Let's put some background colors in our TSI indicator:

```
//@version=6
indicator("TSI")
myTSI = 100 * ta.tsi(close, 25, 13)

plus50Hline = hline( 50, "+50", color.lime)
plus25Hline = hline( 25, "+25", color.green)
zeroHline   = hline( 0, "Zero", color.gray, linestyle = hline.style_dotted)
minus25Hline = hline(-25, "-25", color.maroon)
minus50Hline = hline(-50, "-50", color.red)

// ----- Function returns a color in a light shade for use as a background.
fillColor(color col) =>
    color.new(col, 90)

fill(plus50Hline, plus25Hline, fillColor(color.lime))
fill(plus25Hline, zeroHline, fillColor(color.teal))
fill(zeroHline, minus25Hline, fillColor(color.maroon))
fill(minus25Hline, minus50Hline, fillColor(color.red))

plot(myTSI)
```



Figure 142: image

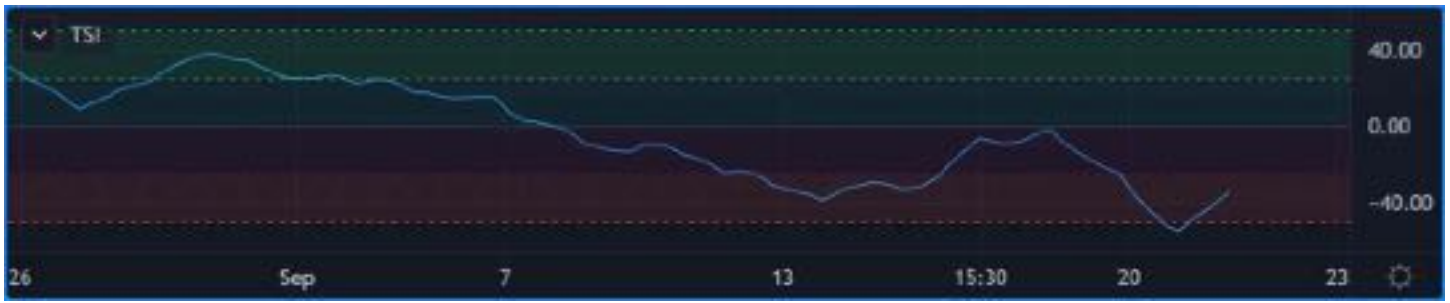


Figure 143: image

Note that:

- We have now used the return value of our `hline()` function calls, which is of the `hline` special type. We use the `plus50Hline`, `plus25Hline`, `zeroHline`, `minus25Hline` and `minus50Hline` variables to store those “hline” IDs because we will need them in our `fill()` calls later.
- To generate lighter color shades for the background colors, we declare a `fillColor()` function that accepts a color and returns its 90 transparency. We use calls to that function for the `color` arguments in our `fill()` calls.
- We make our `fill()` calls for each of the four different fills we want, between four different pairs of levels.
- We use `color.teal` in our second fill because it produces a green that fits the color scheme better than the `color.green` used for the 25 level.

[Previous

Inputs](#inputs)[Next

Libraries](#libraries) User Manual/Concepts/Libraries

Libraries

Introduction

Pine Script™ libraries are publications containing functions that can be reused in indicators, strategies, or in other libraries. They are useful to define frequently-used functions so their source code does not have to be included in every script where they are needed.

A library must be published (privately or publicly) before it can be used in another script. All libraries are published open-source. Public scripts can only use public libraries and they must be open-source. Private scripts or personal scripts saved in the Pine Script™ Editor can use public or private libraries. A library can use other libraries, or even previous versions of itself.

Library programmers should be familiar with Pine’s typing nomenclature, scopes, and user-defined functions. For more information, see the User Manual’s pages on the Type system and User-defined functions.

You can browse public library scripts in the Community Scripts feed.

Creating a library

A library is a special kind of script that begins with the `library()` declaration statement, rather than `indicator()` or `strategy()`. A library contains exportable function, method, UDT, and enum definitions, which constitute the only visible part of the library when imported by another script. Like other script types, libraries can also include Pine Script™ code in their global scopes. Programmers typically use a library’s global code to demonstrate how other scripts can use its exported structures.

A library script has a structure like the following, which must include one or more exportable functions or types:

```
//@version=6

// @description <library_description>
library(title, overlay)

<script_code>
```

```

//@type <type_description>
//@field <field_name> <field_description>
// ...
export type <UDT_identifier>
    <field_type> <field_name>[ = <value>]
    ...

//@enum <enum_description>
//@field <field_name> <field_description>
// ...
export enum <enum_name>
    <field_name>[ = <field_title>]
    ...

//@function <function_description>
//@param <parameter> <parameter_description>
//@returns <return_value_description>
export <function_name>([simple/series] <parameter_type> <parameter_name> [= <default_value>] [, ...]) =>
    <function_code>

<script_code>

```

Note that:

- The `//@description`, `//@enum`, `//@type`, `@field`, `// @function`, `// @param`, and `// @returns` compiler annotations are optional but we highly recommend you use them. These annotations document the library's code and populate the default library description, which authors can use when publishing the library.
- The `export` keyword is mandatory.
- is mandatory, contrary to user-defined function parameter definitions in indicators or strategies, which are typeless.
- can be any code one would normally use in an indicator, including inputs.

This is an example library:

```

//@version=6

// @description Provides functions calculating the all-time high/low of values.
library("AllTimeHighLow", true)

// @function Calculates the all-time high of a series.
// @param val Series to use (`high` is used if no argument is supplied).
// @returns The all-time high for the series.
export hi(float val = high) =>
    var float ath = val
    ath := math.max(ath, val)

// @function Calculates the all-time low of a series.
// @param val Series to use (`low` is used if no argument is supplied).
// @returns The all-time low for the series.
export lo(float val = low) =>
    var float atl = val
    atl := math.min(atl, val)

plot(hi())
plot(lo())

```

Library functions

Exported functions and methods have slightly different requirements and constraints compared to non-exported functions.

In exported library function signatures (their first line):

- The `export` is mandatory.

- The function’s signature must include type keywords to specify the required type for each parameter.
- Programmers can include either the simple or series qualifier keywords to specify the qualified type each parameter accepts. See the next section for more information.

Exported library functions have the following constraints:

- They cannot use variables from the library’s global scope except for those with the “const” qualifier, meaning they cannot use global variables initialized from script inputs, for example, or globally declared arrays.
- They cannot include calls to any `input.*()` functions.
- They *can* include `request.*()` calls in their local scopes (if the `dynamic_requests` parameter is not set to `false` in the library declaration statement), but the `expression` arguments of these calls cannot depend on any exported function parameters. See this section of the Other timeframes and data page to learn more about dynamic requests.

Library functions always return “simple” or “series” results. Consequently, scripts cannot use their returned values in locations requiring “const” or “input” values. For example, a library function cannot calculate an argument for the `show_last` parameter in a `plot()` call because the parameter requires an “input int” qualified type.

Qualified type control

The qualified types of arguments supplied in calls to library functions are autodetected based on how each argument is used inside the function. If the argument can be used as a “series”, it is qualified as such. If it cannot, an attempt is made with the “simple” type qualifier. This explains why this code:

```
export myEma(int x) =>
    ta.ema(close, x)
```

will work when called using `myCustomLibrary.myEma(20)`, even though `ta.ema()`’s `length` parameter requires a “simple int” argument. When the Pine Script™ compiler detects that a “series” length cannot be used with `ta.ema()`, it tries the “simple” qualifier, which in this case is allowed.

While library functions cannot return “const” or “input” values, they can be written to produce “simple” results. This makes them useful in more contexts than functions returning “series” results, as some built-in functions do not allow “series” arguments. For example, `request.security()` requires a “simple string” argument for its `symbol` parameter when a script does not allow dynamic requests. If we wrote a library function to assemble the argument to `symbol` in the following way, the function’s result would not work with a non-dynamic `request.*()` call because it is of the “series string” qualified type:

```
export makeTickerid(string prefix, string ticker) =>
    prefix + ":" + ticker
```

However, by restricting the parameter qualifiers to “simple”, we can force the function to yield a “simple” result. We can achieve this by prefixing the parameters’ type with the simple keyword:

```
export makeTickerid(simple string prefix, simple string ticker) =>
    prefix + ":" + ticker
```

Note that for the function to return a “simple” value, no “series” values can be used in its calculation; otherwise the result will be a “series” value.

One can also use the series keyword to prefix the type of a library function parameter. However, because arguments are qualified as “series” by default, using the series modifier is redundant.

User-defined types and objects

Libraries can export user-defined types (UDTs), and library functions can return objects of these types.

To export a UDT, prefix its definition with the `export` keyword, similar to exporting a function:

```
//@version=6
library("Point")

export type point
    int x
    float y
    bool isHi
    bool wasBreached = false
```

A script importing that library and creating an object of its `point` UDT would look somewhat like this:

```
//@version=6
indicator("")
import userName/Point/1 as pt
newPoint = pt.point.new()
```

Note that:

- This code won't compile because no "Point" library is published, and the script doesn't display anything.
- `userName` would need to be replaced by the TradingView user name of the library's publisher.
- We use the built-in `new()` method to create an object from the `point` UDT.
- We prefix the reference to the library's `point` UDT with the `pt` alias defined in the import statement, just like we would when using a function from an imported library.

A library **must** export a UDT if any exported functions or methods accept or return an object of that type, or if the fields of another exported UDT accept an instance of the type.

When a library only uses a UDT internally, it does not need to export the type. The following library uses the `point` type internally, but it only exports the `drawPivots()` function, which does not have a parameter of the `point` type or return a `pointobject`:

```
//@version=6
library("PivotLabels", true)

// We use this `point` UDT in the library, but it does NOT require exporting because
// 1. The exported function's parameters do not use the UDT.
// 2. The exported function does not return a UDT result.
type point
  int x
  float y
  bool isHi
  bool wasBreached = false

fillPivotsArray(qtyLabels, leftLegs, rightLegs) =>
  // Create an array of the specified qty of pivots to maintain.
  var pivotsArray = array.new<point>(math.max(qtyLabels, 0))

  // Detect pivots.
  float pivotHi = ta.pivohigh(leftLegs, rightLegs)
  float pivotLo = ta.pivotlow(leftLegs, rightLegs)

  // Create a new `point` object when a pivot is found.
  point foundPoint = switch
    not na(pivotHi) => point.new(time[rightLegs], pivotHi, true)
    not na(pivotLo) => point.new(time[rightLegs], pivotLo, false)
    => na

  // Add new pivot info to the array and remove the oldest pivot.
  if not na(foundPoint)
    array.push(pivotsArray, foundPoint)
    array.shift(pivotsArray)

  array<point> result = pivotsArray

detectBreaches(pivotsArray) =>
  // Detect breaches.
  for [i, eachPoint] in pivotsArray
    if not na(eachPoint)
      if not eachPoint.wasBreached
        bool hiWasBreached = eachPoint.isHi and high[1] <= eachPoint.y and high > eachPoint.y
        bool loWasBreached = not eachPoint.isHi and low[1] >= eachPoint.y and low < eachPoint.y
        if hiWasBreached or loWasBreached
          // This pivot was breached; change its `wasBreached` field.
```

```

        point p = array.get(pivotsArray, i)
        p.wasBreached := true
        array.set(pivotsArray, i, p)

drawLabels(pivotsArray) =>
    for eachPoint in pivotsArray
        if not na(eachPoint)
            label.new(
                eachPoint.x,
                eachPoint.y,
                str.toString(eachPoint.y, format.mintick),
                xloc.bar_time,
                color = eachPoint.wasBreached ? color.gray : eachPoint.isHi ? color.teal : color.red,
                style = eachPoint.isHi ? label.style_label_down: label.style_label_up,
                textcolor = eachPoint.wasBreached ? color.silver : color.white)

// @function      Displays a label for each of the last `qtyLabels` pivots.
//               Colors high pivots in green, low pivots in red, and breached pivots in gray.
// @param qtyLabels (simple int) Quantity of last labels to display.
// @param leftLegs  (simple int) Left pivot legs.
// @param rightLegs (simple int) Right pivot legs.
// @returns        Nothing.
export drawPivots(int qtyLabels, int leftLegs, int rightLegs) =>
    // Gather pivots as they occur.
    pointsArray = fillPivotsArray(qtyLabels, leftLegs, rightLegs)

    // Mark breached pivots.
    detectBreaches(pointsArray)

    // Draw labels once.
    if barstate.islastconfirmedhistory
        drawLabels(pointsArray)

// Example use of the function.
drawPivots(20, 10, 5)

```

If the TradingView user published the above library, it could be used like this:

```

//@version=6
indicator("")
import TradingView/PivotLabels/1 as dpl
dpl.drawPivots(20, 10, 10)

```

Enum types

Libraries can also export enum types, allowing other scripts to import sets of predefined, named values that help control the values accepted by variables, conditional expressions, and collections.

For example, this library exports a **State** enum with three fields representing distinct signal states: **long**, **short**, and **neutral**. These fields represent the *possible values* a variable, expression, or collection of the enum type can take on:

```

//@version=6
library("Signal")

//@enum      An enumeration of named signal states.
//@field long    Represents a "Long" signal.
//@field short   Represents a "Short" signal.
//@field neutral Represents a "Neutral" signal.
export enum State
    long    = "Long"
    short   = "Short"
    neutral = "Neutral"

```

A script that imports this library can use the members (values) of the `State` enum as named states in its logic. Here, we show a simple, hypothetical script that imports the “Signal” library published by the `userName` user and uses the `Signal.State` enum to assign one of three possible values to a `mySignal` variable:

```
//@version=6
indicator("")

import userName/Signal/1 as Signal

// Calculate the median and quarter range values.
float medianValue = ta.median(close, 100)
float rangeValue  = ta.range(close, 100) * 0.25
// Calculate upper and lower channel values.
float upper = medianValue + rangeValue
float lower = medianValue - rangeValue

//@variable Returns `Signal.State.long`, `Signal.State.short`, or `Signal.State.neutral` based on the price action
Signal.State mySignal = switch
    close > upper => Signal.State.long
    close < lower => Signal.State.short
    =>             Signal.State.neutral

plot(close, color = mySignal == Signal.State.long ? color.green : mySignal == Signal.State.short ? color.red : color.gray)
```

Similar to exporting UDTs, a library **must** export an enum when its exported functions or methods accept or return the enum’s members, or when the fields of an exported UDT accept values of that enum type.

Publishing a library

Before you or other Pine Script™ programmers can reuse any library, it must be published. If you want to share your library with all TradingView users, publish it publicly. To use it privately, use a private publication. As with indicators or strategies, the active chart when you publish a library will appear in both its widget (the small placeholder denoting libraries in the TradingView scripts stream) and script page (the page users see when they click on the widget).

Private libraries can be used in public Protected or Invite-only scripts.

After adding our example library to the chart and setting up a clean chart showing our library plots the way we want them, we use the Pine Editor’s “Publish Script” button. The “Publish Library” window comes up:

Note that:

- We leave the library’s title as is (the `title` argument in our `library()` declaration statement is used as the default). While you can change the publication’s title, it is preferable to keep its default value because the `title` argument is used to reference imported libraries in the import statement. It makes life easier for library users when your publication’s title matches the actual name of the library.
- A default description is built from the compiler annotations we used in our library. We will publish the library without retouching it.
- We chose to publish our library publicly, so it will be visible to all TradingView users.
- We do not have the possibility of selecting a visibility type other than “Open” because libraries are always open-source.
- The list of categories for libraries is different than for indicators and strategies. We have selected the “Statistics and Metrics” category.
- We have added some custom tags: “all-time”, “high” and “low”.

The intended users of public libraries being other Pine programmers; the better you explain and document your library’s functions, the more chances others will use them. Providing examples demonstrating how to use your library’s functions in your publication’s code will also help.

House Rules

Pine libraries are considered “public domain” code in our House Rules on Script Publishing, which entails that permission is not required from their author if you call their functions or reuse their code in your open-source scripts. However, if you intend to reuse code from a Pine Script™ library’s functions in a public protected or invite-only publication, explicit permission for reuse in that form is required from its author.

Publish Library

Library title and description

AllTimeHighLow

I B [icon] [icon] [icon]

Library [b] AllTimeHighLow [/b]
Provides functions calculating the all-time high/low of values.

[b]hi([val])[/b] Calculates the all-time high of a series.
Parameters:
[list] [""] [b]val[/b]: Series to use ('high' is used if no argument is supplied). [/list]
Returns: The all-time high for the series.

[b]lo([val])[/b] Calculates the all-time low of a series.
Parameters:
[list] [""] [b]val[/b]: Series to use ('low' is used if no argument is supplied). [/list]
Returns: The all-time low for the series.

Privacy settings ⓘ

Public

Private

The library will be visible to all. It'll be listed on public pages (homepage, your profile).

Visibility ⓘ

Open

Protected

Invite-only

Source code of this library will be open.

Category

Statistics and Metrics X -

Select up to 3 categories that best describe your market analysis approach.

Tags

all-time X high X low X |

Publish Public Library

Figure 144: image

Whether using a library's functions or reusing its code, you must credit the author in your publication's description. It is also good form to credit in open-source comments.

Using a library

Using a library from another script (which can be an indicator, a strategy or another library), is done through the import statement:

```
import <username>/<libraryName>/<libraryVersion> [as <alias>]
```

where:

- The `//` path will uniquely identify the library.
- The must be specified explicitly. To ensure the reliability of scripts using libraries, there is no way to automatically use the latest version of a library. Every time a library update is published by its author, the library's version number increases. If you intend to use the latest version of the library, the value will require updating in the import statement.
- The `as <alias>` part is optional. When used, it defines the namespace that will refer to the library's functions. For example, if you import a library using the `allTime` alias as we do in the example below, you will refer to that library's functions as `allTime.<function_name>()`. When no alias is defined, the library's name becomes its namespace.

To use the library we published in the previous section, our next script will require an import statement:

```
import PineCoders/AllTimeHighLow/1 as allTime
```

As you type the user name of the library's author, you can use the Editor's `ctrl + space / cmd` "Auto-complete" command to display a popup providing selections that match the available libraries:



Figure 145: image

This is an indicator that reuses our library:

```
//@version=6
indicator("Using AllTimeHighLow library", "", true)
import PineCoders/AllTimeHighLow/1 as allTime

plot(allTime.hi())
plot(allTime.lo())
plot(allTime.hi(close))
```

Note that:

- We have chosen to use the "allTime" alias for the library's instance in our script. When typing that alias in the Editor, a popup will appear to help you select the particular function you want to use from the library.
- We use the library's `hi()` and `lo()` functions without an argument, so the default high and low built-in variables will be used for their series, respectively.
- We use a second call to `allTime.hi()`, but this time using `close` as its argument, to plot the highest close in the chart's history.

[Previous

Levels](#levels)[Next

Lines and boxes](#lines-and-boxes) User Manual/Concepts/Lines and boxes

Lines and boxes

Introduction

Pine Script™ facilitates drawing lines, boxes, and other geometric formations from code using the `line`, `box`, and `polyline` types. These types provide utility for programmatically drawing support and resistance levels, trend lines, price ranges, and other custom formations on a chart.

Unlike plots, the flexibility of these types makes them particularly well-suited for visualizing current calculated data at virtually any available point on the chart, irrespective of the chart bar the script executes on.

Lines, boxes, and polylines are *objects*, like labels, tables, and other *special types*. Scripts reference objects of these types using IDs, which act like *pointers*. As with other objects, line, box, and polyline IDs are qualified as “series” values, and all functions that manage these objects accept “series” arguments.

Lines drawn by a script may be vertical, horizontal, or angled. Boxes are always rectangular. Polylines sequentially connect multiple vertical, horizontal, angled, or curved line segments. Although all of these drawing types have different characteristics, they do have some things in common:

- Lines, boxes, and polylines can have coordinates at any available location on the chart, including ones at future times beyond the last chart bar.
- Objects of these types can use `chart.point` instances to set their coordinates.
- The x-coordinates of each object can be bar index or time values, depending on their specified `xloc` property.
- Each object can have one of multiple predefined line styles.
- Scripts can call the functions that manage these objects from within the scopes of loops and conditional structures, allowing iterative and conditional control of their drawings.
- There are limits on the number of these objects that a script can reference and display on the chart. A single script instance can display up to 500 lines, 500 boxes, and 100 polylines. Users can specify the maximum number allowed for each type via the `max_lines_count`, `max_boxes_count`, and `max_polylines_count` parameters of the script’s `indicator()` or `strategy()` declaration statement. If unspecified, the default is ~50. As with label and table types, lines, boxes, and polylines utilize a *garbage collection* mechanism that deletes the oldest objects on the chart when the total number of drawings exceeds the script’s limit.

Lines

The built-ins in the `line.*` namespace control the creation and management of line objects:

- The `line.new()` function creates a new line.
- The `line.set_*` functions modify line properties.
- The `line.get_*` functions retrieve values from a line instance.
- The `line.copy()` function clones a line instance.
- The `line.delete()` function deletes an existing line instance.
- The `line.all` variable references a read-only array containing the IDs of all lines displayed by the script. The array’s size depends on the `max_lines_count` of the `indicator()` or `strategy()` declaration statement and the number of lines the script has drawn.

Scripts can call `line.set_*`, `line.get_*`, `line.copy()`, and `line.delete()` built-ins as functions or methods.

Creating lines

The `line.new()` function creates a new line instance to display on the chart. It has the following signatures:

```
line.new(first_point, second_point, xloc, extend, color, style, width, force_overlay) → series line
line.new(x1, y1, x2, y2, xloc, extend, color, style, width, force_overlay) → series line
```

The first overload of this function contains the `first_point` and `second_point` parameters. The `first_point` is a `chart.point` representing the start of the line, and the `second_point` is a `chart.point` representing the line’s end. The function copies the information from these chart points to determine the line’s coordinates. Whether it uses the `index` or `time` fields from the `first_point` and `second_point` as x-coordinates depends on the function’s `xloc` value.

The second overload specifies `x1`, `y1`, `x2`, and `y2` values independently, where `x1` and `x2` are int values representing the starting and ending x-coordinates of the line, and `y1` and `y2` are float values representing the y-coordinates. Whether the line considers the `x` values as bar indices or timestamps depends on the `xloc` value in the function call.

Both overloads share the same additional parameters:

xloc

Controls whether the x-coordinates of the new line use bar index or time values. Its default value is `xloc.bar_index`.

When calling the first overload, using an `xloc` value of `xloc.bar_index` tells the function to use the `index` fields of the `first_point` and `second_point`, and a value of `xloc.bar_time` tells the function to use the `time` fields of the points.

When calling the second overload, an `xloc` value of `xloc.bar_index` prompts the function to treat the `x1` and `x2` arguments as bar index values. When using `xloc.bar_time`, the function will treat `x1` and `x2` as time values.

When the specified x-coordinates represent *bar index* values, it's important to note that the minimum x-coordinate allowed is `bar_index - 9999`. For larger offsets, one can use `xloc.bar_time`.

extend

Determines whether the drawn line will infinitely extend beyond its defined start and end coordinates. It accepts one of the following values: `extend.left`, `extend.right`, `extend.both`, or `extend.none` (default).

color

Specifies the color of the line drawing. The default is `color.blue`.

style

Specifies the line's style, which can be any of the options listed in this page's Line styles section. The default value is `line.style_solid`.

width

Controls the width of the line, in pixels. The default value is 1.

force_overlay

If `true`, the drawing will display on the main chart pane, even when the script occupies a separate pane. Optional. The default is `false`.

The example below demonstrates how one can draw lines in their simplest form. This script draws a new vertical line connecting the open and close prices at the horizontal center of each chart bar:



Figure 146: image

```
//@version=6
indicator("Creating lines demo", overlay = true)

//@variable The `chart.point` for the start of the line. Contains `index` and `time` information.
firstPoint = chart.point.now(open)
//@variable The `chart.point` for the end of the line. Contains `index` and `time` information.
secondPoint = chart.point.now(close)
```

```
// Draw a basic line with a `width` of 5 connecting the `firstPoint` to the `secondPoint`.
// This line uses the `index` field from each point for its x-coordinates.
line.new(firstPoint, secondPoint, width = 5)

// Color the background on the unconfirmed bar.
bgcolor(barstate.isconfirmed ? na : color.new(color.orange, 70), title = "Unconfirmed bar highlight")
```

Note that:

- If the `firstPoint` and `secondPoint` reference identical coordinates, the script will *not* display a line since there is no distance between them to draw. However, the line ID will still exist.
- The script will only display approximately the last 50 lines on the chart, as it does not have a specified `max_lines_count` in the `indicator()` function call. Line drawings persist on the chart until deleted using `line.delete()` or removed by the garbage collector.
- The script *redraws* the line on the open chart bar (i.e., the bar with an orange background highlight) until it closes. After the bar closes, it will no longer update the drawing.

Let's look at a more involved example. This script uses the previous bar's `hl2` price and the current bar's high and low prices to draw a fan with a user-specified number of lines projecting a range of hypothetical price values for the following chart bar. It calls `line.new()` within a for loop to create `linesPerBar` lines on each bar:

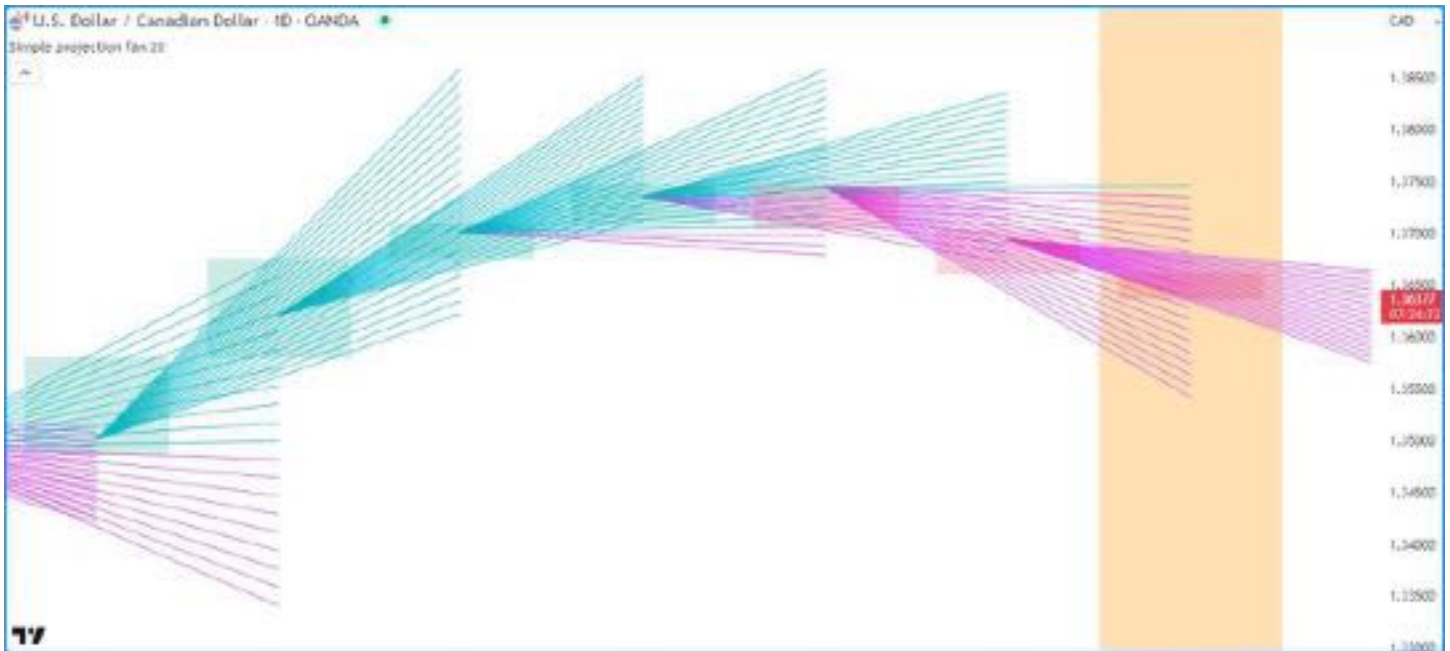


Figure 147: image

```
//@version=6
indicator("Creating lines demo", "Simple projection fan", true, max_lines_count = 500)

//@variable The number of fan lines drawn on each chart bar.
int linesPerBar = input.int(20, "Line drawings per bar", 2, 100)

//@variable The distance between each y point on the current bar.
float step = (high - low) / (linesPerBar - 1)

//@variable The `chart.point` for the start of each line. Does not contain `time` information.
firstPoint = chart.point.from_index(bar_index - 1, hl2[1])
//@variable The `chart.point` for the end of each line. Does not contain `time` information.
secondPoint = chart.point.from_index(bar_index + 1, float(na))

//@variable The stepped y value on the current bar for `secondPoint.price` calculation, starting from the `low`
float barValue = low
```

```
// Loop to draw the fan.
for i = 1 to linesPerBar
    // Update the `price` of the `secondPoint` using the difference between the `barValue` and `firstPoint.price`
    secondPoint.price := 2.0 * barValue - firstPoint.price
    //@variable Is `color.aqua` when the line's slope is positive, `color.fuchsia` otherwise.
    color lineColor = secondPoint.price > firstPoint.price ? color.aqua : color.fuchsia
    // Draw a new `lineColor` line connecting the `firstPoint` and `secondPoint` coordinates.
    // This line uses the `index` field from each point for its x-coordinates.
    line.new(firstPoint, secondPoint, color = lineColor)
    // Add the `step` to the `barValue`.
    barValue += step

// Color the background on the unconfirmed bar.
bgcolor(barstate.isconfirmed ? na : color.new(color.orange, 70), title = "Unconfirmed bar highlight")
```

Note that:

- We've included `max_lines_count = 500` in the `indicator()` function call, meaning the script preserves up to 500 lines on the chart.
- Each `line.new()` call *copies* the information from the `chart.point` referenced by the `firstPoint` and `secondPoint` variables. As such, the script can change the `price` field of the `secondPoint` on each loop iteration without affecting the y-coordinates in other lines.

Modifying lines

The `line.*` namespace contains multiple *setter* functions that modify the properties of line instances:

- `line.set_first_point()` and `line.set_second_point()` respectively update the start and end points of the `id` line using information from the specified `point`.
- `line.set_x1()` and `line.set_x2()` set one of the x-coordinates of the `id` line to a new `x` value, which can represent a bar index or time value depending on the line's `xloc` property.
- `line.set_y1()` and `line.set_y2()` set one of the y-coordinates of the `id` line to a new `y` value.
- `line.set_xy1()` and `line.set_xy2()` update one of the `id` line's points with new `x` and `y` values.
- `line.set_xloc()` sets the `xloc` of the `id` line and updates both of its x-coordinates with new `x1` and `x2` values.
- `line.set_extend()` sets the `extend` property of the `id` line.
- `line.set_color()` updates the `id` line's `color` value.
- `line.set_style()` changes the `style` of the `id` line.
- `line.set_width()` sets the `width` of the `id` line.

All setter functions directly modify the `id` line passed into the call and do not return any value. Each setter function accepts “series” arguments, as a script can change a line's properties throughout its execution.

The following example draws lines connecting the opening price of a `timeframe` to its closing price. The script uses the `var` keyword to declare the `periodLine` and the variables that reference `chart.point` values (`openPoint` and `closePoint`) only on the *first* chart bar, and it assigns new values to these variables over its execution. After detecting a change on the `timeframe`, it sets the `color` of the existing `periodLine` using `line.set_color()`, creates new values for the `openPoint` and `closePoint` using `chart.point.now()`, then assigns a new line using those points to the `periodLine`.

On other bars where the `periodLine` value is not `na`, the script assigns a new `chart.point` to the `closePoint`, then uses `line.set_second_point()` and `line.set_color()` as methods to update the line's properties:

```
//@version=6
indicator("Modifying lines demo", overlay = true)

//@variable The size of each period.
string timeframe = input.timeframe("D", "Timeframe")

//@variable A line connecting the period's opening and closing prices.
var line periodLine = na

//@variable The first point of the line. Contains `time` and `index` information.
var chart.point openPoint = chart.point.now(open)
//@variable The closing point of the line. Contains `time` and `index` information.
var chart.point closePoint = chart.point.now(close)
```



Figure 148: image

```

if timeframe.change(timeframe)
    //@variable The final color of the `periodLine`.
    color finalColor = switch
        closePoint.price > openPoint.price => color.green
        closePoint.price < openPoint.price => color.red
        => color.gray

    // Update the color of the current `periodLine` to the `finalColor`.
    line.set_color(periodLine, finalColor)

    // Assign new points to the `openPoint` and `closePoint`.
    openPoint := chart.point.now(open)
    closePoint := chart.point.now(close)
    // Assign a new line to the `periodLine`. Uses `time` fields from the `openPoint` and `closePoint` as x-co
    periodLine := line.new(openPoint, closePoint, xloc.bar_time, style = line.style_arrow_right, width = 3)

else if not na(periodLine)
    // Assign a new point to the `closePoint`.
    closePoint := chart.point.now(close)

    //@variable The color of the developing `periodLine`.
    color developingColor = switch
        closePoint.price > openPoint.price => color.aqua
        closePoint.price < openPoint.price => color.fuchsia
        => color.gray

    // Update the coordinates of the line's second point using the new `closePoint`.
    // It uses the `time` field from the point for its new x-coordinate.
    periodLine.set_second_point(closePoint)
    // Update the color of the line using the `developingColor`.
    periodLine.set_color(developingColor)

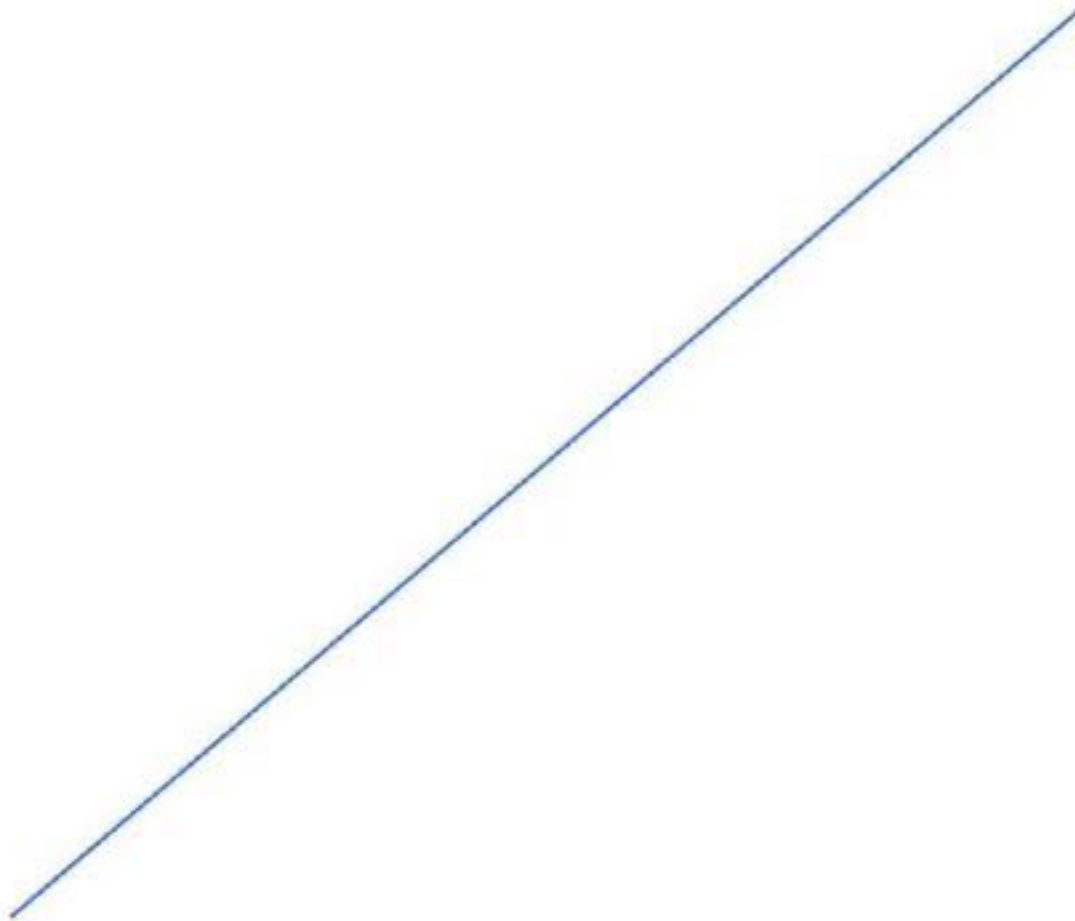
```

Note that:

- Each line drawing in this example uses the `line.style_arrow_right` style. See the Line styles section below for an overview of all available style settings.

Line styles

Users can control the style of their scripts' line drawings by passing one of the following variables as the **style** argument in their `line.new()` or `line.set_style()` function calls:



Argument `line.style_solid`
that:

- *Polylines* can also use any of these variables as their `line_style` value. See the Creating polylines section of this page.

Reading line values

The `line.*` namespace includes *getter* functions, which allow a script to retrieve values from a line object for further use:

- `line.get_x1()` and `line.get_x2()` respectively get the first and second x-coordinate from the `id` line. Whether the value returned represents a bar index or time value depends on the line's `xloc` property.
- `line.get_y1()` and `line.get_y2()` respectively get the `id` line's first and second y-coordinate.
- `line.get_price()` retrieves the price (y-coordinate) from a line `id` at a specified `x` value, including at bar indices outside the line's start and end points. This function is only compatible with lines that use `xloc.bar_index` as the `xloc` value.

The script below draws a new line upon the onset of a rising or falling price pattern forming over `length` bars. It uses the `var` keyword to declare the `directionLine` variable on the first chart bar. The ID assigned to the `directionLine` persists over subsequent bars until the `newDirection` condition occurs, in which case the script assigns a new line to the variable.

On every bar, the script calls the `line.get_y2()`, `line.get_y1()`, `line.get_x2()`, and `line.get_x1()` getters as methods to retrieve values from the current `directionLine` and calculate its `slope`, which it uses to determine the color of each drawing and plot. It retrieves extended values of the `directionLine` from *beyond* its second point using `line.get_price()` and plots them on the chart:



Figure 149: image

```
//@version=6
indicator("Reading line values demo", overlay = true)

//@variable The number of bars for rising and falling calculations.
int length = input.int(2, "Length", 2)

//@variable A line that's drawn whenever `hlc3` starts rising or falling over `length` bars.
var line directionLine = na

//@variable Is `true` when `hlc3` is rising over `length` bars, `false` otherwise.
bool rising = ta.rising(hlc3, length)
//@variable Is `true` when `hlc3` is falling over `length` bars, `false` otherwise.
bool falling = ta.falling(hlc3, length)
//@variable Is `true` when a rising or falling pattern begins, `false` otherwise.
bool newDirection = (rising and not rising[1]) or (falling and not falling[1])

// Update the `directionLine` when `newDirection` is `true`. The line uses the default `xloc.bar_index`.
if newDirection
    directionLine := line.new(bar_index - length, hlc3[length], bar_index, hlc3, width = 3)

//@variable The slope of the `directionLine`.
float slope = (directionLine.get_y2() - directionLine.get_y1()) / (directionLine.get_x2() - directionLine.get_x1())
//@variable The value extrapolated from the `directionLine` at the `bar_index`.
float lineValue = line.get_price(directionLine, bar_index)

//@variable Is `color.green` when the `slope` is positive, `color.red` otherwise.
color slopeColor = slope > 0 ? color.green : color.red

// Update the color of the `directionLine`.
directionLine.set_color(slopeColor)
// Plot the `lineValue`.
plot(lineValue, "Extrapolated value", slopeColor, 3, plot.style_circles)
```

Note that:

- This example calls the second overload of the `line.new()` function, which uses `x1`, `y1`, `x2`, and `y2` parameters to define the start and end points of the line. The `x1` value is `length` bars behind the current `bar_index`, and the `y1` value is the `hlc3` value at that index. The `x2` and `y2` in the function call use the current bar's `bar_index` and `hlc3` values.
- The `line.get_price()` function call treats the `directionLine` as though it extends infinitely, regardless of its `extend` property.
- The script only displays approximately the last 50 lines on the chart, but the plot of extrapolated values spans throughout the chart's history.

Cloning lines

Scripts can clone a line `id` and all its properties with the `line.copy()` function. Any changes to the copied line instance do not affect the original.

For example, this script creates a horizontal line at the the bar's open price once every `length` bars, which it assigns to a `mainLine` variable. On all other bars, it creates a `copiedLine` using `line.copy()` and calls `line.set_*()` functions to modify its properties. As we see below, altering the `copiedLine` does not affect the `mainLine` in any way:

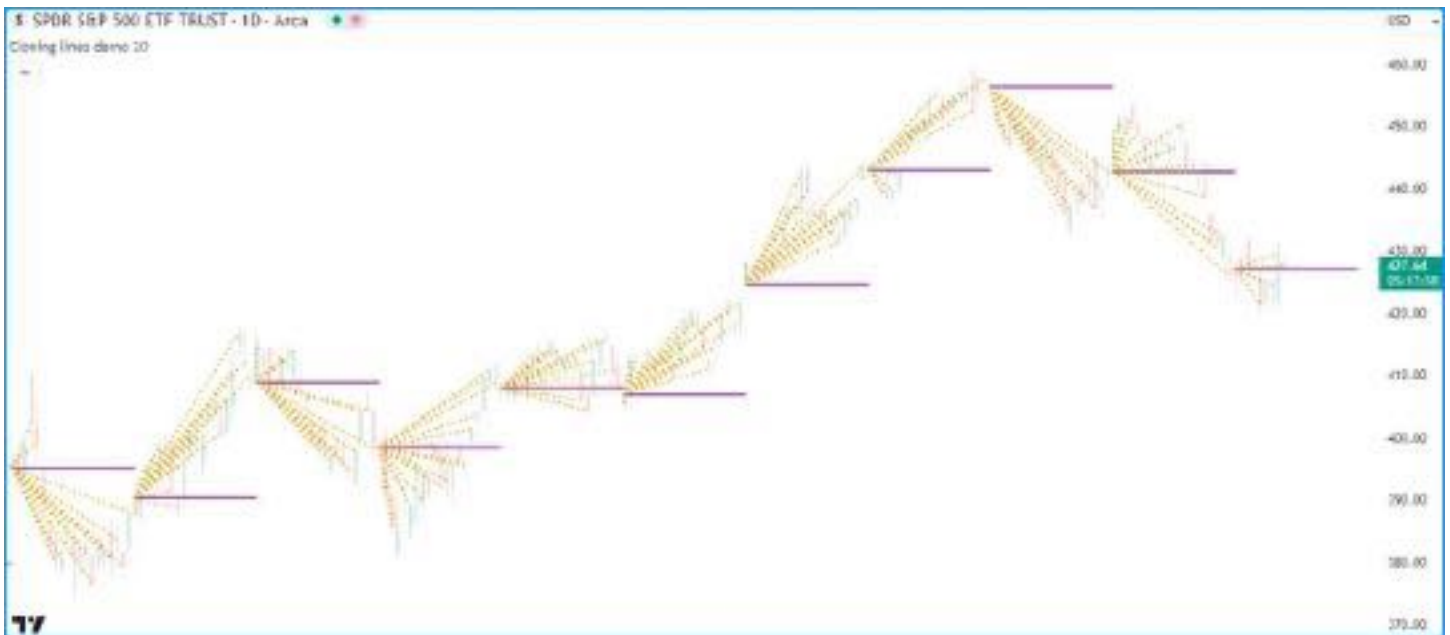


Figure 150: image

```
//@version=6
indicator("Cloning lines demo", overlay = true, max_lines_count = 500)

//@variable The number of bars between each new mainLine assignment.
int length = input.int(20, "Length", 2, 500)

//@variable The first `chart.point` used by the `mainLine`. Contains `index` and `time` information.
firstPoint = chart.point.now(open)
//@variable The second `chart.point` used by the `mainLine`. Does not contain `time` information.
secondPoint = chart.point.from_index(bar_index + length, open)

//@variable A horizontal line drawn at the `open` price once every `length` bars.
var line mainLine = na

if bar_index % length == 0
    // Assign a new line to the `mainLine` that connects the `firstPoint` to the `secondPoint`.
    // This line uses the `index` fields from both points as x-coordinates.
    mainLine := line.new(firstPoint, secondPoint, color = color.purple, width = 2)

//@variable A copy of the `mainLine`. Changes to this line do not affect the original.
```

```

line copiedLine = line.copy(mainLine)

// Update the color, style, and second point of the `copiedLine`.
line.set_color(copiedLine, color.orange)
line.set_style(copiedLine, line.style_dotted)
line.set_second_point(copiedLine, chart.point.now(close))

```

Note that:

- The `index` field of the `secondPoint` is `length` bars beyond the current `bar_index`. Since the maximum x-coordinate allowed with `xloc.bar_index` is `bar_index + 500`, we've set the `maxval` of the `length` input to 500.

Deleting lines

To delete a line id drawn by a script, use the `line.delete()` function. This function removes the line instance from the script and its drawing on the chart.

Deleting line instances is often handy when one wants to only keep a specific number of lines on the chart at any given time or conditionally remove drawings as a chart progresses.

For example, this script draws a horizontal line with the `extend.right` property whenever an RSI crosses its EMA.

The script stores all line IDs in a `lines` array that it uses as a queue to only display the last `numberOfLines` on the chart. When the size of the array exceeds the specified `numberOfLines`, the script removes the array's oldest line ID using `array.shift()` and deletes it with `line.delete()`:



Figure 151: image

```

//@version=6

//@variable The maximum number of lines allowed on the chart.
const int MAX_LINES_COUNT = 500

indicator("Deleting lines demo", "RSI cross levels", max_lines_count = MAX_LINES_COUNT)

//@variable The length of the RSI.
int rsiLength = input.int(14, "RSI length", 2)
//@variable The length of the RSI's EMA.
int emaLength = input.int(28, "RSI average length", 2)
//@variable The maximum number of lines to keep on the chart.
int numberOfLines = input.int(20, "Lines on the chart", 0, MAX_LINES_COUNT)

```

```

//@variable An array containing the IDs of lines on the chart.
var array<line> lines = array.new<line>()

//@variable An `rsiLength` RSI of `close`.
float rsi = ta.rsi(close, rsiLength)
//@variable A `maLength` EMA of the `rsi`.
float rsiMA = ta.ema(rsi, maLength)

if ta.cross(rsi, rsiMA)
    //@variable The color of the horizontal line.
    color lineColor = rsi > rsiMA ? color.green : color.red
    // Draw a new horizontal line. Uses the default `xloc.bar_index`.
    newLine = line.new(bar_index, rsiMA, bar_index + 1, rsiMA, extend = extend.right, color = lineColor, width
    // Push the `newLine` into the `lines` array.
    lines.push(newLine)
    // Delete the oldest line when the size of the array exceeds the specified `numberOfLines`.
    if array.size(lines) > numberOfLines
        line.delete(lines.shift())

// Plot the `rsi` and `rsiMA`.
plot(rsi, "RSI", color.new(color.blue, 40))
plot(rsiMA, "EMA of RSI", color.new(color.gray, 30))

```

Note that:

- We declared a `MAX_LINES_COUNT` variable with the “const int” *qualified type*, which the script uses as the `max_lines_count` in the `indicator()` function and the `maxval` of the `input.int()` assigned to the `numberOfLines` variable.
- This example uses the second overload of the `line.new()` function, which specifies `x1`, `y1`, `x2`, and `y2` coordinates independently.

Filling the space between lines

Scripts can *fill* the space between two line drawings by creating a `linefill` object that references them with the `linefill.new()` function. Linefills automatically determine their fill boundaries using the properties from the `line1` and `line2` IDs that they reference.

For example, this script calculates a simple linear regression channel. On the first chart bar, the script declares the `basisLine`, `upperLine`, and `lowerLine` variables to reference the channel’s line IDs, then it makes two `linefill.new()` calls to create `linefill` objects that fill the upper and lower portions of the channel. The first `linefill` fills the space between the `basisLine` and the `upperLine`, and the second fills the space between the `basisLine` and `lowerLine`.

The script updates the coordinates of the lines across subsequent bars. However, notice that the script never needs to update the `linefills` declared on the first bar. They automatically update their fill regions based on the coordinates of their assigned lines:

```

//@version=6
indicator("Filling the space between lines demo", "Simple linreg channel", true)

//@variable The number of bars in the linear regression calculation.
int lengthInput = input.int(100)

//@variable The basis line of the regression channel.
var line basisLine = line.new(na, na, na, na, extend = extend.right, color = chart.fg_color, width = 2)
//@variable The channel's upper line.
var line upperLine = line.new(na, na, na, na, extend = extend.right, color = color.teal, width = 2)
//@variable The channel's lower line.
var line lowerLine = line.new(na, na, na, na, extend = extend.right, color = color.maroon, width = 2)

//@variable A linefill instance that fills the space between the `basisLine` and `upperLine`.
var linefill upperFill = linefill.new(basisLine, upperLine, color.new(color.teal, 80))
//@variable A linefill instance that fills the space between the `basisLine` and `lowerLine`.
var linefill lowerFill = linefill.new(basisLine, lowerLine, color.new(color.maroon, 80))

```



Figure 152: image

```
// Update the `basisLine` coordinates with current linear regression values.
basisLine.set_xy1(bar_index + 1 - lengthInput, ta.linreg(close, lengthInput, lengthInput - 1))
basisLine.set_xy2(bar_index, ta.linreg(close, lengthInput, 0))

//@variable The channel's standard deviation.
float stDev = 0.0
for i = 0 to lengthInput - 1
    stDev += math.pow(close[i] - line.get_price(basisLine, bar_index - i), 2)
stDev := math.sqrt(stDev / lengthInput) * 2.0

// Update the `upperLine` and `lowerLine` using the values from the `basisLine` and the `stDev`.
upperLine.set_xy1(basisLine.get_x1(), basisLine.get_y1() + stDev)
upperLine.set_xy2(basisLine.get_x2(), basisLine.get_y2() + stDev)
lowerLine.set_xy1(basisLine.get_x1(), basisLine.get_y1() - stDev)
lowerLine.set_xy2(basisLine.get_x2(), basisLine.get_y2() - stDev)
```

To learn more about the linefill type, see this section of the Fills page.

Boxes

The built-ins in the `box.*` namespace create and manage box objects:

- The `box.new()` function creates a new box.
- The `box.set_*` functions modify box properties.
- The `box.get_*` functions retrieve values from a box instance.
- The `box.copy()` function clones a box instance.
- The `box.delete()` function deletes a box instance.
- The `box.all` variable references a read-only array containing the IDs of all boxes displayed by the script. The array's size depends on the `max_boxes_count` of the `indicator()` or `strategy()` declaration statement and the number of boxes the script has drawn.

As with lines, users can call `box.set_*`, `box.get_*`, `box.copy()`, and `box.delete()` built-ins as functions or methods.

Creating boxes

The `box.new()` function creates a new box object to display on the chart. It has the following signatures:

```
box.new(top_left, bottom_right, border_color, border_width, border_style, extend, xloc, bgcolor, text, text_size)
box.new(left, top, right, bottom, border_color, border_width, border_style, extend, xloc, bgcolor, text, text_size)
```

This function's first overload includes the `top_left` and `bottom_right` parameters, which accept `chart.point` objects representing the top-left and bottom-right corners of the box, respectively. The function copies the information from these chart points to set the coordinates of the box's corners. Whether it uses the `index` or `time` fields of the `top_left` and `bottom_right` points as x-coordinates depends on the function's `xloc` value.

The second overload specifies `left`, `top`, `right`, and `bottom` edges of the box. The `left` and `right` parameters accept int values specifying the box's left and right x-coordinates, which can be bar index or time values depending on the `xloc` value in the function call. The `top` and `bottom` parameters accept float values representing the box's top and bottom y-coordinates.

The function's additional parameters are identical in both overloads:

border_color

Specifies the color of all four of the box's borders. The default is `color.blue`.

border_width

Specifies the width of the borders, in pixels. Its default value is 1.

border_style

Specifies the style of the borders, which can be any of the options in the Box styles section of this page.

extend

Determines whether the box's borders extend infinitely beyond the left or right x-coordinates. It accepts one of the following values: `extend.left`, `extend.right`, `extend.both`, or `extend.none` (default).

xloc

Determines whether the left and right edges of the box use bar index or time values as x-coordinates. The default is `xloc.bar_index`.

In the first overload, an `xloc` value of `xloc.bar_index` means that the function will use the `index` fields of the `top_left` and `bottom_right` chart points, and an `xloc` value of `xloc.bar_time` means that it will use their `time` fields.

In the second overload, using an `xloc` value of `xloc.bar_index` means the function treats the `left` and `right` values as bar indices, and `xloc.bar_time` means it will treat them as timestamps.

When the specified x-coordinates represent *bar index* values, it's important to note that the minimum x-coordinate allowed is `bar_index - 9999`. For larger offsets, one can use `xloc.bar_time`.

bgcolor

Specifies the background color of the space inside the box. The default value is `color.blue`.

text

The text to display inside the box. By default, its value is an empty string.

text_size

Specifies the size of the text within the box. It accepts both "int" size values and "string" `size.*` constants. The "int" size can be any positive integer. The `size.*` constants and their equivalent "int" sizes are: `size.auto` (0), `size.tiny` (8), `size.small` (10), `size.normal` (14), `size.large` (20), and `size.huge` (36). The default value is `size.auto`.

text_color

Controls the color of the text. Its default is `color.black`.

text_halign

Specifies the horizontal alignment of the text within the box's boundaries. It accepts one of the following: `text.align_left`, `text.align_right`, or `text.align_center` (default).

text_valign

Specifies the vertical alignment of the text within the box's boundaries. It accepts one of the following: `text.align_top`, `text.align_bottom`, or `text.align_center` (default).

text_wrap

Determines whether the box will wrap the text within it. If its value is `text.wrap_auto`, the box wraps the text to ensure it does not span past its vertical borders. It also clips the wrapped text when it extends past the bottom. If the value is `text.wrap_none`, the box displays the text on a single line that can extend beyond its borders. The default is `text.wrap_none`.

text_font_family

Defines the font family of the box's text. Using `font.family_default` displays the box's text with the system's default font. The `font.family_monospace` displays the text in a monospace format. The default value is `font.family_default`.

force_overlay

If `true`, the drawing will display on the main chart pane, even when the script occupies a separate pane. Optional. The default is `false`.

text_formatting

Specifies the formatting of the box's text. Using `text.format_none` displays the text with no special formatting. This parameter also accepts the arguments `text.format_bold` or `text.format_italic`. Using `text.format_bold + text.format_italic` applies both formats together to display text that is both bold and italicized. The default value is `text.format_none`.

Let's write a simple script to display boxes on a chart. The example below draws a box projecting each bar's high and low values from the horizontal center of the current bar to the center of the next available bar.

On each bar, the script creates `topLeft` and `bottomRight` points via `chart.point.now()` and `chart.point_from_index()`, then calls `box.new()` to construct a new box and display it on the chart. It also highlights the background on the unconfirmed chart bar using `bgcolor()` to indicate that it redraws that box until the bar's last update:



Figure 153: image

```
//@version=6
indicator("Creating boxes demo", overlay = true)

//@variable The `chart.point` for the top-left corner of the box. Contains `index` and `time` information.
topLeft = chart.point.now(high)
//@variable The `chart.point` for the bottom-right corner of the box. Does not contain `time` information.
bottomRight = chart.point.from_index(bar_index + 1, low)

// Draw a box using the `topLeft` and `bottomRight` corner points. Uses the `index` fields as x-coordinates.
box.new(topLeft, bottomRight, color.purple, 2, bgcolor = color.new(color.gray, 70))

// Color the background on the unconfirmed bar.
bgcolor(barstate.isconfirmed ? na : color.new(color.orange, 70), title = "Unconfirmed bar highlight")
```

Note that:

- The `bottomRight` point's `index` field is one bar greater than the `index` in the `topLeft`. If the x-coordinates of the corners were equal, the script would draw a vertical line at the horizontal center of each bar, resembling the example in this page's `Creating lines` section.
- Similar to lines, if the `topLeft` and `bottomRight` contained identical coordinates, the box wouldn't display on the chart since there would be no space between them to draw. However, its ID would still exist.
- This script only displays approximately the last 50 boxes on the chart, as we have not specified a `max_boxes_count` in the `indicator()` function call.

Modifying boxes

Multiple *setter* functions exist in the `box.*` namespace, allowing scripts to modify the properties of box objects:

- `box.set_top_left_point()` and `box.set_bottom_right_point()` respectively update the top-left and bottom-right coordinates of the `id` box using information from the specified `point`.
- `box.set_left()` and `box.set_right()` set the left or right x-coordinate of the `id` box to a new `left/right` value, which can be a bar index or time value depending on the box's `xloc` property.
- `box.set_top()` and `box.set_bottom()` set the top or bottom y-coordinate of the `id` box to a new `top/bottom` value.
- `box.set_lefttop()` sets the `left` and `top` coordinates of the `id` box, and `box.set_rightbottom()` sets its `right` and `bottom` coordinates.
- `box.set_border_color()`, `box.set_border_width()` and `box.set_border_style()` respectively update the `color`, `width`, and `style` of the `id` box's border.
- `box.set_extend()` sets the horizontal `extend` property of the `id` box.
- `box.set_bgcolor()` sets the color of the space inside the `id` box to a new `color`.
- `box.set_text()`, `box.set_text_size()`, `box.set_text_color()`, `box.set_text_halign()`, `box.set_text_valign()`, `box.set_text_wrap()`, `box.set_text_font_family()`, and `box.set_text_formatting()` update the `id` box's text-related properties.

As with setter functions in the `line.*` namespace, all box setters modify the `id` box directly without returning a value, and each setter function accepts "series" arguments.

Note that, unlike lines, the `box.*` namespace does not contain a setter function to modify a box's `xloc`. Users must create a new box with the desired `xloc` setting for such cases.

This example uses boxes to visualize the ranges of upward and downward bars with the highest volume over a user-defined `timeframe`. When the script detects a change in the `timeframe`, it assigns new boxes to its `upBox` and `downBox` variables, resets its `upVolume` and `downVolume` values, and highlights the chart background.

When an upward or downward bar's volume exceeds the `upVolume` or `downVolume`, the script updates the volume-tracking variables and calls `box.set_top_left_point()` and `box.set_bottom_right_point()` to update the `upBox` or `downBox` coordinates. The setters use the information from the chart points created with `chart.point.now()` and `chart.point.from_time()` to project that bar's high and low values from the current time to the closing time of the `timeframe`:

```
//@version=6
indicator("Modifying boxes demo", "High volume boxes", true, max_boxes_count = 100)

//@variable The timeframe of the calculation.
string timeframe = input.timeframe("D", "Timeframe")

//@variable A box projecting the range of the upward bar with the highest `volume` over the `timeframe`.
var box upBox = na
//@variable A box projecting the range of the downward bar with the lowest `volume` over the `timeframe`.
var box downBox = na
//@variable The highest volume of upward bars over the `timeframe`.
var float upVolume = na
//@variable The highest volume of downward bars over the `timeframe`.
var float downVolume = na

// Color variables.
var color upBorder    = color.teal
var color upFill      = color.new(color.teal, 90)
var color downBorder  = color.maroon
var color downFill    = color.new(color.maroon, 90)
```



Figure 154: image

```
//@variable The closing time of the `timeframe`.
int closeTime = time_close(timeframe)
//@variable Is `true` when a new bar starts on the `timeframe`.
bool changeTF = timeframe.change(timeframe)

//@variable The `chart.point` for the top-left corner of the boxes. Contains `index` and `time` information.
topLeft = chart.point.now(high)
//@variable The `chart.point` for the bottom-right corner of the boxes. Does not contain `index` information.
bottomRight = chart.point.from_time(closeTime, low)

if changeTF and not na(volume)
    if close > open
        // Update `upVolume` and `downVolume` values.
        upVolume := volume
        downVolume := 0.0
        // Draw a new `upBox` using `time` and `price` info from the `topLeft` and `bottomRight` points.
        upBox := box.new(topLeft, bottomRight, upBorder, 3, xloc = xloc.bar_time, bgcolor = upFill)
        // Draw a new `downBox` with `na` coordinates.
        downBox := box.new(na, na, na, na, downBorder, 3, xloc = xloc.bar_time, bgcolor = downFill)
    else
        // Update `upVolume` and `downVolume` values.
        upVolume := 0.0
        downVolume := volume
        // Draw a new `upBox` with `na` coordinates.
        upBox := box.new(na, na, na, na, upBorder, 3, xloc = xloc.bar_time, bgcolor = upFill)
        // Draw a new `downBox` using `time` and `price` info from the `topLeft` and `bottomRight` points.
        downBox := box.new(topLeft, bottomRight, downBorder, 3, xloc = xloc.bar_time, bgcolor = downFill)
// Update the ``upVolume`` and change the ``upBox`` coordinates when volume increases on an upward bar.
else if close > open and volume > upVolume
    upVolume := volume
    box.set_top_left_point(upBox, topLeft)
    box.set_bottom_right_point(upBox, bottomRight)
// Update the ``downVolume`` and change the ``downBox`` coordinates when volume increases on a downward bar.
else if close <= open and volume > downVolume
    downVolume := volume
    box.set_top_left_point(downBox, topLeft)
```

```

box.set_bottom_right_point(downBox, bottomRight)

// Highlight the background when a new `timeframe` bar starts.
bgcolor(changeTF ? color.new(color.orange, 70) : na, title = "Timeframe change highlight")

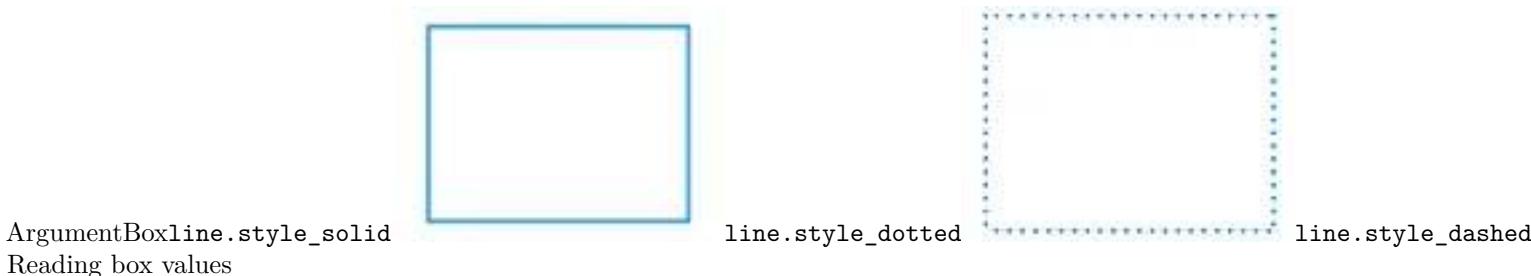
```

Note that:

- The `indicator()` function call contains `max_boxes_count = 100`, meaning the script will preserve the last 100 boxes on the chart.
- We utilized *both overloads* of `box.new()` in this example. On the first bar of the `timeframe`, the script calls the first overload for the `upBox` when the bar is rising, and it calls that overload for the `downBox` when the bar is falling. It uses the second overload to assign a new box with `na` values to the other box variable on that bar.

Box styles

Users can include one of the following `line.style_*` variables in their `box.new()` or `box.set_border_style()` function calls to set the border styles of boxes drawn by their scripts:



The `box.*` namespace features *getter* functions that allow scripts to retrieve coordinate values from a box instance:

- `box.get_left()` and `box.get_right()` respectively get the x-coordinates of the left and right edges of the `id` box. Whether the value returned represents a bar index or time value depends on the box's `xloc` property.
- `box.get_top()` and `box.get_bottom()` respectively get the top and bottom y-coordinates of the `id` box.

The example below draws boxes to visualize hypothetical price ranges over a period of `length` bars. At the start of each new period, it uses the average candle range multiplied by the `scaleFactor` input to calculate the corner points of a box centered at the `hl2` price with an `initialRange` height. After drawing the first box, it creates `numberOfBoxes - 1` new boxes inside a for loop.

Within each loop iteration, the script gets the `lastBoxDrawn` by retrieving the last element from the read-only `box.all` array, then calls `box.get_top()` and `box.get_bottom()` to get its y-coordinates. It uses these values to calculate the coordinates for a new box that's `scaleFactor` times taller than the previous:

```

//@version=6
indicator("Reading box values demo", "Nested boxes", overlay = true, max_boxes_count = 500)

//@variable The number of bars in the range calculation.
int length = input.int(10, "Length", 2, 500)
//@variable The number of nested boxes drawn on each period.
int numberOfBoxes = input.int(5, "Nested box count", 1)
//@variable The scale factor applied to each box.
float scaleFactor = input.float(1.6, "Scale factor", 1)

//@variable The initial box range.
float initialRange = scaleFactor * ta.sma(high - low, length)

if bar_index % length == 0
    //@variable The top-left `chart.point` for the initial box. Does not contain `time` information.
    topLeft = chart.point.from_index(bar_index, hl2 + initialRange / 2)
    //@variable The bottom-right `chart.point` for the initial box. Does not contain `time` information.
    bottomRight = chart.point.from_index(bar_index + length, hl2 - initialRange / 2)

    // Calculate border and fill colors of the boxes.
    borderColor = color.rgb(math.random(100, 255), math.random(0, 100), math.random(100, 255))
    bgColor = color.new(borderColor, math.max(100 * (1 - 1/numberOfBoxes), 90))

```

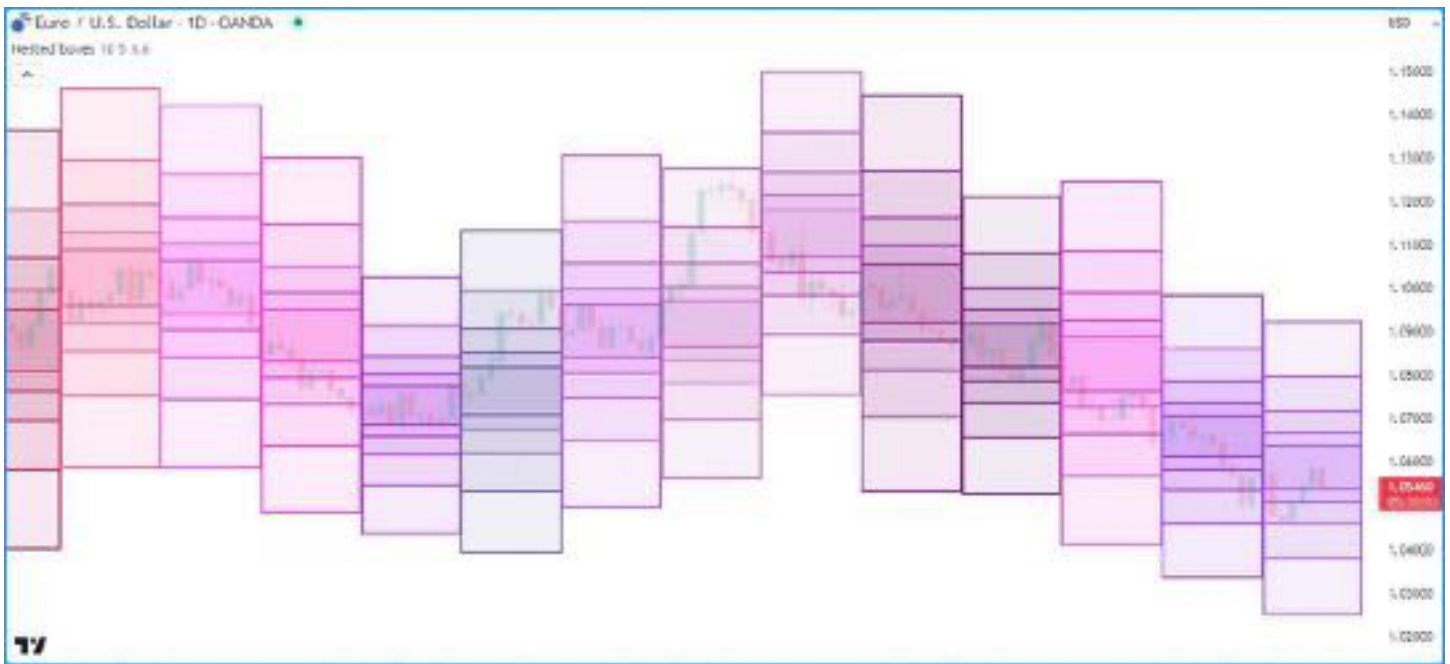


Figure 155: image

```
// Draw a new box using the `topLeft` and `bottomRight` points. Uses their `index` fields as x-coordinates
box.new(topLeft, bottomRight, borderColor, 2, bgcolor = bgColor)

if numberOfBoxes > 1
    // Loop to create additional boxes.
    for i = 1 to numberOfBoxes - 1
        //@variable The last box drawn by the script.
        box lastBoxDrawn = box.all.last()

        //@variable The top price of the last box.
        float top = box.get_top(lastBoxDrawn)
        //@variable The bottom price of the last box.
        float bottom = box.get_bottom(lastBoxDrawn)

        //@variable The scaled range of the new box.
        float newRange = scaleFactor * (top - bottom) * 0.5

        // Update the `price` fields of the `topLeft` and `bottomRight` points.
        // This does not affect the coordinates of previous boxes.
        topLeft.price      := hl2 + newRange
        bottomRight.price := hl2 - newRange

        // Draw a new box using the updated `topLeft` and `bottomRight` points.
        box.new(topLeft, bottomRight, borderColor, 2, bgcolor = bgColor)
```

Note that:

- The indicator() function call uses `max_boxes_count = 500`, meaning the script can display up to 500 boxes on the chart.
- Each drawing has a **right** index **length** bars beyond the **left** index. Since the x-coordinates of these drawings can be up to 500 bars into the future, we've set the `maxval` of the `length` input to 500.
- On each new period, the script uses `randomized.color.rgb()` values for the `border_color` and `bgcolor` of the boxes.
- Each `box.new()` call copies the coordinates from the `chart.point` objects assigned to the `topLeft` and `bottomRight` variables, which is why the script can modify their **price** fields on each loop iteration without affecting the other boxes.

Cloning boxes

To clone a specific box id, use `box.copy()`. This function copies the box and its properties. Any changes to the copied box do not affect the original.

For example, this script declares an `originalBox` variable on the first bar and assigns a new box to it once every `length` bars. On other bars, it uses `box.copy()` to create a `copiedBox` and calls `box.set_*()` functions to modify its properties. As shown on the chart below, these changes do not modify the `originalBox`:

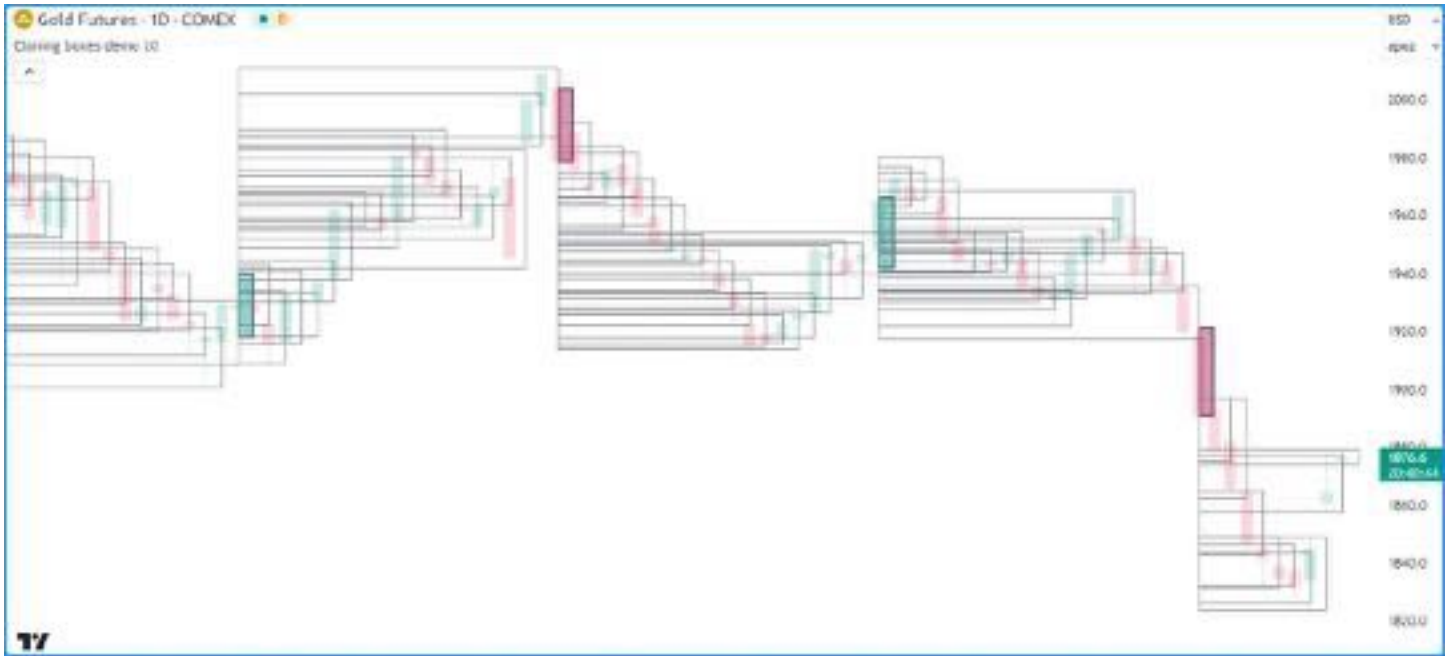


Figure 156: image

```
//@version=6
indicator("Cloning boxes demo", overlay = true, max_boxes_count = 500)

//@variable The number of bars between each new mainLine assignment.
int length = input.int(20, "Length", 2)

//@variable The `chart.point` for the top-left of the `originalBox`. Contains `time` and `index` information.
topLeft = chart.point.now(high)
//@variable The `chart.point` for the bottom-right of the `originalBox`. Does not contain `time` information.
bottomRight = chart.point.from_index(bar_index + 1, low)

//@variable A new box with `topLeft` and `bottomRight` corners on every `length` bars.
var box originalBox = na

//@variable Is teal when the bar is rising, maroon when it's falling.
color originalColor = close > open ? color.teal : color.maroon

if bar_index % length == 0
    // Assign a new box using the `topLeft` and `bottomRight` info to the `originalBox`.
    // This box uses the `index` fields from the points as x-coordinates.
    originalBox := box.new(topLeft, bottomRight, originalColor, 2, bgcolor = color.new(originalColor, 60))
else
    //@variable A clone of the `originalBox`.
    box copiedBox = box.copy(originalBox)
    // Modify the `copiedBox`. These changes do not affect the `originalBox`.
    box.set_top(copiedBox, high)
    box.set_bottom_right_point(copiedBox, bottomRight)
    box.set_border_color(copiedBox, color.gray)
    box.set_border_width(copiedBox, 1)
```

```
box.set_bgcolor(copiedBox, na)
```

Deleting boxes

To delete boxes drawn by a script, use `box.delete()`. As with `*.delete()` functions in other drawing namespaces, this function is handy for conditionally removing boxes or maintaining a specific number of boxes on the chart.

This example displays boxes representing periodic cumulative volume values. The script creates a new box ID and stores it in a `boxes` array once every `length` bars. If the array's size exceeds the specified `numberOfBoxes`, the script removes the oldest box from the array using `array.shift()` and deletes it using `box.delete()`.

On other bars, it accumulates volume over each period by modifying the `top` of the last box in the `boxes` array. The script then uses for loops to find the `highestTop` of all the array's boxes and set the `bgcolor` of each box with a gradient color based on its `box.get_top()` value relative to the `highestTop`:

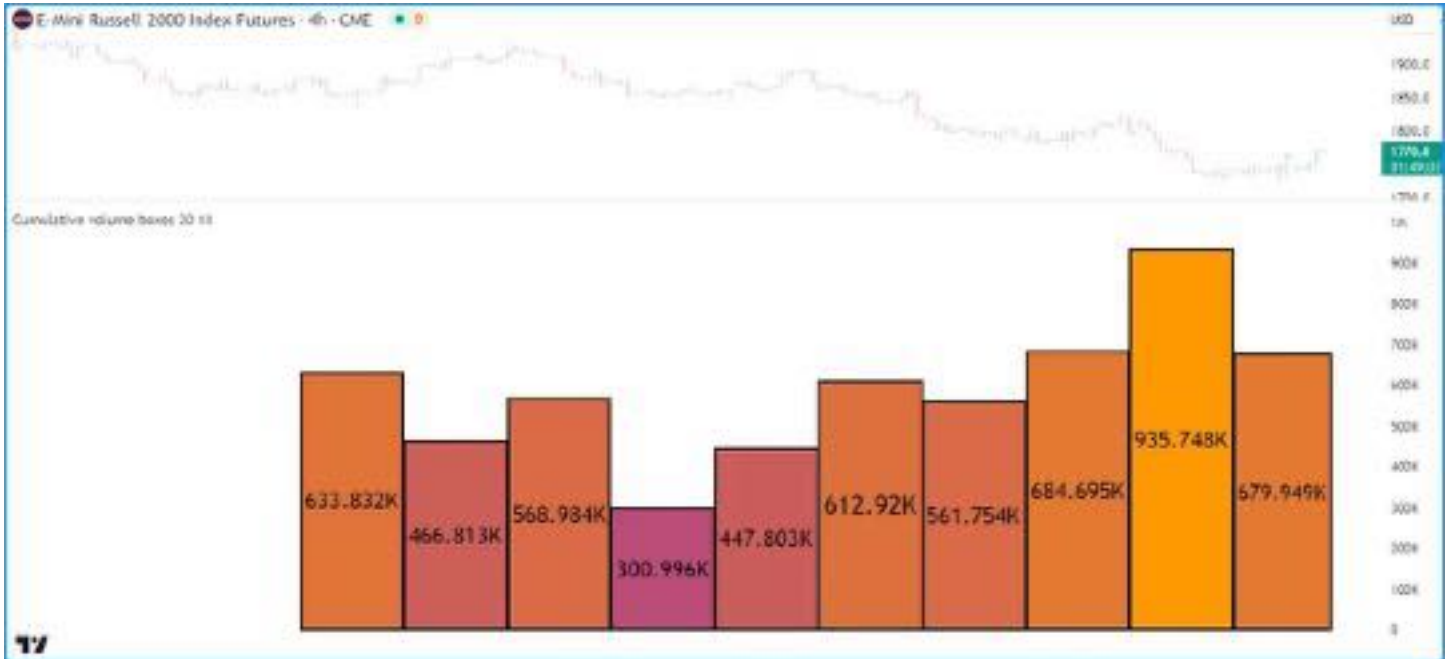


Figure 157: image

```
//@version=6
```

```
//@variable The maximum number of boxes to show on the chart.
const int MAX_BOXES_COUNT = 500
```

```
indicator("Deleting boxes demo", "Cumulative volume boxes", format = format.volume, max_boxes_count = MAX_BOXES_COUNT)
```

```
//@variable The number of bars in each period.
```

```
int length = input.int(20, "Length", 1)
```

```
//@variable The maximum number of volume boxes in the calculation.
```

```
int numberOfBoxes = input.int(10, "Number of boxes", 1, MAX_BOXES_COUNT)
```

```
//@variable An array containing the ID of each box displayed by the script.
```

```
var boxes = array.new<box>()
```

```
if bar_index % length == 0
```

```
    // Push a new box into the `boxes` array. The box has the default `xloc.bar_index` property.
```

```
    boxes.push(box.new(bar_index, 0, bar_index + 1, 0, #000000, 2, text_color = #000000))
```

```
    // Shift the oldest box out of the array and delete it when the array's size exceeds the `numberOfBoxes`.
```

```
    if boxes.size() > numberOfBoxes
```

```
        box.delete(boxes.shift())
```

```
//@variable The last box drawn by the script as of the current chart bar.
```



```

box lastBox = boxes.last()
// Add the current bar's volume to the top of the `lastBox` and update the `right` index.
lastBox.set_top(lastBox.get_top() + volume)
lastBox.set_right(bar_index + 1)
// Display the top of the `lastBox` as volume-formatted text.
lastBox.set_text(str.toString(lastBox.get_top(), format.volume))

//@variable The highest `top` of all boxes in the `boxes` array.
float highestTop = 0.0
for id in boxes
    highestTop := math.max(id.get_top(), highestTop)

// Set the `bgcolor` of each `id` in `boxes` with a gradient based on the ratio of its `top` to the `highestTop`.
for id in boxes
    id.set_bgcolor(color.from_gradient(id.get_top() / highestTop, 0, 1, color.purple, color.orange))

```

Note that:

- At the top of the code, we've declared a `MAX_BOXES_COUNT` variable with the “const int” *qualified type*. We use this value as the `max_boxes_count` in the `indicator()` function and the maximum possible value of the `numberOfBoxes` input.
- This script uses the second overload of the `box.new()` function, which specifies the box's `left`, `top`, `right`, and `bottom` coordinates separately.
- We've included `format.volume` as the `format` argument in the `indicator()` call, which tells the script that the y-axis of the chart pane represents *volume* values. Each box also displays its top value as volume-formatted text.

Polylines

Pine Script™ polylines are **advanced** drawings that sequentially connect the coordinates from an array of `chart.point` instances using straight or *curved* line segments.

These powerful drawings can connect up to 10,000 points at any available location on the chart, allowing scripts to draw custom series, polygons, and other complex geometric formations that are otherwise difficult or impossible to draw using line or box objects.

The `polyline.*` namespace features the following built-ins for creating and managing polyline objects:

- The `polyline.new()` function creates a new polyline instance.
- The `polyline.delete()` function deletes an existing polyline instance.
- The `polyline.all` variable references a read-only array containing the IDs of all polylines displayed by the script. The array's size depends on the `max_polylines_count` of the `indicator()` or `strategy()` declaration statement and the number of polylines drawn by the script.

Unlike lines or boxes, polylines do not have functions for modification or reading their properties. To redraw a polyline on the chart, one can *delete* the existing instance and *create* a new polyline with the desired changes.

Creating polylines

The `polyline.new()` function creates a new polyline instance to display on the chart. It has the following signature:

```
polyline.new(points, curved, closed, xloc, line_color, fill_color, line_style, line_width, force_overlay) → series
```

The following eight parameters affect the behavior of a polyline drawing:

points

Accepts an array of `chart.point` objects that determine the coordinates of each point in the polyline. The drawing connects the coordinates from each element in the array sequentially, starting from the *first*. Whether the polyline uses the `index` or `time` field from each chart point for its x-coordinates depends on the `xloc` value in the function call.

curved

Specifies whether the drawing uses curved line segments to connect each `chart.point` in the `points` array. The default value is `false`, meaning it uses straight line segments.

closed

Controls whether the polyline will connect the last `chart.point` in the `points` array to the first, forming a *closed polyline*. The default value is `false`.

`xloc`

Specifies which field from each `chart.point` in the `points` array the polyline uses for its x-coordinates. When its value is `xloc.bar_index`, the function uses the `index` fields to create the polyline. When its value is `xloc.bar_time`, the function uses the `time` fields. The default value is `xloc.bar_index`.

`line_color`

Specifies the color of all line segments in the polyline drawing. The default is `color.blue`.

`fill_color`

Controls the color of the closed space filled by the polyline drawing. Its default value is `na`.

`line_style`

Specifies the style of the polyline, which can be any of the available options in the Line styles section of this page. The default is `line.style_solid`.

`line_width`

Specifies the width of the polyline, in pixels. The default value is 1.

`force_overlay`

If `true`, the drawing will display on the main chart pane, even when the script occupies a separate pane. Optional. The default is `false`.

This script demonstrates a simple example of drawing a polyline on the chart. It pushes a new `chart.point` with an alternating price value into a `points` array and colors the background with `bgcolor()` once every `length` bars.

On the last confirmed historical bar, the script draws a new polyline on the chart, connecting the coordinates from each `chart.point` in the array, starting from the first:

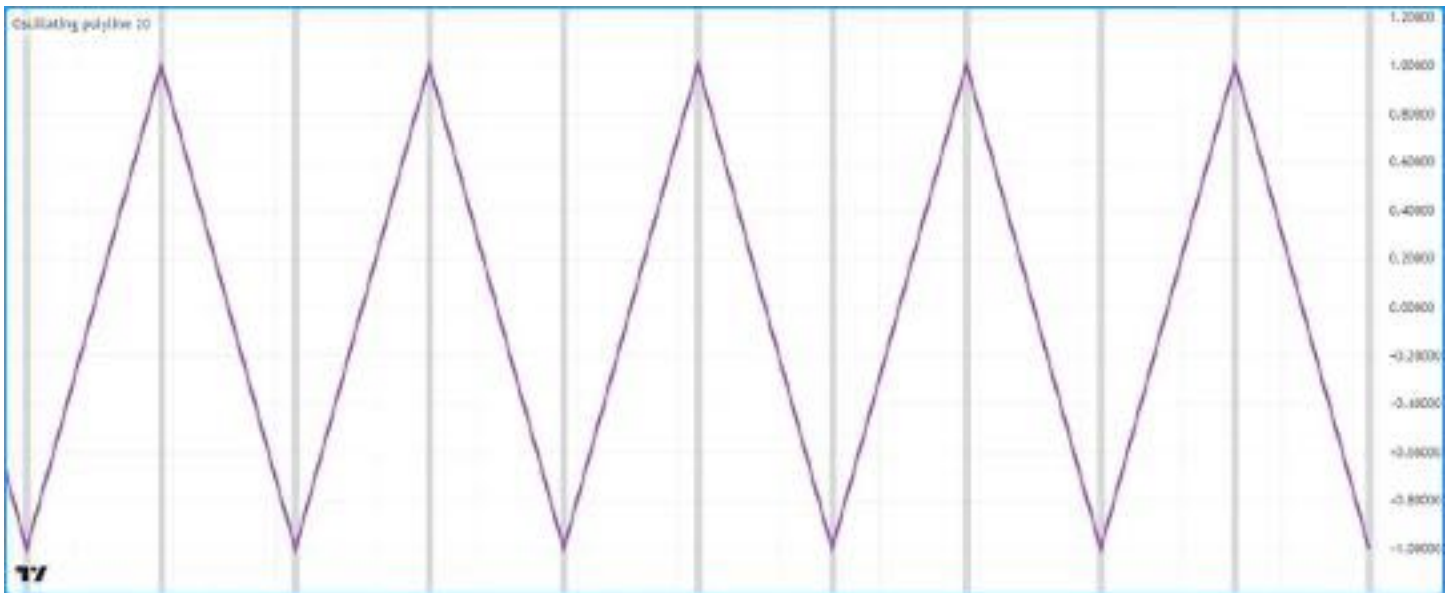


Figure 158: image

```
//@version=6
indicator("Creating polylines demo", "Oscillating polyline")

//@variable The number of bars between each point in the drawing.
int length = input.int(20, "Length between points", 2)

//@variable An array of `chart.point` objects to sequentially connect with a polyline.
var points = array.new<chart.point>()
```

```
//@variable The y-coordinate of each point in the `points`. Alternates between 1 and -1 on each `newPoint`.
var int yValue = 1

//@variable Is `true` once every `length` bars, `false` otherwise.
bool newPoint = bar_index % length == 0

if newPoint
    // Push a new `chart.point` into the `points`. The new point contains `time` and `index` info.
    points.push(chart.point.now(yValue))
    // Change the sign of the `yValue`.
    yValue *= -1

// Draw a new `polyline` on the last confirmed historical chart bar.
// The polyline uses the `time` field from each `chart.point` in the `points` array as x-coordinates.
if barstate.islastconfirmedhistory
    polyline.new(points, xloc = xloc.bar_time, line_color = #9151A6, line_width = 3)

// Highlight the chart background on every `newPoint` condition.
bgcolor(newPoint ? color.new(color.gray, 70) : na, title = "New point highlight")
```

Note that:

- This script uses only *one* polyline to connect each chart point from the array with straight line segments, and this drawing spans throughout the available chart data, starting from the first bar.
- While one can achieve a similar effect using lines, doing so would require a new line instance on each occurrence of the `newPoint` condition, and such a drawing would be limited to a maximum of 500 line segments. This single unclosed polyline drawing, on the other hand, can contain up to 9,999 line segments.

Curved drawings Polyline can draw *curves* that are otherwise impossible to produce with lines or boxes. When enabling the `curved` parameter of the `polyline.new()` function, the resulting polyline interpolates *nonlinear* values between the coordinates from each `chart.point` in its array of `points` to generate a curvy effect.

For instance, the “Oscillating polyline” script in our previous example uses *straight* line segments to produce a drawing resembling a triangle wave, meaning a waveform that zig-zags between its peaks and valleys. If we set the `curved` parameter in the `polyline.new()` call from that example to `true`, the resulting drawing would connect the points using *curved* segments, producing a smooth, nonlinear shape similar to a sine wave:

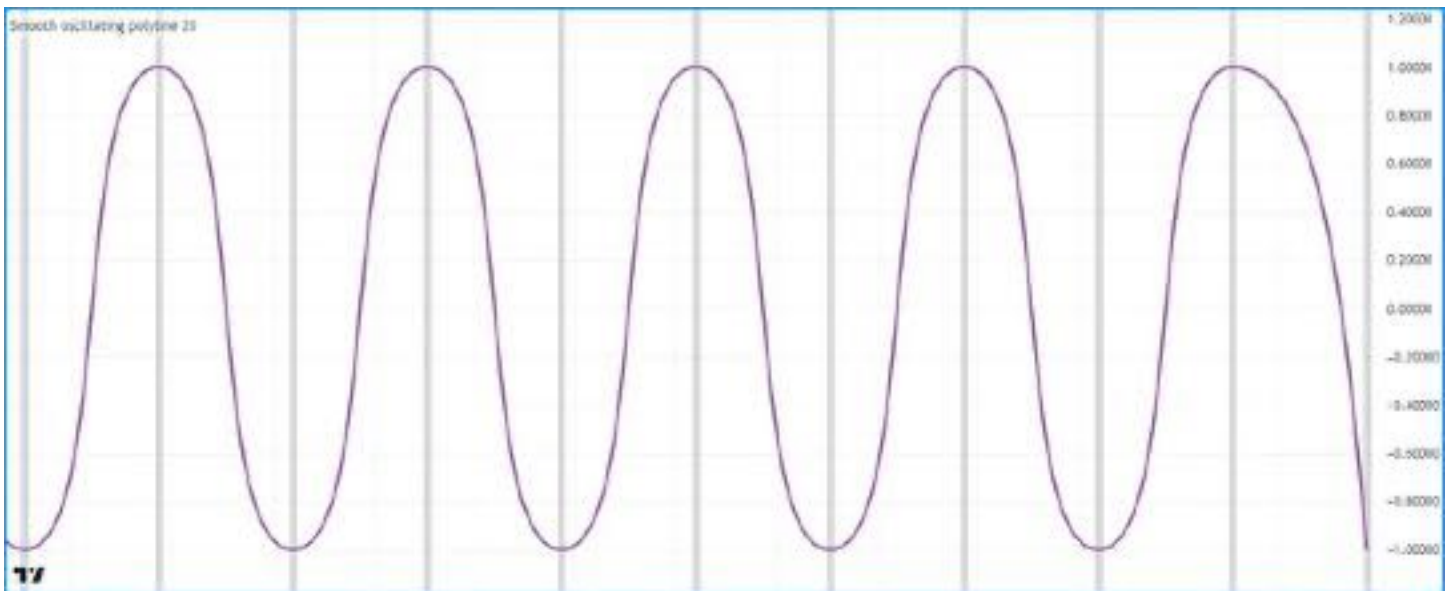


Figure 159: image

```
//@version=6
indicator("Curved drawings demo", "Smooth oscillating polyline")
```

```

//@variable The number of bars between each point in the drawing.
int length = input.int(20, "Length between points", 2)

//@variable An array of `chart.point` objects to sequentially connect with a polyline.
var points = array.new<chart.point>()

//@variable The y-coordinate of each point in the `points`. Alternates between 1 and -1 on each `newPoint`.
var int yValue = 1

//@variable Is `true` once every `length` bars, `false` otherwise.
bool newPoint = bar_index % length == 0

if newPoint
    // Push a new `chart.point` into the `points`. The new point contains `time` and `index` info.
    points.push(chart.point.now(yValue))
    // Change the sign of the `yValue`.
    yValue *= -1

// Draw a new curved `polyline` on the last confirmed historical chart bar.
// The polyline uses the `time` field from each `chart.point` in the `points` array as x-coordinates.
if barstate.islastconfirmedhistory
    polyline.new(points, curved = true, xloc = xloc.bar_time, line_color = #9151A6, line_width = 3)

// Highlight the chart background on every `newPoint` condition.
bgcolor(newPoint ? color.new(color.gray, 70) : na, title = "New point highlight")

```

Notice that in this example, the smooth curves have relatively consistent behavior, and no portion of the drawing extends past its defined coordinates, which is not always the case when drawing curved polylines. The data used to construct a polyline heavily impacts the smooth, piecewise function it interpolates between its points. In some cases, the interpolated curve *can* reach beyond its actual coordinates.

Let's add some variation to the chart points in our example's **points** array to demonstrate this behavior. In the version below, the script multiplies the **yValue** by a random value in the **chart.point.now()** calls.

To visualize the behavior, this script also creates a horizontal line at the **price** value from each **chart.point** in the **points** array, and it displays another polyline connecting the same points with straight line segments. As we see on the chart, both polylines pass through all coordinates from the **points** array. However, the curvy polyline occasionally reaches *beyond* the vertical boundaries indicated by the horizontal lines, whereas the polyline drawn using straight segments does not:

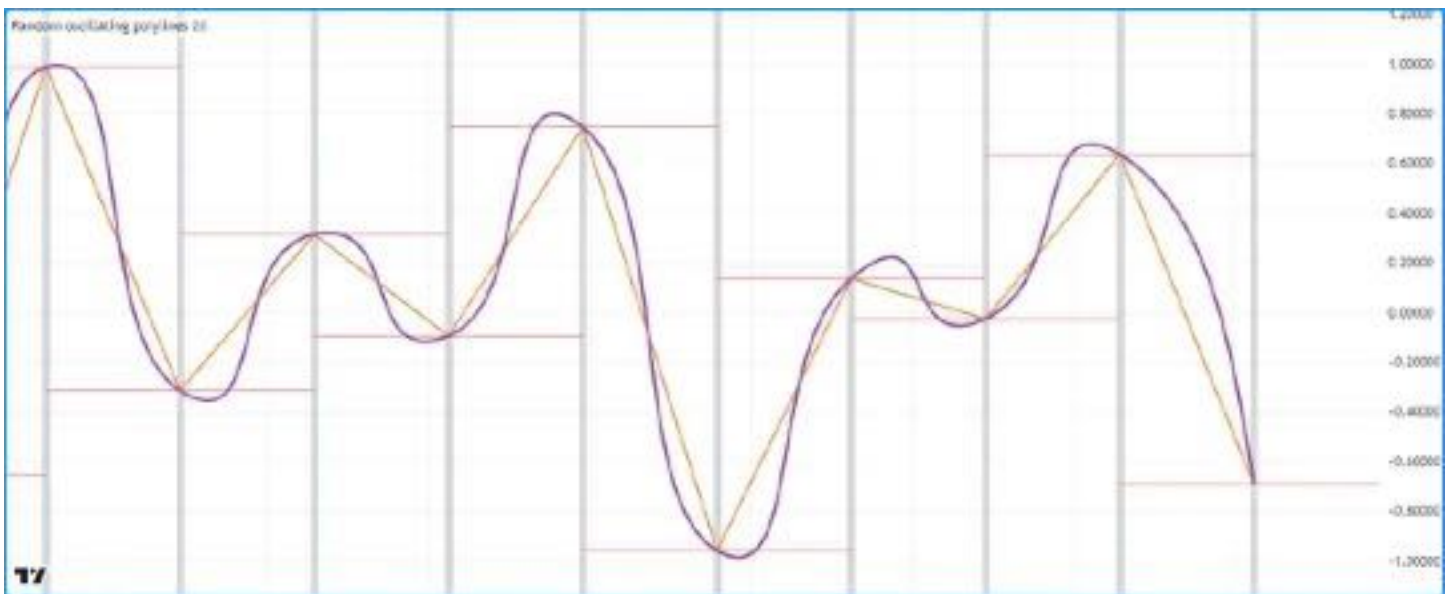


Figure 160: image

```

//@version=6

```

```

indicator("Curved drawings demo", "Random oscillating polylines")

//@variable The number of bars between each point in the drawing.
int length = input.int(20, "Length between points", 2)

//@variable An array of `chart.point` objects to sequentially connect with a polyline.
var points = array.new<chart.point>()

//@variable The sign of each `price` in the `points`. Alternates between 1 and -1 on each `newPoint`.
var int yValue = 1

//@variable Is `true` once every `length` bars.
bool newPoint = bar_index % length == 0

if newPoint
    // Push a new `chart.point` with a randomized `price` into the `points`.
    // The new point contains `time` and `index` info.
    points.push(chart.point.now(yValue * math.random()))
    // Change the sign of the `yValue`.
    yValue *= -1

    //@variable The newest `chart.point`.
    lastPoint = points.last()
    // Draw a horizontal line at the `lastPoint.price`. This line uses the default `xloc.bar_index`.
    line.new(lastPoint.index - length, lastPoint.price, lastPoint.index + length, lastPoint.price, color = col

// Draw two `polyline` instances on the last confirmed chart bar.
// Both polylines use the `time` field from each `chart.point` in the `points` array as x-coordinates.
if barstate.islastconfirmedhistory
    polyline.new(points, curved = false, xloc = xloc.bar_time, line_color = #EB8A3B, line_width = 2)
    polyline.new(points, curved = true, xloc = xloc.bar_time, line_color = #9151A6, line_width = 3)

// Highlight the chart background on every `newPoint` condition.
bgcolor(newPoint ? color.new(color.gray, 70) : na, title = "New point highlight")

```

Closed shapes Since a single polyline can contain numerous straight or curved line segments, and the `closed` parameter allows the drawing to connect the coordinates from the first and last `chart.point` in its array of `points`, we can use polylines to draw many different types of closed polygonal shapes.

Let's draw some polygons in Pine. The following script periodically draws randomized polygons centered at `hl2` price values.

On each occurrence of the `newPolygon` condition, it clears the `points` array, calculates the `numberOfSides` and `rotationOffset` of the new polygon drawing based on `math.random()` values, then uses a for loop to push `numberOfSides` new chart points into the array that contain stepped coordinates from an elliptical path with `xScale` and `yScale` semi-axes. The script draws the polygon by connecting each `chart.point` from the `points` array using a *closed polyline* with straight line segments:

```

//@version=6
indicator("Closed shapes demo", "N-sided polygons", true)

//@variable The size of the horizontal semi-axis.
float xScale = input.float(3.0, "X scale", 1.0)
//@variable The size of the vertical semi-axis.
float yScale = input.float(1.0, "Y scale") * ta.atr(2)

//@variable An array of `chart.point` objects containing vertex coordinates.
var points = array.new<chart.point>()

//@variable The condition that triggers a new polygon drawing. Based on the horizontal axis to prevent overlap
bool newPolygon = bar_index % int(math.round(2 * xScale)) == 0 and barstate.isconfirmed

```

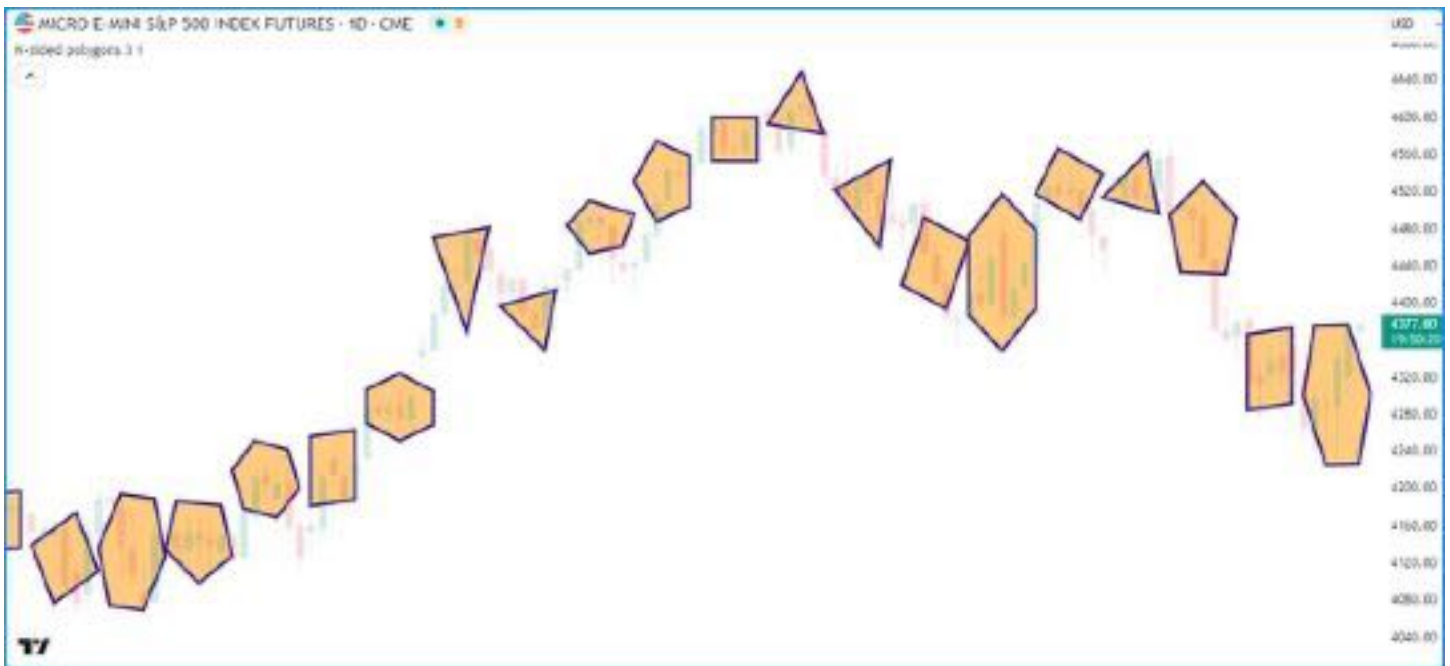


Figure 161: image

```

if newPolygon
    // Clear the `points` array.
    points.clear()

    //@variable The number of sides and vertices in the new polygon.
    int numberOfSides = int(math.random(3, 7))
    //@variable A random rotation offset applied to the new polygon, in radians.
    float rotationOffset = math.random(0.0, 2.0) * math.pi
    //@variable The size of the angle between each vertex, in radians.
    float step = 2 * math.pi / numberOfSides

    //@variable The counter-clockwise rotation angle of each vertex.
    float angle = rotationOffset

    for i = 1 to numberOfSides
        //@variable The approximate x-coordinate from an ellipse at the `angle`, rounded to the nearest integer.
        int xValue = int(math.round(xScale * math.cos(angle))) + bar_index
        //@variable The y-coordinate from an ellipse at the `angle`.
        float yValue = yScale * math.sin(angle) + hl2

        // Push a new `chart.point` containing the `xValue` and `yValue` into the `points` array.
        // The new point does not contain `time` information.
        points.push(chart.point.from_index(xValue, yValue))
        // Add the `step` to the `angle`.
        angle += step

    // Draw a closed polyline connecting the `points`.
    // The polyline uses the `index` field from each `chart.point` in the `points` array.
    polyline.new(
        points, closed = true, line_color = color.navy, fill_color = color.new(color.orange, 50), line_width :
    )

```

Note that:

- This example shows the last ~50 polylines on the chart, as we have not specified a `max_polylines_count` value in the `indicator()` function call.

- The `yScale` calculation multiplies an `input.float()` by `ta.ATR(2)` to adapt the vertical scale of the drawings to recent price ranges.
- The resulting polygons have a maximum width of twice the horizontal semi-axis ($2 * xScale$), rounded to the nearest integer. The `newPolygon` condition uses this value to prevent the polygon drawings from overlapping.
- The script rounds the `xValue` calculation to the nearest integer because the `index` field of a `chart.point` only accepts an int value, as the x-axis of the chart does not include fractional bar indices.

Deleting polylines

To delete a specific polyline `id`, use `polyline.delete()`. This function removes the polyline object from the script and its drawing on the chart.

As with other drawing objects, we can use `polyline.delete()` to maintain a specific number of polyline drawings or conditionally remove drawings from a chart.

For example, the script below periodically draws approximate arithmetic spirals and stores their polyline IDs in an array, which it uses as a queue to manage the number of drawings it displays.

When the `newSpiral` condition occurs, the script creates a `points` array and adds chart points within a for loop. On each loop iteration, it calls the `spiralPoint()` user-defined function to create a new `chart.point` containing stepped values from an elliptical path that grows with respect to the `angle`. The script then creates a randomly colored *curved polyline* connecting the coordinates from the `points` and pushes its ID into the `polylines` array.

When the array's size exceeds the specified `numberOfSpirals`, the script removes the oldest polyline using `array.shift()` and deletes the object using `polyline.delete()`:

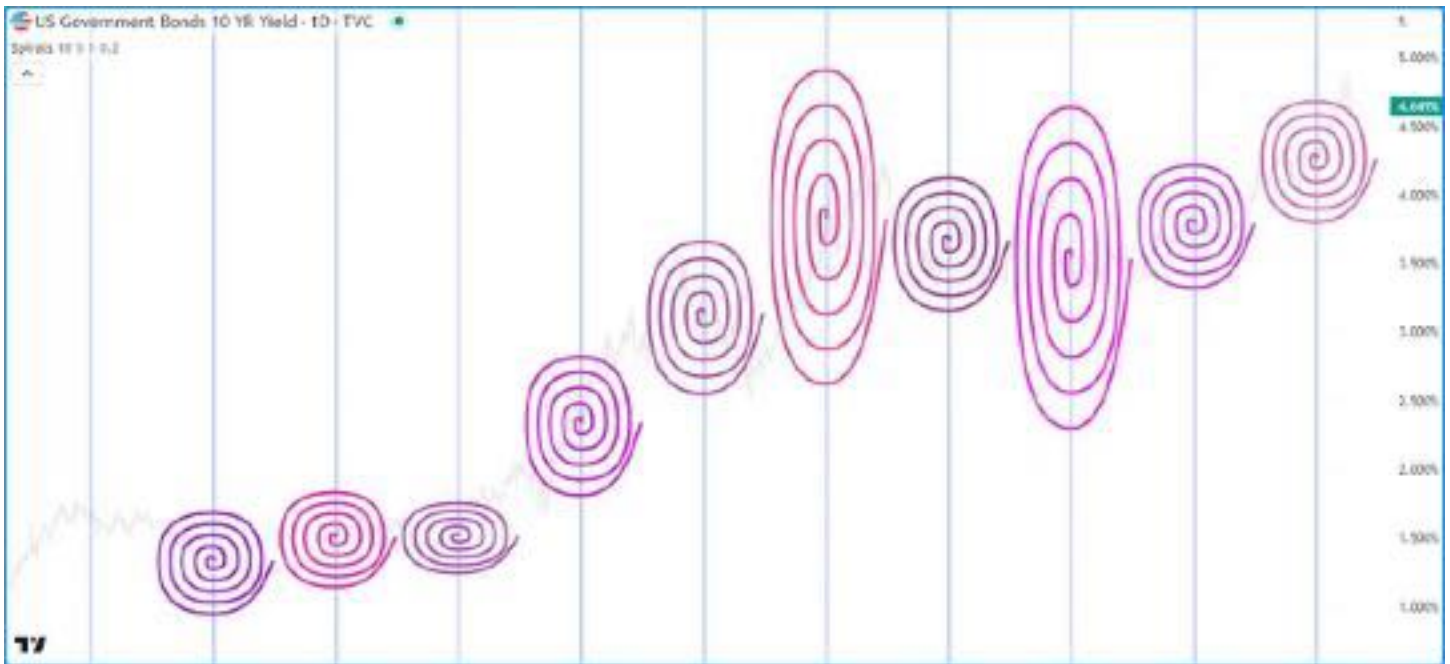


Figure 162: image

```
//@version=6

//@variable The maximum number of polylines allowed on the chart.
const int MAX_POLYLINES_COUNT = 100

indicator("Deleting polylines example", "Spirals", true, max_polylines_count = MAX_POLYLINES_COUNT)

//@variable The number of spiral drawings on the chart.
int numberOfSpirals = input.int(10, "Spirals shown", 1, MAX_POLYLINES_COUNT)
//@variable The number of full spiral rotations to draw.
int rotations = input.int(5, "Rotations", 1)
//@variable The scale of the horizontal semi-axis.
float xScale = input.float(1.0, "X scale")
```

```

//@variable The scale of the vertical semi-axis.
float yScale = input.float(0.2, "Y scale") * ta.atr(2)

//@function Calculates an approximate point from an elliptically-scaled arithmetic spiral.
//@returns A `chart.point` with `index` and `price` information.
spiralPoint(float angle, int xOffset, float yOffset) =>
    result = chart.point.from_index(
        int(math.round(angle * xScale * math.cos(angle))) + xOffset,
        angle * yScale * math.sin(angle) + yOffset
    )

//@variable An array of polylines.
var polylines = array.new<polyline>()

//@variable The condition to create a new spiral.
bool newSpiral = bar_index % int(math.round(4 * math.pi * rotations * xScale)) == 0

if newSpiral
    //@variable An array of `chart.point` objects for the `spiral` drawing.
    points = array.new<chart.point>()
    //@variable The counter-clockwise angle between calculated points, in radians.
    float step = math.pi / 2
    //@variable The rotation angle of each calculated point on the spiral, in radians.
    float theta = 0.0
    // Loop to create the spiral's points. Creates 4 points per full rotation.
    for i = 0 to rotations * 4
        //@variable A new point on the calculated spiral.
        chart.point newPoint = spiralPoint(theta, bar_index, ohlc4)
        // Add the `newPoint` to the `points` array.
        points.push(newPoint)
        // Add the `step` to the `theta` angle.
        theta += step

    //@variable A random color for the new `spiral` drawing.
    color spiralColor = color.rgb(math.random(150, 255), math.random(0, 100), math.random(150, 255))
    //@variable A new polyline connecting the spiral points. Uses the `index` field from each point as x-coord
    polyline spiral = polyline.new(points, true, line_color = spiralColor, line_width = 3)

    // Push the new `spiral` into the `polylines` array.
    polylines.push(spiral)
    // Shift the first polyline out of the array and delete it when the array's size exceeds the `numberOfSpirals`
    if polylines.size() > numberOfSpirals
        polyline.delete(polylines.shift())

// Highlight the background when `newSpiral` is `true`.
bgcolor(newSpiral ? color.new(color.blue, 70) : na, title = "New drawing highlight")

```

Note that:

- We declared a `MAX_POLYLINES_COUNT` global variable with a constant value of 100. The script uses this constant as the `max_polylines_count` value in the `indicator()` function and the `maxval` of the `numberOfSpirals` input.
- As with our “N-sided polygons” example in the previous section, we round the calculation of x-coordinates to the nearest integer since the `index` field of a `chart.point` can only accept an int value.
- Despite the smooth appearance of the drawings, each polyline’s `points` array only contains *four* `chart.point` objects per spiral rotation. Since the `polyline.new()` call includes `curved = true`, each polyline uses *smooth curves* to connect their `points`, producing a visual approximation of the spiral’s actual curvature.
- The width of each spiral is approximately `4 * math.pi * rotations * xScale`, rounded to the nearest integer. We use this value in the `newSpiral` condition to space each drawing and prevent overlaps.

Redrawing polylines

It may be desirable in some cases to change a polyline drawing throughout a script's execution. While the `polyline.*` namespace does not contain built-in setter functions, we can *redraw* polylines referenced by variables or collections by *deleting* the existing polylines and assigning *new instances* with the desired changes.

The following example uses `polyline.delete()` and `polyline.new()` calls to update the value of a polyline variable.

This script draws closed polylines that connect the open, high, low, and close points of periods containing `length` bars. It creates a `currentDrawing` variable on the first bar and assigns a polyline ID to it on every chart bar. It uses the `openPoint`, `highPoint`, `lowPoint`, and `closePoint` variables to reference chart points that track the period's developing OHLC values. As new values emerge, the script assigns new `chart.point` objects to the variables, collects them in an array using `array.from`, then creates a new polyline connecting the coordinates from the array's points and assigns it to the `currentDrawing`.

When the `newPeriod` condition is `false` (i.e., the current period is not complete), the script deletes the polyline referenced by the `currentDrawing` before creating a new one, resulting in a dynamic drawing that changes over the developing period:

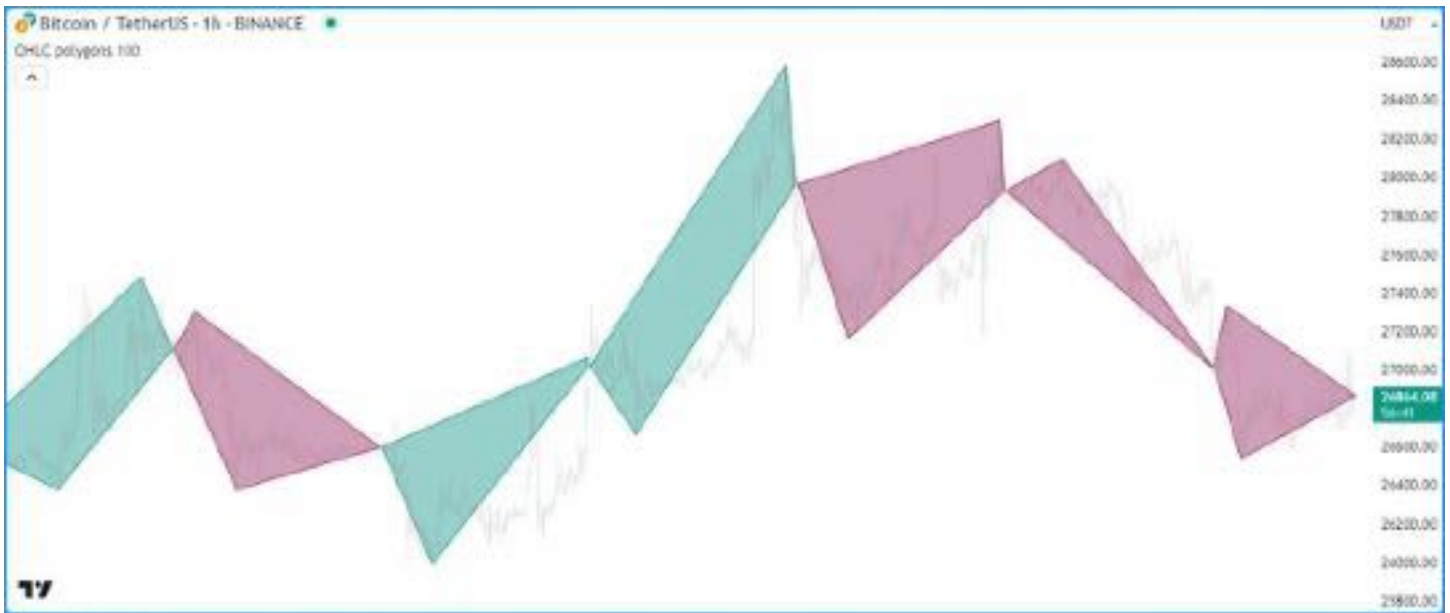


Figure 163: image

```
//@version=6
indicator("Redrawing polylines demo", "OHLC polygons", true, max_polylines_count = 100)

//@variable The length of the period.
int length = input.int(100, "Length", 1)

//@variable A `chart.point` representing the start of each period.
var chart.point openPoint = na
//@variable A `chart.point` representing the highest point of each period.
var chart.point highPoint = na
//@variable A `chart.point` representing the lowest point of each period.
var chart.point lowPoint = na
//@variable A `chart.point` representing the current bar's closing point.
closePoint = chart.point.now(close)

//@variable The current period's polyline drawing.
var polyline currentDrawing = na

//@variable Is `true` once every `length` bars.
bool newPeriod = bar_index % length == 0

if newPeriod
    // Assign new chart points to the `openPoint`, `highPoint`, and `closePoint`.
```

```

openPoint := chart.point.now(open)
highPoint := chart.point.now(high)
lowPoint  := chart.point.now(low)
else
    // Assign a new `chart.point` to the `highPoint` when the `high` is greater than its `price`.
    if high > highPoint.price
        highPoint := chart.point.now(high)
    // Assign a new `chart.point` to the `lowPoint` when the `low` is less than its `price`.
    if low < lowPoint.price
        lowPoint := chart.point.now(low)

//@variable Is teal when the `closePoint.price` is greater than the `openPoint.price`, maroon otherwise.
color drawingColor = closePoint.price > openPoint.price ? color.teal : color.maroon

// Delete the polyline assigned to the `currentDrawing` if it's not a `newPeriod`.
if not newPeriod
    polyline.delete(currentDrawing)
// Assign a new polyline to the `currentDrawing`.
// Uses the `index` field from each `chart.point` in its array as x-coordinates.
currentDrawing := polyline.new(
    array.from(openPoint, highPoint, closePoint, lowPoint), closed = true,
    line_color = drawingColor, fill_color = color.new(drawingColor, 60)
)

```

Realtime behavior

Lines, boxes, and polylines are subject to both *commit* and *rollback* actions, which affect the behavior of a script when it executes on a realtime bar. See the page on Pine Script™'s Execution model.

This script demonstrates the effect of rollback when it executes on the realtime, *unconfirmed* chart bar:



Figure 164: image

```

//@version=6
indicator("Realtime behavior demo", overlay = true)

//@variable Is orange when the `line` is subject to rollback and gray after the `line` is committed.
color lineColor = barstate.isconfirmed ? color.gray : color.orange

line.new(bar_index, hl2, bar_index + 1, hl2, color = lineColor, width = 4)

```

The `line.new()` call in this example creates a new line ID on each iteration when values change on the unconfirmed bar. The script automatically deletes the objects created on each change in that bar because of the *rollback* before each iteration. It only *commits* the last line created before the bar closes, and that line instance is the one that persists on the confirmed bar.

Limitations

Total number of objects

Lines, boxes, and polylines consume server resources, which is why there are limits on the total number of drawings per script. When a script creates more drawing objects than the allowed limit, the Pine Script™ runtime automatically deletes the oldest ones in a process referred to as *garbage collection*.

A single script can contain up to 500 lines, 500 boxes, and 100 polylines. Users can control the garbage collection limits by specifying the `max_lines_count`, `max_boxes_count`, and `max_polylines_count` values in their script's `indicator()` or `strategy()` declaration statement.

This script demonstrates how garbage collection works in Pine. It creates a new line, box, and polyline on each chart bar. We haven't specified values for the `max_lines_count`, `max_boxes_count`, or `max_polylines_count` parameters in the `indicator()` function call, so the script will maintain the most recent ~50 lines, boxes, and polylines on the chart, as this is the default setting for each parameter:



Figure 165: image

```
//@version=6
indicator("Garbage collection demo", overlay = true)

//@variable A new `chart.point` at the current `bar_index` and `high`.
firstPoint = chart.point.now(high)
//@variable A new `chart.point` one bar into the future at the current `low`.
secondPoint = chart.point.from_index(bar_index + 1, low)
//@variable A new `chart.point` one bar into the future at the current `high`.
thirdPoint = chart.point.from_index(bar_index + 1, high)

// Draw a new `line` connecting the `firstPoint` to the `secondPoint`.
line.new(firstPoint, secondPoint, color = color.red, width = 2)
// Draw a new `box` with the `firstPoint` top-left corner and `secondPoint` bottom-right corner.
box.new(firstPoint, secondPoint, color.purple, 2, bgcolor = na)
// Draw a new `polyline` connecting the `firstPoint`, `secondPoint`, and `thirdPoint` sequentially.
polyline.new(array.from(firstPoint, secondPoint, thirdPoint), true, line_width = 2)
```

Note that:

- We've used TradingView's "Measure" drawing tool to measure the number of bars covered by the script's drawing objects.

Future references with `xloc.bar_index`

Objects positioned using `xloc.bar_index` can contain x-coordinates no further than 500 bars into the future.

Other contexts

Scripts cannot use lines, boxes, or polylines in `request.*()` functions. Instances of these types can use the values from `request.*()` calls, but scripts can only create and draw them in the chart's context.

This limitation is also why drawing objects will not work when using the `timeframe` parameter in the `indicator()` declaration statement.

Historical buffer and `maxBarsBack`

Using `barstate.isrealtime` in combination with drawings may sometimes produce unexpected results. For example, the intention of this script is to ignore all historical bars and draw horizontal lines spanning 300 bars back on *realtime* bars:

```
//@version=6
indicator("Historical buffer demo", overlay = true)

//@variable A `chart.point` at the `bar_index` from 300 bars ago and current `close`.
firstPoint = chart.point.from_index(bar_index[300], close)
//@variable The current bar's `chart.point` containing the current `close`.
secondPoint = chart.point.now(close)

// Draw a new line on realtime bars.
if barstate.isrealtime
    line.new(firstPoint, secondPoint)
```

However, it will fail at runtime and raise an error. The script fails because it cannot determine the buffer size for historical values of the underlying time series. Although the code doesn't contain the built-in time variable, the built-in `bar_index` uses the time series in its inner workings. Therefore, accessing the value of the `bar_index` from 300 bars back requires the history buffer of the time series to be at least 300 bars.

Pine Script™ includes a mechanism that detects the required historical buffer size automatically in most cases. It works by letting the script access historical values any number of bars back for a limited duration. In this script's case, using `barstate.isrealtime` to control the drawing of lines prevents it from accessing the historical series, so it cannot infer the required historical buffer size, and the script fails.

The simple solution to this issue is to use the `maxBarsBack()` function to *explicitly define* the historical buffer of the time series before evaluating the conditional structure:

```
//@version=6
indicator("Historical buffer demo", overlay = true)

//@variable A `chart.point` at the `bar_index` from 300 bars ago and current `close`.
firstPoint = chart.point.from_index(bar_index[300], close)
//@variable The current bar's `chart.point` containing the current `close`.
secondPoint = chart.point.now(close)

// Explicitly set the historical buffer of the `time` series to 300 bars.
maxBarsBack(time, 300)

// Draw a new line on realtime bars.
if barstate.isrealtime
    line.new(firstPoint, secondPoint)
```

Such issues can be confusing, but they're quite rare. The Pine Script™ team hopes to eliminate them over time.

[Previous

Libraries](#libraries)[Next

Non-standard charts data](#non-standard-charts-data) User Manual/Concepts/Non-standard charts data

Non-standard charts data

Introduction

Pine Script™ features several `ticker.*()` functions that generate *ticker identifiers* for requesting data from *non-standard* chart feeds. The available functions that create these ticker IDs are `ticker.heikinashi()`, `ticker.renko()`, `ticker.linebreak()`, `ticker.kagi()`, and `ticker.pointfigure()`. Scripts can use these functions' returned values as the `symbol` argument in `request.security()` calls to access non-standard chart data while running on *any* chart type.

`ticker.heikinashi()`

Heikin-Ashi means *average bar* in Japanese. The open/high/low/close values of Heikin-Ashi candlesticks are synthetic; they are not actual market prices. They are calculated by averaging combinations of real OHLC values from the current and previous bar. The calculations used make Heikin-Ashi bars less noisy than normal candlesticks. They can be useful to make visual assessments, but are unsuited to backtesting or automated trading, as orders execute on market prices — not Heikin-Ashi prices.

The `ticker.heikinashi()` function creates a special ticker identifier for requesting Heikin-Ashi data with the `request.security()` function.

This script requests the close value of Heikin-Ashi bars and plots them on top of the normal candlesticks:



Figure 166: image

```
//@version=6
indicator("HA Close", "", true)
haTicker = ticker.heikinashi(syminfo.tickerid)
haClose = request.security(haTicker, timeframe.period, close)
plot(haClose, "HA Close", color.black, 3)
```

Note that:

- The close values for Heikin-Ashi bars plotted as the black line are very different from those of real candles using market prices. They act more like a moving average.
- The black line appears over the chart bars because we have selected “Visual Order/Bring to Front” from the script’s “More” menu.

If you wanted to omit values for extended hours in the last example, an intermediary ticker without extended session information would need to be created first:

```
//@version=6
indicator("HA Close", "", true)
```



Figure 167: image

```
regularSessionTicker = ticker.new(syminfo.prefix, syminfo.ticker, session.regular)
haTicker = ticker.heikinashi(regularSessionTicker)
haClose = request.security(haTicker, timeframe.period, close, gaps = barmerge.gaps_on)
plot(haClose, "HA Close", color.black, 3, plot.style_linebr)
```

Note that:

- We use the `ticker.new()` function first, to create a ticker without extended session information.
- We use that ticker instead of `syminfo.tickerid` in our `ticker.heikinashi()` call.
- In our `request.security()` call, we set the `gaps` parameter's value to `barmerge.gaps_on`. This instructs the function not to use previous values to fill slots where data is absent. This makes it possible for it to return `na` values outside of regular sessions.
- To be able to see this on the chart, we also need to use a special `plot.style_linebr` style, which breaks the plots on `na` values.

This script plots Heikin-Ashi candles under the chart:

```
//@version=6
indicator("Heikin-Ashi candles")
CANDLE_GREEN = #26A69A
CANDLE_RED = #EF5350

haTicker = ticker.heikinashi(syminfo.tickerid)
[haO, haH, haL, haC] = request.security(haTicker, timeframe.period, [open, high, low, close])
candleColor = haC >= haO ? CANDLE_GREEN : CANDLE_RED
plotcandle(haO, haH, haL, haC, color = candleColor)
```

Note that:

- We use a tuple with `request.security()` to fetch four values with the same call.
- We use `plotcandle()` to plot our candles. See the Bar plotting page for more information.

`ticker.renko()`

Renko bars only plot price movements, without taking time or volume into consideration. They look like bricks stacked in adjacent columns. A new brick is only drawn after the price passes the top or bottom by a predetermined amount. The



Figure 168: image

`ticker.renko()` function creates a ticker id which can be used with `request.security()` to fetch Renko values, but there is no Pine Script™ function to draw Renko bars on the chart:

```
//@version=6
indicator("", "", true)
renkoTicker = ticker.renko(syminfo.tickerid, "ATR", 10)
renkoLow = request.security(renkoTicker, timeframe.period, low)
plot(renkoLow)
```

ticker.linebreak()

The *Line Break* chart type displays a series of vertical boxes that are based on price changes. The `ticker.linebreak()` function creates a ticker id which can be used with `request.security()` to fetch “Line Break” values, but there is no Pine Script™ function to draw such bars on the chart:

```
//@version=6
indicator("", "", true)
lineBreakTicker = ticker.linebreak(syminfo.tickerid, 3)
lineBreakClose = request.security(lineBreakTicker, timeframe.period, close)
plot(lineBreakClose)
```

ticker.kagi()

Kagi charts are made of a continuous line that changes directions. The direction changes when the price changes beyond a predetermined amount. The `ticker.kagi()` function creates a ticker id which can be used with `request.security()` to fetch “Kagi” values, but there is no Pine Script™ function to draw such bars on the chart:

```
//@version=6
indicator("", "", true)
kagiBreakTicker = ticker.linebreak(syminfo.tickerid, 3)
kagiBreakClose = request.security(kagiBreakTicker, timeframe.period, close)
plot(kagiBreakClose)
```


ticker.pointfigure()

Point and Figure (PnF) charts only plot price movements, without taking time into consideration. A column of X's is plotted as the price rises, and O's are plotted when price drops. The `ticker.pointfigure()` function creates a ticker id which can be used with `request.security()` to fetch “PnF” values, but there is no Pine Script™ function to draw such bars on the chart. Every column of X's or O's is represented with four numbers. You may think of them as synthetic OHLC PnF values:

```
//@version=6
indicator("", "", true)
pnfTicker = ticker.pointfigure(syminfo.tickerid, "hl", "ATR", 14, 3)
[pnf0, pnfC] = request.security(pnfTicker, timeframe.period, [open, close], barmerge.gaps_on)
plot(pnf0, "PnF Open", color.green, 4, plot.style_linebr)
plot(pnfC, "PnF Close", color.red, 4, plot.style_linebr)
```

[Previous

Lines and boxes](#lines-and-boxes)[Next

Other timeframes and data](#other-timeframes-and-data) User Manual/Concepts/Other timeframes and data

Other timeframes and data

Introduction

Pine Script™ allows users to request data from sources and contexts other than those their charts use. The functions we present on this page can fetch data from a variety of alternative sources:

- `request.security()` retrieves data from another symbol, timeframe, or other context.
- `request.security_lower_tf()` retrieves *intrabar* data, i.e., data from a timeframe lower than the chart timeframe.
- `request.currency_rate()` requests a *daily rate* to convert a value expressed in one currency to another.
- `request.dividends()`, `request.splits()`, and `request.earnings()` respectively retrieve information about an issuing company's dividends, splits, and earnings.
- `request.financial()` retrieves financial data from FactSet.
- `request.economic()` retrieves economic and industry data.
- `request.seed()` retrieves data from a *user-maintained* GitHub repository.

These are the signatures of the functions in the `request.*` namespace:

```
request.security(symbol, timeframe, expression, gaps, lookahead, ignore_invalid_symbol, currency, calc_bars_count) → series float
request.security_lower_tf(symbol, timeframe, expression, ignore_invalid_symbol, currency, ignore_invalid_timeframe) → series float
request.currency_rate(from, to, ignore_invalid_currency) → series float
request.dividends(ticker, field, gaps, lookahead, ignore_invalid_symbol, currency) → series float
request.splits(ticker, field, gaps, lookahead, ignore_invalid_symbol) → series float
request.earnings(ticker, field, gaps, lookahead, ignore_invalid_symbol, currency) → series float
request.financial(symbol, financial_id, period, gaps, ignore_invalid_symbol, currency) → series float
request.economic(country_code, field, gaps, ignore_invalid_symbol) → series float
request.seed(source, symbol, expression, ignore_invalid_symbol, calc_bars_count) → series <type>
```

The `request.*()` family of functions has numerous potential applications. Throughout this page, we discuss in detail these functions and some of their typical use cases.

Common characteristics

Many functions in the `request.*()` namespace share some common properties and parameters. Before we explore each function in depth, let's familiarize ourselves with these characteristics.

Behavior

All `request.*()` functions have similar internal behavior, even though they do not all share the same required parameters. Every unique `request.*()` call in a script requests a dataset from a defined *context* (i.e., ticker ID and timeframe) and evaluates an *expression* across the retrieved data.

The `request.security()` and `request.security_lower_tf()` functions allow programmers to specify the context of a request and the expression directly via the `symbol`, `timeframe`, and `expression` parameters, making them suitable for a wide range of data requests.

For example, the `request.security()` call in this simple script requests daily “AMEX:SPY” data, and it calculates the slope of a 20-bar linear regression line using the retrieved `hl2` prices. The first two arguments specify the context of the request, and the third specifies the expression to evaluate across the requested data:

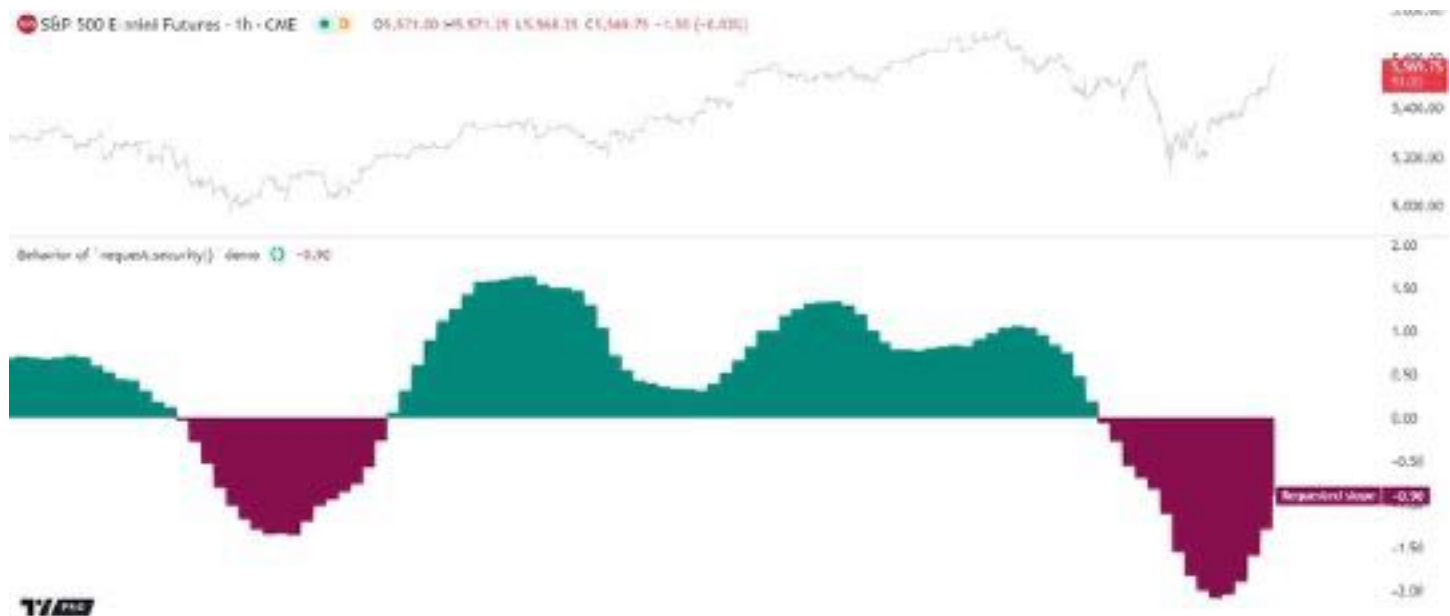


Figure 169: image

```
//@version=6
indicator("Behavior of `request.security()` demo")

//@variable The 20-bar linear regression slope of `hl2` prices from the "AMEX:SPY" symbol on the "1D" timeframe
float requestedSlope = request.security("AMEX:SPY", "1D", ta.linreg(hl2, 20, 0) - ta.linreg(hl2, 20, 1))

//@variable Is `color.teal` when the `requestedSlope` is positive, and `color.maroon` otherwise.
color plotColor = requestedSlope > 0 ? color.teal : color.maroon

// Plot the `requestedSlope` with the `plotColor`.
plot(requestedSlope, "Requested slope", plotColor, 1, plot.style_area)
```

Other functions within the `request.*()` namespace *do not* allow programmers to directly define the full context of a request or the evaluated expression. Instead, these functions determine some of the necessary information *internally* because they perform only specific types of requests.

For instance, `request.financial()` exclusively retrieves periodic financial data. Its required parameters (`symbol`, `financial_id`, and `period`) all define parts of a specific financial *ticker ID*. The function does not allow specification of the timeframe or expression, as it determines these details internally. The script below demonstrates a simple call to this function that retrieves the annual cost of goods data for the chart symbol’s issuing company:

```
//@version=6
indicator("Behavior of `request.financial()` demo", format = format.volume)

//@variable The annual cost of goods sold by the chart symbol's issuing company.
float costOfGoods = request.financial(syminfo.tickerid, "COST_OF_GOODS", "FY")

// Plot the `costOfGoods`.
plot(costOfGoods, "Cost of goods", color.purple, 3, plot.style_stepline_diamond)
```

Scripts can perform up to 40 unique requests using any combination of `request.*()` function calls. Only unique `request.*()` calls count toward this limit because they are the only calls that fetch *new data*. Redundant calls to the same `request.*()`

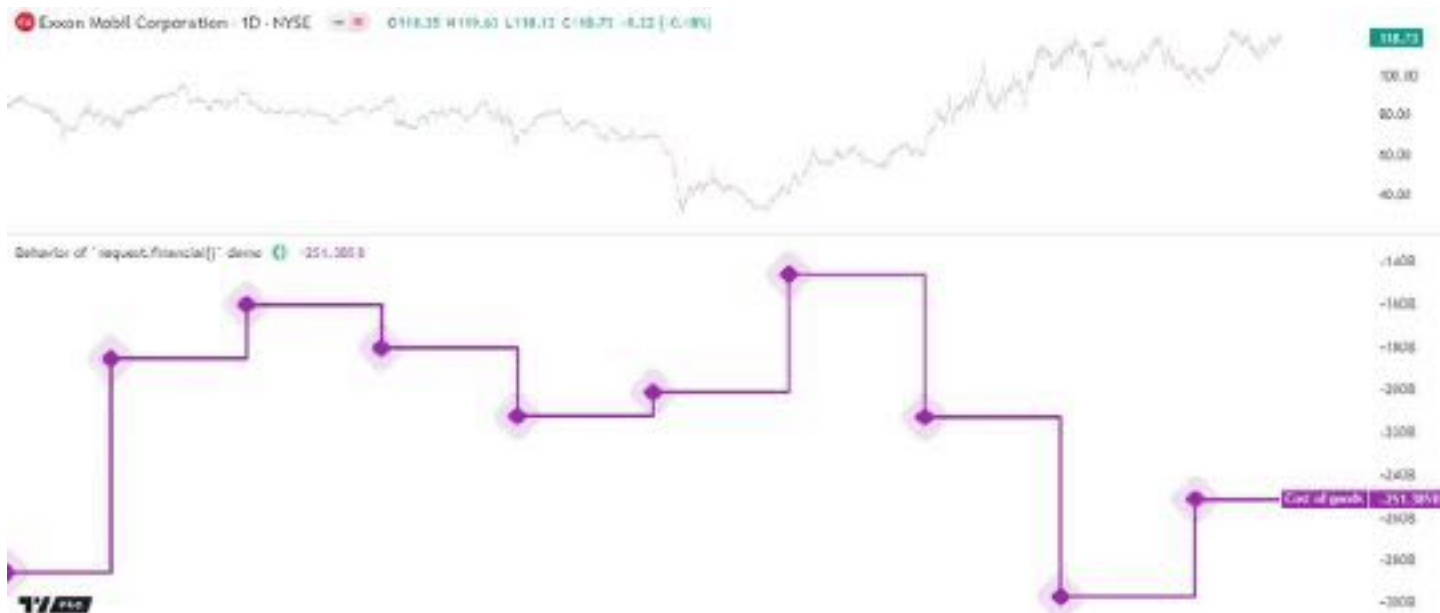


Figure 170: image

function with identical arguments *do not* retrieve new data. Instead, they *reuse* the data from the first executed call. See this section of the Limitations page for more information.

gaps

When using a `request.*()` function to retrieve data from another context, the data may not come in on each new bar as it would with the current chart. The `gaps` parameter of a `request.*()` function controls how the function responds to nonexistent values in the requested series.

Suppose we have a script executing on an 1-minute chart that requests hourly data for the chart's symbol using `request.security()`. The function call returns new values only on the 1-minute bars that cover the opening and closing times of the symbol's hourly bars. On other chart bars, we can decide whether the function returns `na` values or the last available values via the `gaps` parameter.

When the `gaps` parameter uses `barmerge.gaps_on`, the function returns `na` results on all chart bars where new data is not yet confirmed from the requested context. Otherwise, when the parameter uses `barmerge.gaps_off`, the function fills the gaps in the requested data with the last confirmed values on historical bars and the most recent developing values on realtime bars.

The script below demonstrates the difference in behavior by plotting the results from two `request.security()` calls that fetch the close price of the current symbol from the hourly timeframe on a 1-minute chart. The first call uses `gaps = barmerge.gaps_off` and the second uses `gaps = barmerge.gaps_on`:

```
//@version=6
indicator("`gaps` demo", overlay = true)

//@variable The `close` requested from the hourly timeframe without gaps.
float dataWithoutGaps = request.security(syminfo.tickerid, "60", close, gaps = barmerge.gaps_off)
//@variable The `close` requested from the hourly timeframe with gaps.
float dataWithGaps = request.security(syminfo.tickerid, "60", close, gaps = barmerge.gaps_on)

// Plot the requested data.
plot(dataWithoutGaps, "Data without gaps", color.blue, 3, plot.style_linebr)
plot(dataWithGaps, "Data with gaps", color.purple, 15, plot.style_linebr)

// Highlight the background for realtime bars.
bgcolor(barstate.isrealtime ? color.new(color.aqua, 70) : na, title = "Realtime bar highlight")
```

Note that:

- `barmerge.gaps_off` is the default value for the `gaps` parameter in all applicable `request.*()` functions.



Figure 171: image

- The script plots the requested series as lines with breaks (`plot.style_linebr`), which do not bridge over na values as the default style (`plot.style_line`) does.
- When using `barmerge.gaps_off`, the `request.security()` function returns the last confirmed close from the hourly time-frame on all historical bars. When running on *realtime bars* (the bars with the color.aqua background in this example), it returns the symbol's current close value, regardless of confirmation. For more information, see the Historical and realtime behavior section of this page.

ignore_invalid_symbol

The `ignore_invalid_symbol` parameter of `request.*()` functions determines how a function handles invalid data requests, e.g.:

- Using a `request.*()` function with a nonexistent ticker ID as the `symbol/ticker` parameter.
- Using `request.financial()` to retrieve information that does not exist for the specified `symbol` or `period`.
- Using `request.economic()` to request a `field` that does not exist for a `country_code`.

A `request.*()` function call produces a *runtime error* and halts the execution of the script when making an erroneous request if its `ignore_invalid_symbol` parameter is `false`. When this parameter's value is `true`, the function returns na values in such a case instead of raising an error.

This example uses `request.*()` calls within a user-defined function to retrieve data for estimating an instrument's market capitalization (market cap). The user-defined `calcMarketCap()` function calls `request.financial()` to retrieve the total shares outstanding for a symbol and `request.security()` to retrieve a tuple containing the symbol's close price and currency. We've included `ignore_invalid_symbol = true` in both of these `request.*()` calls to prevent runtime errors for invalid requests.

The script displays a formatted string representing the symbol's estimated market cap value and currency in a table on the chart and uses a plot to visualize the `marketCap` history:

```
//@version=6
indicator("`ignore_invalid_symbol` demo", "Market cap estimate", format = format.volume)

//@variable The symbol to request data from.
string symbol = input.symbol("TSX:SHOP", "Symbol")

//@function Estimates the market capitalization of the specified `tickerID` if the data exists.
calcMarketCap(simple string tickerID) =>
    //@variable The quarterly total shares outstanding for the `tickerID`. Returns `na` when the data isn't av
    float tso = request.financial(tickerID, "TOTAL_SHARES_OUTSTANDING", "FQ", ignore_invalid_symbol = true)
    //@variable The `close` price and currency for the `tickerID`. Returns `[na, na]` when the `tickerID` is i
```



Figure 172: image

```
[price, currency] = request.security(
    tickerID, timeframe.period, [close, syminfo.currency], ignore_invalid_symbol = true
)
// Return a tuple containing the market cap estimate and the quote currency.
[tso * price, currency]

//@variable A `table` object with a single cell that displays the `marketCap` and `quoteCurrency`.
var table infoTable = table.new(position.top_right, 1, 1)
// Initialize the table's cell on the first bar.
if barstate.isfirst
    table.cell(infoTable, 0, 0, "", text_color = color.white, text_size = size.huge, bgcolor = color.teal)

// Get the market cap estimate and quote currency for the `symbol`.
[marketCap, quoteCurrency] = calcMarketCap(symbol)

if barstate.islast
    //@variable The formatted text displayed inside the `infoTable`.
    string tableText = str.format("Market cap:\n{0} {1}", str.tostring(marketCap, format.volume), quoteCurrency)
    // Update the `infoTable`.
    table.cell_set_text(infoTable, 0, 0, tableText)

// Plot the `marketCap` value.
plot(marketCap, "Market cap", color.new(color.purple, 60), style = plot.style_area)
```

Note that:

- The `calcMarketCap()` function only returns non-na values on valid instruments with total shares outstanding data, such as the one we selected for this example. It returns na on others that do not have financial data, including forex, crypto, and derivatives.
- Not all issuing companies publish quarterly financial reports. If the issuing company of the `symbol` does not report on a quarterly basis, change the “FQ” value in this script to the company’s minimum reporting period. See the `request.financial()` section for more information.
- We included `format.volume` in the `indicator()` and `str.tostring()` calls to specify that the y-axis of the chart pane represents volume-formatted values and the “string” representation of the `marketCap` value shows as volume-formatted text.
- This script creates a table and initializes its cell on the first chart bar, then updates the cell’s text on the last chart bar. To learn more about working with tables, see the Tables page.

currency

The `currency` parameter of a `request.*()` function allows users to specify the currency of the requested data. When this parameter's value differs from the `syminfo.currency` of the symbol, the function converts the requested values to express them in the specified `currency`. This parameter accepts a built-in variable from the `currency.*` namespace, such as `currency.JPY`, or a "string" representing a valid currency code (e.g., "JPY"). By default, this parameter accepts a "series" argument that can change across executions. If dynamic requests are disabled, it accepts a value with only a "simple" or weaker qualifier.

The conversion rate between the `syminfo.currency` of the requested data and the specified `currency` depends on the *previous daily value* of the corresponding currency pair from the most popular exchange. If no exchange provides the rate directly, the function derives the rate using a spread symbol.

lookahead

The `lookahead` parameter in `request.security()`, `request.dividends()`, `request.splits()`, and `request.earnings()` specifies the lookahead behavior of the function call. Its default value is `barmerge.lookahead_off`.

When requesting data from a higher-timeframe (HTF) context, the `lookahead` value determines whether the `request.*()` function can return values from times *beyond* those of the historical bars it executes on. In other words, the `lookahead` parameter determines whether the requested data may contain *lookahead bias* on historical bars.

When requesting data from a lower-timeframe (LTF) context, the `lookahead` parameter determines whether the function requests values from the first or last *intraday* (LTF bar) of each chart-timeframe bar.

Programmers should exercise extreme caution when using lookahead in their requests, namely when requesting data from higher timeframes. When using `barmerge.lookahead_on` as the `lookahead` value, ensure that it does not compromise the integrity of the script's logic by leaking *future data* into historical chart bars.

The following scenarios are cases where enabling lookahead is acceptable in a `request.*()` call:

- The **expression** in `request.security()` references a series with a *historical offset* (e.g., `close[1]`), which prevents the function from requesting future values that it would **not** have access to on a realtime basis.
- The specified **timeframe** in the call is the same as the chart the script executes on, i.e., `timeframe.period`.
- The function call requests data from an intraday timeframe, i.e., a timeframe smaller than the `timeframe.period`. See this section for more information.

This example demonstrates how the `lookahead` parameter affects the behavior of higher-timeframe data requests and why enabling lookahead in `request.security()` without offsetting the **expression** is misleading. The script calls `request.security()` to get the HTF high price for the current chart's symbol in three different ways and plots the resulting series on the chart for comparison.

The first call uses `barmerge.lookahead_off` (default), and the others use `barmerge.lookahead_on`. However, the third `request.security()` call also *offsets* its **expression** using the history-referencing operator `[]` to avoid leaking future data into the past.

As we see on the chart, the plot of the series requested using `barmerge.lookahead_on` without an offset (fuchsia line) shows final HTF high prices *before* they are actually available on historical bars, whereas the other two calls do not:

```
//@version=6
indicator("`lookahead` demo", overlay = true)

//@variable The timeframe to request the data from.
string timeframe = input.timeframe("30", "Timeframe")

//@variable The requested `high` price from the current symbol on the `timeframe` without lookahead bias.
//      On realtime bars, it returns the current `high` of the `timeframe`.
float lookaheadOff = request.security(syminfo.tickerid, timeframe, high, lookahead = barmerge.lookahead_off)

//@variable The requested `high` price from the current symbol on the `timeframe` with lookahead bias.
//      Returns values that should NOT be accessible yet on historical bars.
float lookaheadOn = request.security(syminfo.tickerid, timeframe, high, lookahead = barmerge.lookahead_on)

//@variable The requested `high` price from the current symbol on the `timeframe` without lookahead bias or re
//      Behaves the same on historical and realtime bars.
float lookaheadOnOffset = request.security(syminfo.tickerid, timeframe, high[1], lookahead = barmerge.lookahead_on)
```



Figure 173: image

```
// Plot the values.
plot(lookaheadOff, "High, no lookahead bias", color.new(color.blue, 40), 5)
plot(lookaheadOn, "High with lookahead bias", color.fuchsia, 3)
plot(lookaheadOnOffset, "High, no lookahead bias or repaint", color.aqua, 3)
// Highlight the background on realtime bars.
bgcolor(barstate.isrealtime ? color.new(color.orange, 60) : na, title = "Realtime bar highlight")
```

Note that:

- The series requested using `barmerge.lookahead_off` has a new historical value at the *end* of each HTF period, and both series requested using `barmerge.lookahead_on` have new historical data at the *start* of each period.
- On realtime bars, the plot of the series without lookahead (blue) and the series with lookahead and no historical offset (fuchsia) show the *same value* (i.e., the HTF period's unconfirmed high price), as no data exists beyond those points to leak into the past. Both of these plots *repaint* their results after restarting the script's executions, as realtime bars become historical bars.
- The series that uses lookahead and a historical offset (aqua) *does not* repaint its values, as it always references the last *confirmed* value from the higher timeframe. See the Avoiding repainting section of this page for more information.

Dynamic requests

By default, unlike all previous Pine Script™ versions, all v6 script's `request.*()` functions are *dynamic*.

In contrast to non-dynamic requests, dynamic requests can:

- Access data from different data feeds using a single `request.*()` instance with “series” arguments.
- Execute within the local scopes of conditional structures, loops, and exported functions.
- Execute nested requests.

Aside from the features listed above, there are insignificant differences in the behavior of dynamic and non-dynamic requests. However, for backward compatibility, programmers can deactivate dynamic requests by specifying `dynamic_requests = false` in the `indicator()`, `strategy()`, or `library()` declaration statement.

“series” arguments Scripts without dynamic requests enabled cannot use “series” arguments for most `request.*()` function parameters, which means the argument values *cannot change*. The only exception is the `expression` parameter in `request.security()`, `request.security_lower_tf()`, and `request.seed()`, which *always* allows “series” values.

In contrast, when a script allows dynamic requests, all `request.*()` function parameters that define parts of the ticker ID or timeframe of a request accept “series” arguments that *can change* with each script execution. In other words, with dynamic requests, it's possible for a single `request.*()` instance to fetch data from *different contexts* in different executions.

Some other optional parameters, such as `ignore_invalid_symbol`, can also accept “series” arguments, allowing additional flexibility in `request.*()` call behaviors.

The following script declares a `symbolSeries` variable that is assigned four different symbol strings in 20-bar cycles, with its value changing after every five bars. The `request.security()` call uses this variable as the `symbol` argument. The script plots the `requestedClose` values, which therefore represent a different symbol’s close prices for each five-bar period.

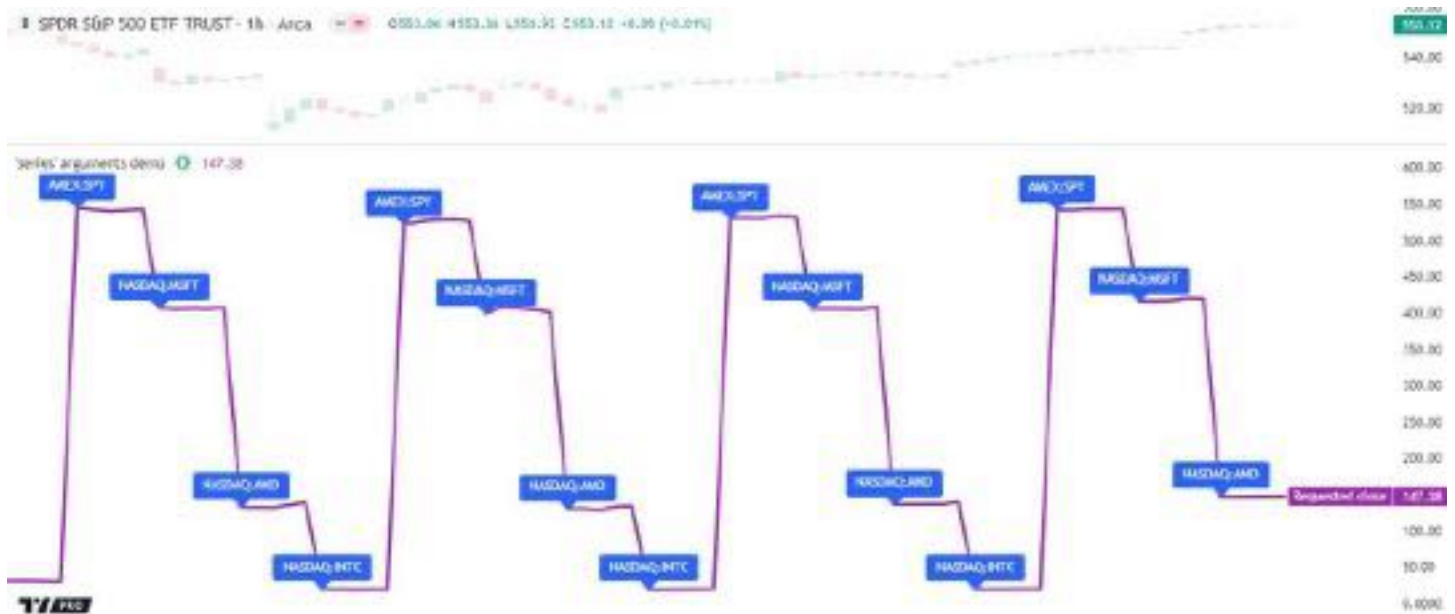


Figure 174: image

```
//@version=6
indicator("'series' arguments demo")

//@variable A "series" that cycles through four different symbol strings. Its value changes every five bars.
string symbolSeries = switch int(bar_index / 5) % 4
    1 => "NASDAQ:MSFT"
    2 => "NASDAQ:AMD"
    3 => "NASDAQ:INTC"
    => "AMEX:SPY"

//@variable The requested `close` value from one of the four `symbolSeries` values on the chart's timeframe.
float requestedClose = request.security(symbolSeries, timeframe.period, close)

// Plot the `requestedClose`.
plot(requestedClose, "Requested close", color.purple, 3)

// Draw a label displaying the requested symbol each time the `symbolSeries` changes.
if symbolSeries != symbolSeries[1]
    label.new(bar_index, requestedClose, symbolSeries, textcolor = color.white)
```

Note that:

- The script draws a label every time the `symbolSeries` changes, to signify which symbol’s data the `requestedClose` currently represents.
- Pine v6 scripts enable dynamic requests by default, allowing this script to use a “series string” `symbol` argument in its `request.security()` call without error. If the dynamic behavior is disabled by including `dynamic_requests = false` in the `indicator()` declaration, then the “series” argument causes a compilation error.

An important limitation is that when using dynamic `request.*()` calls with “series” arguments or within local scopes, scripts must request all required datasets while executing on **historical bars**. All `request.*()` calls on *realtime* bars can retrieve data from the datasets that the script previously accessed on historical bars, but they **cannot** request a new context or evaluate a new expression.

To illustrate this limitation, let’s revisit the above script. Notice that it requests close data for all four symbols on the

chart's timeframe during its historical executions. The external datasets for those four contexts are the **only** ones that any `request.*()` call on realtime bars can access.

Below, we changed the `timeframe` argument in the script's `request.security()` call to specify that it requests `symbolSeries` data from the chart's timeframe on historical bars and the "240" (240 minutes = 4H) timeframe on realtime bars. This version raises a runtime error on the first realtime tick, if it is run on any timeframe other than the 4H timeframe, because it **cannot** access the 4H data feeds without requesting them on historical bars first:

```
//@version=6
indicator("Invalid realtime request demo")

//@variable A "series" that cycles through four different symbol strings. Its value changes every five bars.
string symbolSeries = switch int(bar_index / 5) % 4
    1 => "NASDAQ:MSFT"
    2 => "NASDAQ:AMD"
    3 => "NASDAQ:INTC"
    => "AMEX:SPY"

// Request the `close` of the `symbolSeries` from the chart's timeframe on historical bars and the "240" (4H)
// on realtime bars. Causes a runtime error on the first realtime tick because the script did not previously a
// data from the "240" timeframe on any historical bars.
float requestedClose = request.security(symbolSeries, barstate.isrealtime ? "240" : timeframe.period, close)

// Plot the `requestedClose`.
plot(requestedClose, "Requested close", color.purple, 3)

// Draw a label displaying the requested symbol each time the `symbolSeries` changes.
if symbolSeries != symbolSeries[1]
    label.new(bar_index, requestedClose, symbolSeries, textcolor = color.white)
```

In local scopes When scripts do not allow dynamic requests, all `request.*()` calls execute once on *every* bar or realtime tick, which prevents their use within most local scopes. The only exception is for `request.*()` calls in the scopes of *non-exported* functions and methods, because the Pine Script™ compiler extracts such calls into the *global scope* during translation.

Scripts that allow dynamic requests *do not* restrict the execution of `request.*()` calls to the global scope. They can call `request.*()` functions directly within the scopes of conditional structures and loops, meaning that each `request.*()` instance in the code can activate zero, one, or several times on each script execution.

The following example uses a single `request.security()` instance within a loop to request data from multiple forex data feeds. The script declares an array of `symbols` on the first chart bar, which it iterates through on all bars using a `for...in` loop. Each loop iteration calls `request.security()` to retrieve the volume value for one of the symbols and pushes the result into the `requestedData` array. After the loop terminates, the script calculates the average, maximum, and minimum values from the `requestedData` array and plots those values on the chart:

```
//@version=6
indicator("In local scopes demo", format = format.volume)

//@variable An array of "string" values representing different symbols to request.
var array<string> symbols = array.from(
    "EURUSD", "USDJPY", "GBPUSD", "AUDUSD", "USDCAD", "USDCHF", "NZDUSD", "EURJPY", "GBPJPY", "EURGBP"
)

//@variable An array containing the data retrieved for each requested symbol.
array<float> requestedData = array.new<float>()

// Retrieve `volume` data for each symbol in the `symbols` array and push the results into the `requestedData`
for symbol in symbols
    float data = request.security("OANDA:" + symbol, timeframe.period, volume)
    requestedData.push(data)

// Calculate the average, maximum, and minimum tick volume in the `requestedData`.
float avgVolume = requestedData.avg()
```



Figure 175: image

```
float maxVolume = requestedData.max()
float minVolume = requestedData.min()
```

```
// Plot the `avgVolume`, `maxVolume`, and `minVolume`.
plot(avgVolume, "Average volume", color.gray, 3)
plot(maxVolume, "Highest volume", color.teal, 3)
plot(minVolume, "Lowest volume", color.maroon, 3)
```

Notice that the `expression` argument in the above example (`volume`) is *loop-invariant*, i.e., it does not change on any loop iteration. When using `request.*()` calls within a loop, all parameters defining parts of the requested *context* can accept arguments that depend on variables from the loop's header or mutable variables that change within the loop's local scope. However, a `request.*()` call's evaluated expression **cannot** depend on the values of those variables.

Here, we modified the above script to use the *second form* of the `for...in` loop statement, which creates a tuple containing the index and value of each element in the `symbols` array. The `request.security()` instance in this version uses the index (`i`) in its `expression` argument, resulting in a *compilation error*:

```
//@version=6
indicator("Loop-dependent expression demo", format = format.volume)

//@variable An array of "string" values representing different symbols to request.
var array<string> symbols = array.from(
    "EURUSD", "USDJPY", "GBPUSD", "AUDUSD", "USDCAD", "USDCHF", "NZDUSD", "EURJPY", "GBPJPY", "EURGBP"
)

//@variable An array containing the data retrieved for each requested symbol.
array<float> requestedData = array.new<float>()

// Retrieve `volume` data for each symbol in the `symbols` array, weighted using the element index.
// Causes a compilation error because the `expression` in `request.security()` cannot depend on loop variables
// or mutable variables that change within the loop's scope.
for [i, symbol] in symbols
    float data = request.security("OANDA:" + symbol, timeframe.period, volume * (10 - i))
    requestedData.push(data)

// Calculate the average, maximum, and minimum tick volume in the `requestedData`.
float avgVolume = requestedData.avg()
float maxVolume = requestedData.max()
```

```
float minVolume = requestedData.min()

// Plot the `avgVolume`, `maxVolume`, and `minVolume`.
plot(avgVolume, "Average volume", color.gray, 3)
plot(maxVolume, "Highest volume", color.teal, 3)
plot(minVolume, "Lowest volume", color.maroon, 3)
```

In libraries Libraries with dynamic requests enabled can *export* functions and methods that utilize `request.*()` calls within their local scopes, provided that the evaluated expressions **do not** depend on any exported function parameters.

For example, this simple library exports an `htfPrices()` function that requests a tuple of confirmed open, high, low, and close prices using a specified `tickerID` and `timeframe`. If we publish this library, another script can *import* the function to request higher-timeframe prices without explicitly calling `request.security()`.

```
//@version=6
library("DynamicRequests")

//@function      Requests a tuple containing confirmed HTF OHLC data for a specified `tickerID` and `timeframe`
//@param tickerID The ticker identifier to request data for.
//@param timeframe The timeframe of the requested data.
//@returns       A tuple containing the last confirmed `open`, `high`, `low`, and `close` from the requested
export htfPrices(string tickerID, string timeframe) =>
    if timeframe.in_seconds() >= timeframe.in_seconds(timeframe)
        runtime.error("The `timeframe` argument of `getHTFPrices()` must be higher than the chart's timeframe.")
    request.security(tickerID, timeframe, [open[1], high[1], low[1], close[1]], lookahead = barmerge.lookahead)
```

Note that:

- The tuple that the `request.security()` call includes as the *expression* argument *does not* depend on the `htfPrices()` parameters.
- The `htfPrices()` function raises a custom runtime error when the `timeframe` argument is not higher than the chart's timeframe. See the higher timeframes section for more information.
- The `request.security()` call uses `barmerge.lookahead_on` and offsets each item in the tuple by one bar. This is the only recommended method to avoid repainting.

Nested requests Scripts can use dynamic requests to execute *nested requests*, i.e., `request.*()` calls that dynamically evaluate other `request.*()` calls that their *expression* arguments depend on.

When a `request.security()` or `request.security_lower_tf()` call uses an empty string or `syminfo.tickerid` for its *symbol* argument, or if it uses an empty string or `timeframe.period` for the *timeframe* argument, the requested ticker ID or timeframe *depends* on the context where the call executes. This context is normally the ticker ID or timeframe of the chart that the script is running on. However, if such a `request.security()` or `request.security_lower_tf()` function call is evaluated by another `request.*()` call, the nested request *inherits* that `request.*()` call's ticker ID or timeframe information.

For example, the script below contains two `request.security()` calls and uses Pine Logs to display their results. The first call uses empty strings as its *symbol* and *timeframe* arguments, meaning that the requested context depends on where the call executes. It evaluates a concatenated string containing the call's requested ticker ID and timeframe, and the script assigns its result to the `info1` variable.

The second call requests data for a specific *symbol* and *timeframe* using the `info1` variable as its *expression* argument. Since the `info1` variable depends on the first `request.security()` call, the second call evaluates the first call *within* its own context. Therefore, the first call adopts the second call's ticker ID and timeframe while executing within that context, resulting in a different returned value:

```
//@version=6
indicator("Nested requests demo")

//@variable A concatenated string containing the current `syminfo.tickerid` and `timeframe.period`.
string info1 = request.security("", "", syminfo.tickerid + "_" + timeframe.period)
//@variable A concatenated string representing the `info1` value calculated within the "NASDAQ:AAPL, 240" context
//      This call evaluates the call on line 5 within its context to determine its result because the script
//      allows dynamic requests.
string info2 = request.security("NASDAQ:AAPL", "240", info1)
```



Figure 176: image

```
// Log the results from both calls in the Pine Logs pane on the last historical bar.
if barstate.islastconfirmedhistory
    log.info("First request: {0}", info1)
    log.info("Second request: {0}", info2)
```

This script allows the execution of the first `request.security()` call within the context of the second call because Pine v6 scripts enable dynamic `request.*()` calls by default. We can disable this behavior by including `dynamic_requests = false` in the `indicator()` declaration statement. Without dynamic requests enabled, the script evaluates each call *independently*, passing the first call's calculated value directly into the second call rather than executing the first call within the second context. Consequently, the second call's returned value is the *same* as the first call's value, as we see below:



Figure 177: image

```
//@version=6
indicator("Nested requests demo", dynamic_requests = false)

//@variable A concatenated string containing the current `syminfo.tickerid` and `timeframe.period`.
string info1 = request.security("", "", syminfo.tickerid + "_" + timeframe.period)
//@variable The same value as `info1`. This call does not evaluate the call on line 5 because dynamic request
// allowed. Instead, it only uses the value of `info1`, meaning its result does not change.
string info2 = request.security("NASDAQ:AAPL", "240", info1)

// Log the results from both calls in the Pine Logs pane on the last historical bar.
if barstate.islastconfirmedhistory
    log.info("First request: {0}", info1)
    log.info("Second request: {0}", info2)
```

Data feeds

TradingView's data providers supply different data feeds that scripts can access to retrieve information about an instrument, including:

- Intraday historical data (for timeframes < 1D)
- End-of-day (EOD) historical data (for timeframes >= 1D)
- Realtime data (which may be delayed, depending on your account type and extra data services)
- Extended hours data

Not all of these data feed types exist for every instrument. For example, the symbol “BNC:BLX” only has EOD data available.

For some instruments with intraday and EOD historical feeds, volume data may not be the same since some trades (block trades, OTC trades, etc.) may only be available at the *end* of the trading day. Consequently, the EOD feed will include this volume data, but the intraday feed will not. Differences between EOD and intraday volume feeds are almost nonexistent for instruments such as cryptocurrencies, but they are commonplace in stocks.

Slight price discrepancies may also occur between EOD and intraday feeds. For example, the high value on one EOD bar may not match any intraday high values supplied by the data provider for that day.

Another distinction between EOD and intraday data feeds is that EOD feeds do not contain information from *extended hours*.

When retrieving information on realtime bars with `request.*()` functions, it's important to note that historical and realtime data reported for an instrument often rely on *different* data feeds. A broker/exchange may retroactively modify values reported on realtime bars, which the data will only reflect after refreshing the chart or restarting the script.

Another important consideration is that the chart's data feeds and feeds requested from providers by the script are managed by *independent*, concurrent processes. Consequently, in some *rare* cases, it's possible for races to occur where requested results temporarily fall out of synch with the chart on a realtime bar, which a script retroactively adjusts after restarting its executions.

These points may account for variations in the values retrieved by `request.*()` functions when requesting data from other contexts. They may also result in discrepancies between data received on realtime bars and historical bars. There are no steadfast rules about the variations one may encounter in their requested data feeds.

When using data feeds requested from other contexts, it's also crucial to consider the *time axis* differences between the chart the script executes on and the requested feeds since `request.*()` functions adapt the returned series to the chart's time axis. For example, requesting “BTCUSD” data on the “SPY” chart with `request.security()` will only show new values when the “SPY” chart has new data as well. Since “SPY” is not a 24-hour symbol, the “BTCUSD” data returned will contain gaps that are otherwise not present when viewing its chart directly.

`request.security()`

The `request.security()` function allows scripts to request data from other contexts than the chart the script executes on, such as:

- Other symbols, including spread symbols
- Other timeframes (see our User Manual's page on Timeframes to learn about timeframe specifications in Pine Script™)
- Custom contexts, including alternative sessions, price adjustments, chart types, etc. using `ticker.*()` functions

This is the function's signature:

```
request.security(symbol, timeframe, expression, gaps, lookahead, ignore_invalid_symbol, currency, calc_bars_co
```

The `symbol` value is the ticker identifier representing the symbol to fetch data from. This parameter accepts values in any of the following formats:

- A “string” representing a symbol (e.g., “IBM” or “EURUSD”) or an “*Exchange:Symbol*” pair (e.g., “NYSE:IBM” or “OANDA:EURUSD”). When the value does not contain an exchange prefix, the function selects the exchange automatically. We recommend specifying the exchange prefix when possible for consistent results. Users can also pass an empty string to this parameter, which prompts the function to use the current chart's symbol.
- A “string” representing a spread symbol (e.g., “AMD/INTC”). Note that “Bar Replay” mode does not work with these symbols.
- The `syminfo.ticker` or `syminfo.tickerid` built-in variables, which return the symbol or the “Exchange:Symbol” pair that the current chart references. We recommend using `syminfo.tickerid` to avoid ambiguity unless the exchange information does not matter in the data request. For more information on `syminfo.*` variables, see this section of our Chart information page.